How to Get P-Values Less Than 0.05 in Biological Research

Second Edition

Michael Walker

Table of contents

Preface

A note on what this book is and isn't

The title of this book is deliberately provocative, and I want to be direct about what it means and what it doesn't. Every method described here is about choosing the most appropriate, most powerful statistical analysis for your data and your experimental design. These are decisions you make before you look at your results. None of them are about running analysis after analysis until one happens to cross the $p < 0.05$ line, then reporting only that one. That practice has a name: data dredging, or "p-hacking." It produces false positives, wastes other scientists' time trying to replicate effects that were never real, and is a problem I take seriously enough to devote two full chapters to later in this book (see "The wrong way and the right way to get $p < 0.05$" and "Is $p < 0.05$ the right goal?").

Websites, PDF version of the book, and R code from the book

The book is available at these two websites.

https://mwalkerstatistics.com

This website has a free PDF version of the book. It also has all the R code in the book in an R script file that you can download.

https://stats001.github.io/pvalues/

This website is the github repository for the book.

On the website, click here for the free PDF version of the book:

Download the PDF version of the book

On the website, click here for the R code in the book in an R script file that you can download:

Download the R code from the book

The book will also be available through Amazon Kindle, in Kindle Unlimited.

Companion book: A Gentle Introduction to Statistics for Biological Research

In *A Gentle Introduction to Statistics for Biological Research*, I introduce and explain the methods that I recommend in *How to Get P-Values Less Than 0.05 in Biological Research*.

The website mwalkerstatistics.com has a free PDF version of the companion book.

If you have no experience with statistics, I suggest that you start with the companion book. If you have some exposure to statistics, including standard deviation and t-tests, you can begin reading *How to Get P-Values Less Than 0.05 …*, and refer to *A Gentle Introduction …* when you need or would like more explanation of a particular concept or method.

R packages used in the book

These are R packages used in the examples. It may save some time if you install these now.

install.packages(c("ggplot2", "Rfit", "RobStatTM", "car", "vcdExtra", "tibble", "dplyr", "tidyverse", "survival", "survminer", "coin", "rankFD"))

Dedication

To Kai Kai, with hope that biological research will make the world a better place for her and all children. To my family for their love and support.

Acknowledgements

Thank you to my teachers and advisors, who encouraged exploration and showed their own love of science and learning. Thank you to my colleagues and students who have asked the questions that made me think about and better understand statistics. Thank you to my family for supporting this work over many years. Thank you to the reviewers who have made this book better. Whatever is good in this book has come from these friends and colleagues. The errors, of course, are my responsibility, and show that I still have things to learn.

About the author

Michael Walker received his Ph.D. from Stanford University and taught statistics and clinical trial design at Stanford and UC San Diego for 25 years. He is a statistics consultant for pharmaceutical and biotechnology companies. His consulting work has led to FDA approval for new drugs, medical devices, and diagnostic tests. He has published in the New England Journal of Medicine and in Lancet, among over 70 publications. He provides statistics consulting for venture capital companies to evaluate investments in life sciences.

Copyright

of merchantability or fitness for a particular purpose. No liability is assumed for losses or damages due to the information provided.

1 Avoid t-tests: Control for variables using multiple regression or ANOVA

If you want $p < 0.05$ with the minimum sample size and minimum effort, use multiple regression or analysis of variance (ANOVA) instead of t-tests. T-tests only let you test the effect of one treatment variable. This is a problem, because often more than one variable affects the response. The added variability in the response caused by those other variables makes it harder to detect the effect of the one variable you are interested in. T-tests do not let you control for the other variables that affect the response.

Think of it as a signal to noise problem: if you are in a noisy room, it is hard to hear what a person is saying. That is the t-test version. Multiple regression and ANOVA remove the noise, and let you detect the signal.

The following short story illustrates why you should avoid t-tests. It is based on actual events.

1.1 Short story: Of mice and marriage

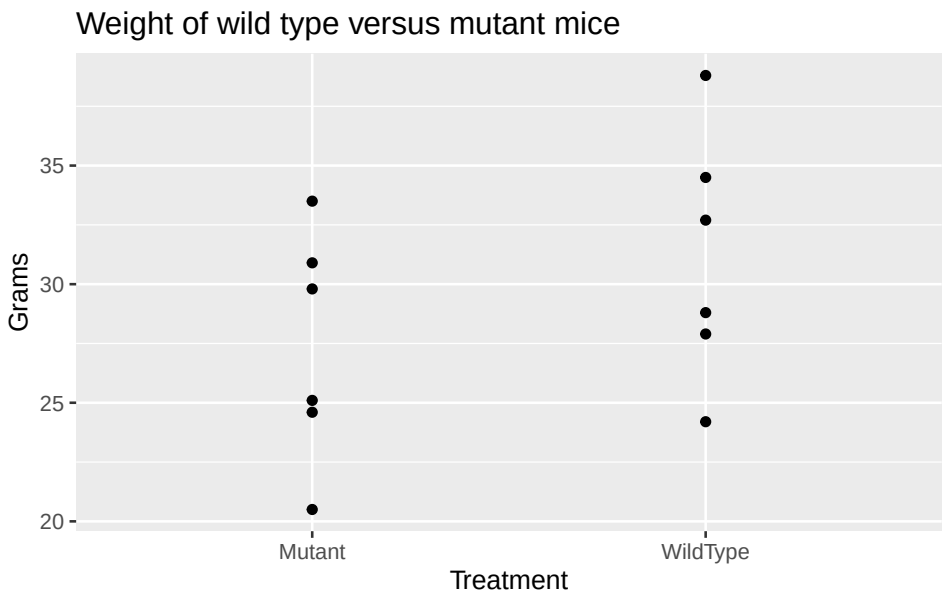
A post-doc, whom we'll call John, was having problems with his experiment. John was studying mice in which he genetically modified a gene he thought might be related to weight. For his experiment, he created knock-out mutant mice that lacked the gene. Then he measured the difference in weight between 24-week-old wild type mice and 24-week-old mutant mice.

Creating the knock-out mice was difficult and time consuming. After almost a year working on his post-doc, his data looked like this.

```
library(tidyverse) # For graphs and data manipulation

mutation.weight.data = tibble::tribble(
~Treatment, ~Sex, ~Grams,
  "WildType", "Female", 24.2,
  "WildType", "Female", 27.9,
```

```
  "WildType", "Female", 28.8,
  "WildType", "Male", 32.7,
  "WildType", "Male", 34.5,
  "WildType", "Male", 38.8,
  "Mutant", "Female", 20.5,
  "Mutant", "Female", 24.6,
  "Mutant", "Female", 25.1,
  "Mutant", "Male", 29.8,
  "Mutant", "Male", 30.9,
  "Mutant", "Male", 33.5
)
```

```
# Create the plot "Weight of wild type versus mutant mice".
ggplot(mutation.weight.data, aes(y=Grams, x=Treatment)) +
geom_point() +
  ggtitle("Weight of wild type versus mutant mice")
```



John tried a t-test for the difference in weight between the mutations and wild type. Here is the R code to run the t-test. The notation "Grams ~ Treatment" is the formula that specifies Grams is the response (y) variable and Treatment is the explanatory (x) variable.

```
t.test(Grams ~ Treatment, var.equal=TRUE, data =
mutation.weight.data)
```

```
    Two Sample t-test

data:  Grams by Treatment
t = -1.2926, df = 10, p-value = 0.2252
alternative hypothesis: true difference in means between group
Mutant and group WildType is not equal to 0
95 percent confidence interval:
 -10.214099   2.714099
sample estimates:
  mean in group Mutant mean in group WildType
                 27.40                  31.15
```

The p-value is not significant, p = 0.2252.

Equivalently, you can do the t-test analysis using the R function lm(), which performs a linear regression. Notice the same formula, "Grams ~ Treatment". In the lm output, the p-value of 0.225 is in the row labeled "TreatmentWildType" in the column labeled "Pr(>|t|).

```
summary(lm(Grams ~ Treatment, data = mutation.weight.data))
```

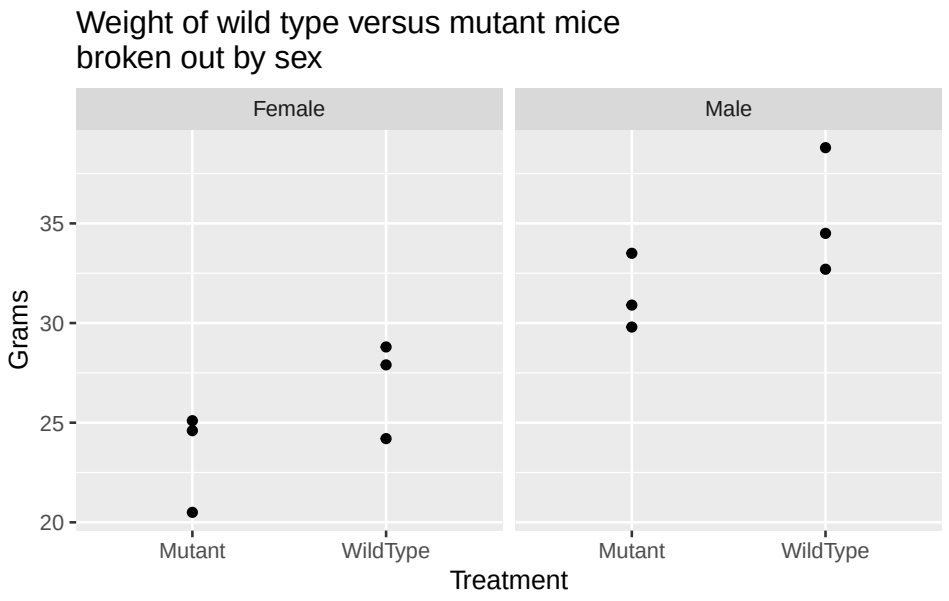
```
Call:
lm(formula = Grams ~ Treatment, data = mutation.weight.data)

Residuals:
   Min     1Q Median     3Q    Max
-6.950 -2.913 -0.375  3.388  7.650

Coefficients:
                  Estimate Std. Error t value Pr(>|t|)
(Intercept)         27.400      2.051  13.357 1.06e-07 ***
TreatmentWildType    3.750      2.901   1.293    0.225
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 5.025 on 10 degrees of freedom
Multiple R-squared:  0.1432,    Adjusted R-squared:  0.05748
F-statistic: 1.671 on 1 and 10 DF,  p-value: 0.2252
```

The t-test and the linear regression both indicate that treatment is not significant, p = 0.2252. We get the same p-value using either the t.test or the lm function.

John was afraid that his academic research career might be coming to an end. Without positive results from his experiment, he couldn't publish and couldn't get a faculty position.

But John believed there was a difference between the mutation and wild type mice. When he plotted his data separately for males and females, he saw a big difference between males and females.

```
# Create the plot "Weight of wild type versus mutant mice
broken out by sex".

ggplot(mutation.weight.data, aes(y=Grams, x=Treatment)) +
geom_point() +
  facet_wrap(~Sex) +
  ggtitle("Weight of wild type versus mutant mice  \nbroken
  out by sex")
```



Sex was a second variable, besides treatment, that affected the weight.

John decided to try a separate t-test for mutant versus wild type for each sex. We'll use the lm function to do the t-tests.

Here is the t-test for males, giving p = 0.137.

```
summary(lm(Grams ~ Treatment,
data=subset(mutation.weight.data, Sex=="Male")))
```

```
Call:
lm(formula = Grams ~ Treatment, data =
subset(mutation.weight.data,
    Sex == "Male"))

Residuals:
      1       2       3       4       5       6
-2.6333 -0.8333  3.4667 -1.6000 -0.5000  2.1000

Coefficients:
                 Estimate Std. Error t value Pr(>|t|)
(Intercept)        31.400      1.496  20.985 3.05e-05 ***
TreatmentWildType   3.933      2.116   1.859    0.137
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 2.592 on 4 degrees of freedom
Multiple R-squared:  0.4635,    Adjusted R-squared:  0.3293
F-statistic: 3.455 on 1 and 4 DF,  p-value: 0.1366
```

Here is the t-test for females, giving p = 0.153.

```
summary(lm(Grams ~ Treatment,
data=subset(mutation.weight.data, Sex=="Female")))
```

```
Call:
lm(formula = Grams ~ Treatment, data =
subset(mutation.weight.data,
    Sex == "Female"))

Residuals:
      1       2       3       4       5       6
-2.7667  0.9333  1.8333 -2.9000  1.2000  1.7000

Coefficients:
```

```
                  Estimate Std. Error t value Pr(>|t|)
(Intercept)         23.400      1.433   16.33 8.22e-05 ***
TreatmentWildType    3.567      2.026    1.76    0.153
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 2.481 on 4 degrees of freedom
Multiple R-squared:  0.4366,    Adjusted R-squared:  0.2957
F-statistic: 3.099 on 1 and 4 DF,  p-value: 0.1531
```

His t-test p-values were p = 0.137 for the males and p = 0.153 for the females. Still not a significant result. He didn't want to go back and start over, using only males or only females, as that would take at least another 6 months. He wanted to use the data he already had.

At this point, a colleague in his lab suggested that John talk to me.

John showed me his results, and the problem with the t-tests. I told John that, instead of t-tests, we should use a two-way analysis of variance (ANOVA). John wanted to compare two treatment groups (mutation vs. wild type). But sex (male vs. female) was an additional important source of variability in the response.

Two-way ANOVA is a way to control for the differences in weight due to sex, while testing for the effect of treatment group. John had used a t-test for treatment group, which doesn't remove the unexplained variability due to sex, so he got a non-significant result. John and I agreed to do a two-way ANOVA, including treatment and sex as the explanatory variables.

We performed the two-way analysis of variance using the same lm() function that we used for the t-test. Notice that the only change in the R code is the addition of "Sex" in the formula, "Grams ~ Sex + Treatment". Here are the results for the two-way ANOVA, which include both Sex and Treatment as explanatory variables.

```
# Do the two-way ANOVA.
# Only change: add Grams ~ Sex + Treatment

summary(lm(Grams ~ Sex + Treatment, data=
mutation.weight.data))
```

```

Call:
```

```
lm(formula = Grams ~ Sex + Treatment, data =
mutation.weight.data)

Residuals:
   Min     1Q Median     3Q    Max
-2.858 -1.904  0.125  1.754  3.558

Coefficients:
                  Estimate Std. Error t value Pr(>|t|)
(Intercept)         23.308      1.197  19.470 1.15e-08 ***
SexMale              8.183      1.382   5.920 0.000224 ***
TreatmentWildType    3.750      1.382   2.713 0.023889 *
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 2.394 on 9 degrees of freedom
Multiple R-squared:  0.8249,    Adjusted R-squared:  0.786
F-statistic:  21.2 on 2 and 9 DF,  p-value: 0.0003932
```

The two-way ANOVA showed that the difference between males and females is significant ($p = 0.0002$), confirming that sex was a major source of otherwise unexplained variability in the response. More important, the two-way ANOVA also showed that Treatment was significant, with $p = 0.0239$. John was ecstatic. The mice were pleased. His wife was happy. His experiment was a success, he could publish, his academic career was back on track. All thanks to using two-way analysis of variance to control for the unexplained variability that the t-test failed to control.

If other variables (besides treatment) affect the response, then we want to include those variables in our analysis. Use multiple regression or multi-way ANOVA to control for the effects of factors that cause unexplained variance.

The t-test is the most commonly used hypothesis test. It does not control for variables other than treatment that affect the response. The t-test minimizes the power to detect treatment effects. The t-test maximizes the sample size required to make discoveries.

1.2 Example: Get a better p-value by controlling for sources of variability in an experiment.

In this example, we'll use two-way ANOVA to get a better p-value by controlling for an undesirable source of variability in an experiment. Our specific example will be controlling for variability across the columns in a 96 well plate, but the concept, and the method, apply more generally to other sources of variability.

If you have run an experiment using a 96-well plate, you likely know that there can be substantial unexplained variation across the rows, columns, and edges. These can be caused by a plate that doesn't sit flat on the heating block, a cover that doesn't fit well and allows evaporation, problems with pipettes becoming partially blocked, or other sources.

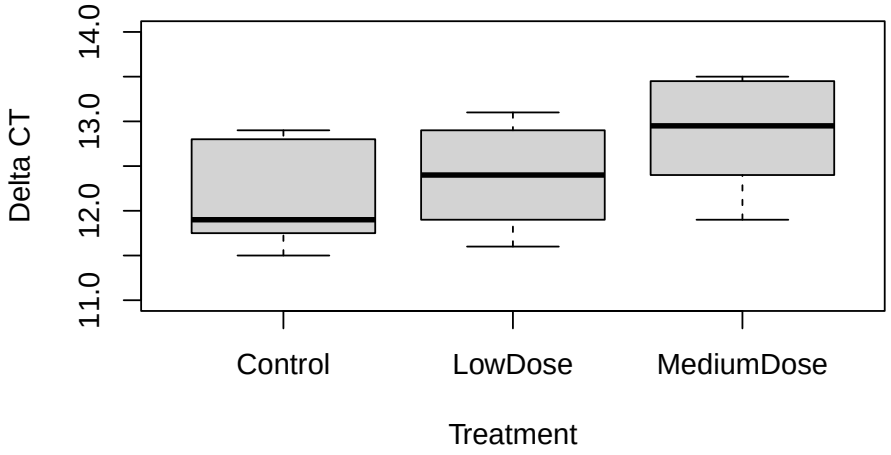
Suppose you have data from experiment where you want to know if Treatment (control, low dose, medium dose) affects gene expression. Expression is measured as Delta CT from a real-time polymerase chain reaction (PCR) analysis. Here is the data set.

```
pcr.data = tibble::tribble(
  ~Treatment,    ~Column, ~DeltaCT,
  "Control",     "A",     11.5,
  "LowDose",     "A",     11.6,
  "MediumDose",  "A",     11.9,
  "Control",     "B",     11.6,
  "LowDose",     "B",     11.9,
  "MediumDose",  "B",     12.0,
  "Control",     "C",     11.9,
  "LowDose",     "C",     12.3,
  "MediumDose",  "C",     12.8,
  "Control",     "D",     11.9,
  "LowDose",     "D",     12.5,
  "MediumDose",  "D",     13.5,
  "Control",     "E",     12.7,
  "LowDose",     "E",     11.9,
  "MediumDose",  "E",     12.9,
  "Control",     "F",     11.9,
  "LowDose",     "F",     12.8,
  "MediumDose",  "F",     13.0,
  "Control",     "G",     12.9,
  "LowDose",     "G",     13.1,
```

```
  "MediumDose",  "G",     13.5,
  "Control",     "H",     12.9,
  "LowDose",     "H",     13.0,
  "MediumDose",  "H",     13.4
)
```

Here is a plot showing Delta CT by Treatment.

```
with(pcr.data, boxplot(DeltaCT ~ Treatment, xlab="Treatment",
ylab = "Delta CT", ylim=c(11,14)))
```



We'll start with a one-way ANOVA that only considers the effect of treatment (as does the t-test), ignoring the effect of column. Here is the R code and the output Analysis of Variance table.

```
# Do one-way ANOVA for DeltaCT ~ Treatment
lm.pcr.1factor = lm(DeltaCT ~ Treatment, data= pcr.data)
anova(lm.pcr.1factor)
```

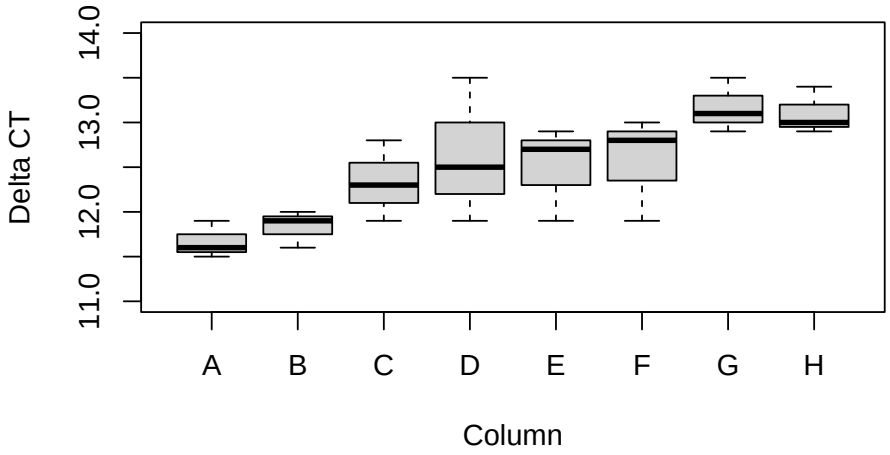
```
Analysis of Variance Table

Response: DeltaCT
          Df Sum Sq Mean Sq F value  Pr(>F)
Treatment  2 2.1225 1.06125  3.0519 0.06863 .
Residuals 21 7.3025 0.34774
```

```
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

The p-value is in the column in the table labeled "Pr(>F). The row labeled "Treatment" gives the p-value for treatment, p = 0.0686. Using a one-way analysis of variance that ignores column, the treatment does not reach statistical significance.

However, if we plot the DeltaCT values for each column, we see that there is a great deal of otherwise unexplained variability due to column.

```
with(pcr.data,
  boxplot(DeltaCT ~ Column, xlab="Column", ylab = "Delta CT",
  ylim=c(11,14)))
```



The graph shows considerable variability in the DeltaCT due to the Column effect. Column should be included as a covariate in the analysis.

Here is the two-way analysis of variance when we include Column as a factor. Including the Column in the model reduces the unexplained variability in the DeltaCT.

```
# Do two-way ANOVA, controlling for Column effect
lm.pcr.2factor = lm(DeltaCT ~ Treatment + Column , data=
pcr.data)
anova(lm.pcr.2factor)
```

```
Analysis of Variance Table

Response: DeltaCT
          Df Sum Sq Mean Sq F value    Pr(>F)
Treatment  2 2.1225 1.06125 11.1084 0.0012898 **
Column     7 5.9650 0.85214  8.9196 0.0003026 ***
Residuals 14 1.3375 0.09554
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

The ANOVA controlling for column effect shows that Treatment is significant ($p = 0.00129$). By including the Column as an explanatory variable that influenced the DeltaCT we are better able to determine if the treatment is effective. If we just do a one-way ANOVA for Treatment, ignoring the variability due to Column, the result is not significant.

Conclusion: Instead of t-tests, use multiple regression and multi-way ANOVA to include more variables.

These examples show that you can get better p-values by using multiple regression and two-way ANOVA to control for variables that affect the response.

In every experiment I have seen, there are other variables, similar to column, that add to the unexplained variability. The other variables might be plate, instrument, operator, rows or columns on a plate, the day, month, or year each specimen was collected, the order in which the specimens were analyzed, the source (clinic, lab, vendor) that the specimens came from, the age of the animal, the sex, the litter, the litter size, and so on. If you test such variables to see if they affect the response, and include them in your analysis, you are more likely to get $p < 0.05$.

1.3 Identify important variables quickly using fractional factorial designs:

What if you don't know which variables are having a big effect on your response variables? How can you quickly find out which are most important? A good way to identify the important variables is to use fractional factorial designs. The chapter "Rescue failed experiments using fractional factorial design" describes efficient ways to design such experiments to look at many factors with minimum effort.

In the companion book, "A Gentle Introduction to Statistics for Biological Research", I introduce and provide in depth explanations and examples of the concepts and methods described in this chapter. A free PDF of the companion book is available on the website, mwalkerstatistics.com.

1.4 P-value, null hypothesis, and alternative hypothesis

You will likely encounter the terms p-value, null hypothesis, and alternative hypothesis as you read and use statistics. I'll give brief explanations here.

Suppose we are planning an experiment to test if there is any difference between two treatments, which we'll call A and B. Before we begin the experiment, we assume that there is no difference between treatment A and treatment B. This assumption is the null hypothesis: there is no difference between treatment A and treatment B in their effect on the response. We perform the experiment to gather evidence that there is a difference (so that we can reject the null hypothesis). For example, in the experiment on mutations and wild type introduced above, the null hypothesis was that there is no difference between mutations and wild type in their effect on weight.

The alternative hypothesis is that there is a difference between treatment A and treatment B. For example, in the experiment above, the alternative hypothesis was that there is a difference between mutations and wild type in their effect on weight.

A p-value is the probability of the observed result, or a more extreme result, if the null hypothesis is true. For example, in the experiment above, the p-value is the probability of observing as large or a larger difference in weight between mutations and wild type (due to random sampling) if in fact there is no difference in their effect on weight.

A high p-value does not prove the null hypothesis is true. There can be many reasons for a p-value greater than 0.05, as described in the chapter "What if you still get $p > 0.05$?".

The chapter "What is a p-value?" in the companion book, "A Gentle Introduction to Statistics for Biological Research", has simulations that illustrate these concepts.

1.5 R code and data sets

Text files of the R code in the book are available on the website, mwalker-statistics.com.

2 Calculate power and sample size if you want $p < 0.05$

Before you begin an experiment, would you like to know your chances of getting $p < 0.05$?

Would you like to know, before you began, if you only have a 10% chance of getting $p < 0.05$? Would you think about changing the experiment? What if you had only a 50% chance?

The probability that you will get $p < 0.05$, assuming the treatment works, is the power of your experiment. Typically, anything less than 70% is considered low power. Low power makes it hard to get $p < 0.05$. Institutional Review Boards and funding agencies usually want to see power of 80% or higher before they approve or fund a project.

In this chapter, we'll look at what determines power and sample size. We'll see easy ways to calculate power using software. If you know how to calculate the power and sample size for your proposed experiment, you can design better experiments. You can increase the probability that you will get $p < 0.05$.

It can be difficult to get the necessary information to calculate power for an analysis that includes more than one explanatory variable. For that reason, we often do the power calculation for a t-test, with the expectation that we will get a conservative estimate of the required sample size and power. But the actual study will use more powerful methods than t-tests.

2.1 What determines your power and sample size?

Let's look at a few scenarios to get an intuitive idea of what affects power and sample size.

2.1.1 Scenario 1: A good situation

Suppose we have Group A with mean 8 and Group B with mean 12, and the standard deviation within group is sd = 2. The difference between the means is delta = 12 – 8 = 4.

Because the difference between the means delta = 4 is large compared to the standard deviation within group of sd = 2, this scenario will give us good power and/or will require a relatively smaller sample size.

Here is R code to calculate sample size for this scenario, using the function power.t.test. We'll see more on how to calculate power and sample size shortly.

Assumptions:

- delta = difference between group means (effect size) = 4
- sd = standard deviation within group = 2
- sig.level = desired alpha = 0.05
- power = 0.8

```
power.t.test(delta = 4, sd = 2, sig.level = 0.05, power = 0.8)
```

```
     Two-sample t test power calculation

              n = 5.090008
          delta = 4
             sd = 2
      sig.level = 0.05
          power = 0.8
    alternative = two.sided

NOTE: n is number in *each* group
```

In the output from power.t.test, the words "Two-sample" indicate the two groups. So, the output indicates that, for the given assumptions, the required sample size is n = 5.09 per group. Rounding up to 6 per group, we need a total sample size of 2*6=12.

2.1.2 Scenario 2: An OK situation

In this scenario, the standard deviation within each group is still small (sd=2), but the means of the two groups are not so far apart (delta = 10 – 8 = 2). Compared to scenario 1, we will need a bigger sample size, or, for a given sample size, we will have less power to show a significant difference between the two groups.

Here is the R code to calculate sample size for this scenario.

Assumptions:

- delta = difference between group means (effect size) = 2
- sd = standard deviation within group = 2
- sig.level = desired alpha = 0.05
- power = 0.8

```
power.t.test(delta = 2, sd = 2, sig.level = 0.05, power = 0.8)
```

```
     Two-sample t test power calculation

              n = 16.71477
          delta = 2
             sd = 2
      sig.level = 0.05
          power = 0.8
    alternative = two.sided

NOTE: n is number in *each* group
```

The output from power.t.test indicates that, for the given assumptions, the required sample size is n = 16.7 per group. Rounding up to 17 per group, we need a total sample size of 2*17=34.

Because the difference between the means is smaller, the sample size is larger than in scenario 1.

2.1.3 Scenario 3: An unfavorable situation

For this scenario, the difference between the group means is delta = 10 – 8 = 2. The standard deviation within each group is sd = 3, which is 50% larger than the standard deviation for scenarios 1 and 2.

This is an unfavorable situation: a small difference between the means (small effect size) and large standard deviation within each group. We expect that this scenario will require a larger sample size to detect the difference between the groups with $p < 0.05$.

Here is the R code to calculate sample size for this scenario.

Assumptions:

- delta = difference between group means (effect size) = 2
- sd = standard deviation within group = 3
- sig.level = desired alpha = 0.05
- power = 0.8

```
power.t.test(delta = 2, sd = 3, sig.level = 0.05, power = 0.8)
```

```
     Two-sample t test power calculation

              n = 36.3058
          delta = 2
             sd = 3
      sig.level = 0.05
          power = 0.8
    alternative = two.sided

NOTE: n is number in *each* group
```

The output from power.t.test indicates that, for the given assumptions, the required sample size is n = 36.3 per group. Rounding up to 37 per group, we need a total sample size of 2*37=74.

Scenario 4: A wasteful, overpowered experiment with too large a sample size.

Is it ever possible to have too large a sample size? Well, if you can run two experiments and get $p < 0.05$ using the same resources that your competitor uses to run one experiment, who is going to get results faster? Who will publish first? Who will get to market first?

Usually, we want the power for our experiments to be 80% or higher. Power of 90% or 95% is reasonable. But much more power than that can be wasted.

Suppose that colleagues in your lab tell you that, traditionally, they use a sample size of 25 specimens per group. Nobody knows why: it is just

tradition. You examine results from previous experiments in your lab and see that the typical standard deviation within group is 4. The effect size you want to detect is 6, with alpha = 0.05. We can use the power.t.test function to calculate the power for n = 25 subjects per group:

Assumptions:

- delta = difference between group means (effect size) = 6
- sd = standard deviation within group = 4
- sig.level = desired alpha = 0.05
- sample per group = n = 25

```
power.t.test(delta = 6, sd = 4, sig.level = 0.05, n=25)
```

```
     Two-sample t test power calculation

              n = 25
          delta = 6
             sd = 4
      sig.level = 0.05
          power = 0.9993912
    alternative = two.sided

NOTE: n is number in *each* group
```

The output from power.t.test indicates that, for the given assumptions, a sample size of 25 per group gives power = 0.99939. Now, perhaps you do want that much power to be certain of getting a confirmatory result, in which case n=25 may be appropriate. However, you could have gotten 95% power with only 13 per group, instead of 25 per group:

```
power.t.test(delta = 6, sd = 4, sig.level = 0.05, power =
0.95)
```

```
     Two-sample t test power calculation

              n = 12.59872
          delta = 6
             sd = 4
      sig.level = 0.05
          power = 0.95
```

```
    alternative = two.sided

NOTE: n is number in *each* group
```

So, you could run two experiments with 13 per group and have 95% power to get $p < 0.05$ for each experiment. Or you can run a single experiment with 25 per group and have power greater than 0.99 to get $p < 0.05$. It's worth thinking about, and worth calculating the sample size and power you really need to be successful.

These scenarios illustrate that power for a t-test depends on 4 things:

- Effect size: what is the difference between the means of the two treatment groups?
- Standard deviation: what is the standard deviation within each treatment group?
- Sample size: how many subjects will be in each group?
- Alpha: Do we want a p-value less than alpha = 0.05 or 0.01?

Notice in scenario 3 that the within-group standard deviation has a particularly large effect on power. This is the reason for the chapter "Increase sample size last. Reduce unexplained variability first". It is also the reason why you need to calculate power and sample size before you begin your experiment. If you don't, you will likely either be underpowered and not get $p < 0.05$, or your experiment will be overpowered and waste resources.

2.2 Software for power and sample size

There are several quite good commercial software packages for power and sample size calculations.

The review by Serdar et al, "Sample size, power and effect size revisited: simplified and practical approaches in pre-clinical, clinical and laboratory studies" is very helpful (Serdar et al. 2021). Table 2 in the paper has notes on software performance, user friendliness, and if the software is free.

Of the commercial power and sample size software packages, I use NCSS PASS the most. The NCSS PASS documentation (https://www.ncss.com/software/pass/pass-documentation/) is particularly good at explaining the concepts and giving very clear, easy to understand examples. Even if you don't buy the software, it is worthwhile looking at their documentation.

As shown above, R has a few basic functions to calculate power and sample size. These functions are power.t.test, power.anova.test and power.prop.test. The R function power.anova.test calculates power or sample size for one-way ANOVA. See the companion book for examples.

The free R package TrialSize accompanies the textbook "Sample Size Calculation in Clinical Research" by Shein-Chung Chow et al (Chow et al. 2017). The textbook gives very clear explanations of the concepts.

2.3 Examples of power and sample size calculations using the base R functions

Here are a few examples using the base R functions for power and sample size calculations.

Sample size for a two-sample t-test.

- difference in means, delta = 0.5
- standard deviation, sd = 0.5
- alpha, sig.level = 0.01
- desired power, power = 0.9

```
power.t.test(delta = 0.5, sd = 0.5, sig.level = 0.01, power =
0.9)
```

```
     Two-sample t test power calculation

              n = 31.46245
          delta = 0.5
             sd = 0.5
      sig.level = 0.01
          power = 0.9
    alternative = two.sided

NOTE: n is number in *each* group
```

Estimate power for a two-sample t-test.

- difference in means, delta = 0.5
- standard deviation, sd = 0.5
- alpha, sig.level = 0.01
- sample size, n = 31

```
power.t.test(delta = 0.5, sd = 0.5, sig.level = 0.01, n = 31)
```

```
     Two-sample t test power calculation

              n = 31
          delta = 0.5
             sd = 0.5
      sig.level = 0.01
          power = 0.8946133
    alternative = two.sided

NOTE: n is number in *each* group
```

2.4 Estimate sample size using data from a pilot study

Suppose you plan an experiment, and want to estimate sample size. You don't have information on mean effect size or standard deviation. You perform a pilot study to collect information on the mean and standard deviation of two groups, and get the following results.

```
group1=c(1.1, 2.3, 4.8, 5.4, 7.9)
group2=c(3.3, 4.2, 4.7, 7.6, 9.2)

mean(group2)-mean(group1)
```

```
[1] 1.5
```

```
sd(group1)
```

```
[1] 2.676752
```

```
sd(group2)
```

```
[1] 2.490984
```

The estimated effect size is the difference between the means, which is 1.5. The standard deviations of the two groups are 2.49 and 2.68. It is usually better to use the larger estimate of the standard deviation when estimating sample size, so we'll use sd=2.7. We'll use power=0.8.

```
power.t.test(delta = 1.5, sd = 2.7, sig.level = 0.05,
power=0.8)
```

```
     Two-sample t test power calculation

              n = 51.83884
          delta = 1.5
             sd = 2.7
      sig.level = 0.05
          power = 0.8
    alternative = two.sided

NOTE: n is number in *each* group
```

The power calculation indicates we need 52 subjects per group, for a total of 104 subjects. The main reason for the large sample size is that the standard deviation (sd = 2.7) is large compared to the difference between the means (delta = 1.5). If we want to reduce the sample size, we need to find a way to reduce the within group standard deviation, which is unexplained variance. You can include additional explanatory variables in your analysis to reduce the unexplained variance. We'll see more methods to reduce this unexplained variance in later chapters.

2.5 Assay variability reduces power, increases sample size, and gives worse p-values

Suppose you plan an experiment using a 96 well plate. You run identical specimens in all 96 wells. You find that the standard deviation of the identical specimens in the 96 wells is 10 units. The effect size you hope to detect is 5 units. What sample size will you need?

```
power.t.test(delta=5, sd=10, sig.level = 0.05, power=0.8)
```

```
     Two-sample t test power calculation

              n = 63.76576
          delta = 5
             sd = 10
      sig.level = 0.05
          power = 0.8
    alternative = two.sided

NOTE: n is number in *each* group
```

You think that by fixing the assay, you may be able to reduce the standard deviation of identical specimens in the 96 wells to 5 units. How does this affect sample size?

```
power.t.test(delta=5, sd=5, sig.level = 0.05, power=0.8)
```

```
     Two-sample t test power calculation

              n = 16.71477
          delta = 5
             sd = 5
      sig.level = 0.05
          power = 0.8
    alternative = two.sided

NOTE: n is number in *each* group
```

What do you prefer, n = 64 per group or n = 17 per group? It is pretty clear that reducing the unexplained variability in an assay can give you dramatic improvements in increased power, decreased sample size, and getting $p < 0.05$. I often see such variability due to row effects, column effects, or edge effects (such as evaporation from poor sealing).

In this illustration we only examined one source of variability: the standard deviation of the identical specimens. For actual experiments the sample size calculation would need to account for additional sources of variability, such as the variability of subjects within treatment groups.

3 Increase sample size last: reduce unexplained variability first

When you want to increase the chance of getting $p < 0.05$, the first thing you may think of is to increase the sample size. In this chapter, I'll show you why increasing the sample size should be the last thing you do. Reducing unexplained variability first is more effective and requires less work than increasing sample size. Let's see why.

We'll start with one simulation where we increase sample size, and then do a second simulation where we reduce unexplained variability. You may be surprised at how different the results are. We are going to use the idea of power, so we'll start with a brief explanation.

Power is the probability that you will get $p < 0.05$ in your experiment, assuming that the treatment has an effect. Typically, we want our experiments to have power of 80% or 90%, meaning that there is an 80% or 90% chance that we will get $p < 0.05$ if our assumptions about variability and effect size in our experiment are correct.

Suppose you do a power analysis and find that you only have 30% power; that is, the probability that you will get $p < 0.05$ is 30%. What do you do if your experiment has such low power?

Power for a t-test depends on four things:

- The difference between the means of the two treatment groups (effect size)
- The variability of the treatment effect with each treatment group. This is represented by the standard deviation within each treatment group.
- Sample size: how many subjects will be in each group?
- The required p-value (alpha). Alpha is most often 0.05, which means that we must get a p-value less than 0.05 to conclude that the result is significant and the treatment has an effect.

Let's do some simulations to illustrate how effective decreasing unexplained variability is compared to increasing sample size.

3.1 Simulations to illustrate the effect of sample size and within-group variability on power

We'll use the R software to simulate how power changes depending on the variability within treatment groups and the sample size.

3.1.1 Increase sample size

We'll simulate a clinical trial for a drug that lowers mean blood pressure by 10 units. We'll create two populations for the simulation:

- A population of 10,000 patients who receive a placebo.
- A population of 10,000 patients who receive a drug to reduce blood pressure.

Create a population of 10,000 patients who receive a placebo. We'll generate these to have approximately mean = 150, standard deviation = 20. Here is the R code. We use the R function set.seed() so that the results will be reproducible when you run the code.

```
set.seed(1234)
placebo= rnorm(10000, 150, 20)
mean(placebo)
```

```
[1] 150.1223
```

```
sd(placebo)
```

```
[1] 19.75059
```

Create a population of 10,000 patients who receive a drug to reduce blood pressure. We'll generate these to have approximately mean = 140, standard deviation = 20.

```
set.seed(3456)
drug = rnorm(10000, 140, 20)
mean(drug)
```

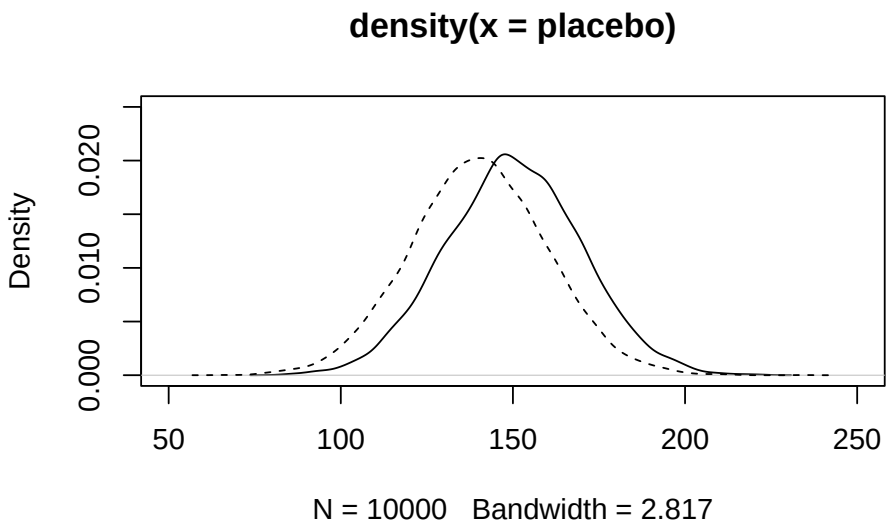
```
[1] 140.0547
```

```
sd(drug)
```

```
[1] 19.91002
```

Plot the two populations.

```
plot(density(placebo), xlim= c(50, 250),ylim=c(0,.025))
lines(density(drug), lty=2)
```

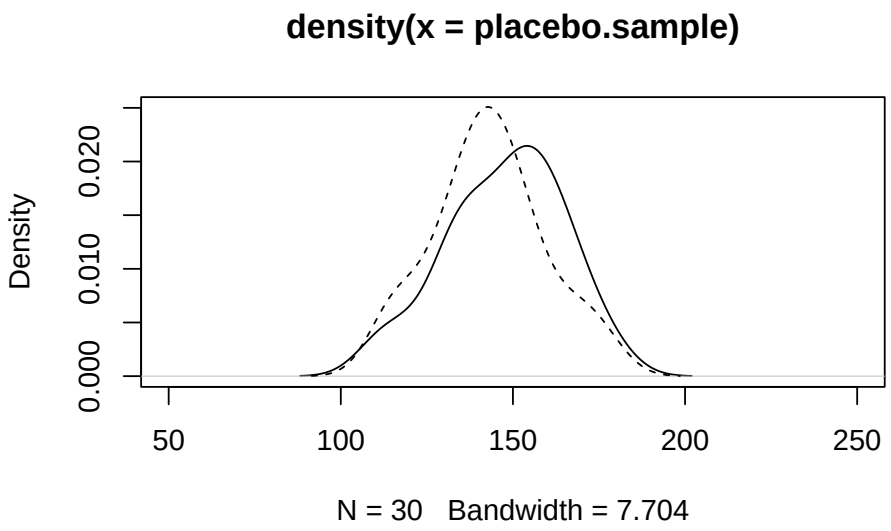


Take samples of size n = 30 from the placebo group and the drug group.

```
placebo.sample = sample(placebo, size=30)
drug.sample = sample(drug, size=30)
```

Plot the two samples.

```
plot(density(placebo.sample), xlim= c(50, 250),ylim=c(0,
.025))
lines(density(drug.sample), lty=2)
```



Perform a t-test to compare the two samples.

```
ttest.result = t.test(placebo.sample, drug.sample,
var.equal=TRUE)
```

Notice that, in this sample, even though we know that the two populations are different (mean of 150 vs. mean of 140), the t-test gives us p = 0.2812, a non-significant result. We would conclude, incorrectly, that there is no difference between the two groups, and that the drug has no effect.

We can extract the p-value from the t-test output, so that we can pass it to other functions, which we will do shortly.

```
ttest.result$p.value
```

```
[1] 0.2812291
```

What happens if we repeat the process of taking a sample and doing a t-test several times? Here is code to take 4 samples, perform 4 t-tests, and output the 4 p-values.

```
set.seed(1234)
for (i in 1:4) {
  placebo.sample = sample(placebo, size=30)
  drug.sample = sample(drug, size=30)
```

```
  ttest.result = t.test(placebo.sample, drug.sample,
var.equal=TRUE)
  print(ttest.result$p.value)
}
```

```
[1] 0.02992261
[1] 0.81321
[1] 0.0461481
[1] 0.3368997
```

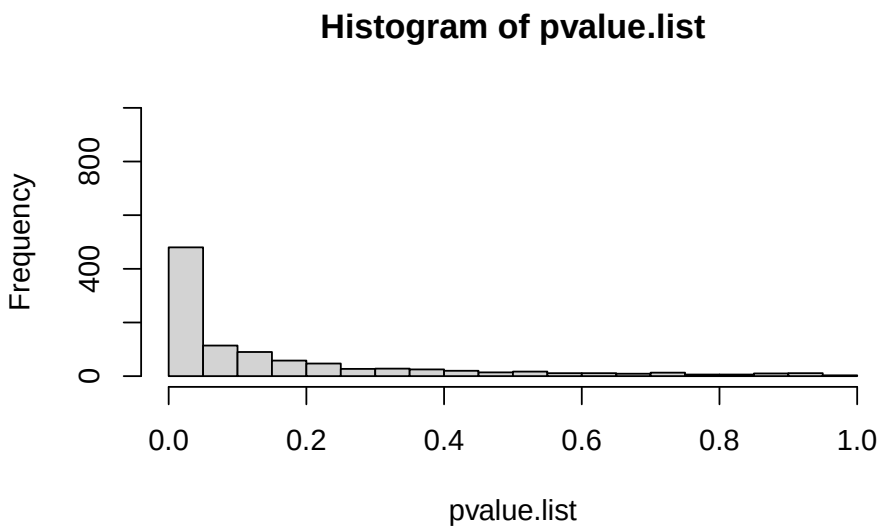
Even with just these 4 samples, we have p-values ranging from 0.0299 to 0.81. The p-values are going up or down by an order of magnitude, just as a result of taking another sample from the identical population. Nothing else is changing; we are just taking another random sample.

What is the probability that we will detect a significant difference ($p < 0.05$) if we take many samples of size n=30? To answer this question, we'll do a simulation of 1000 samples and t-tests, and look at the distribution of p-values.

```
set.seed(12345)
n = 30
pvalue.list = c()
for (i in 1:1000)
{
  placebo.sample = sample(placebo, size=n)
  drug.sample = sample(drug, size=n)
  pvalue.list[i] = t.test(placebo.sample, drug.sample,
var.equal=TRUE)$p.value
}
```

Here is a plot of the 1000 p-values we get from taking 1000 random samples (stored in the variable pvalue.list).

```
hist(pvalue.list, xlim= c(0, 1), breaks=seq(0,1,.05),
ylim=c(0,1000))
```



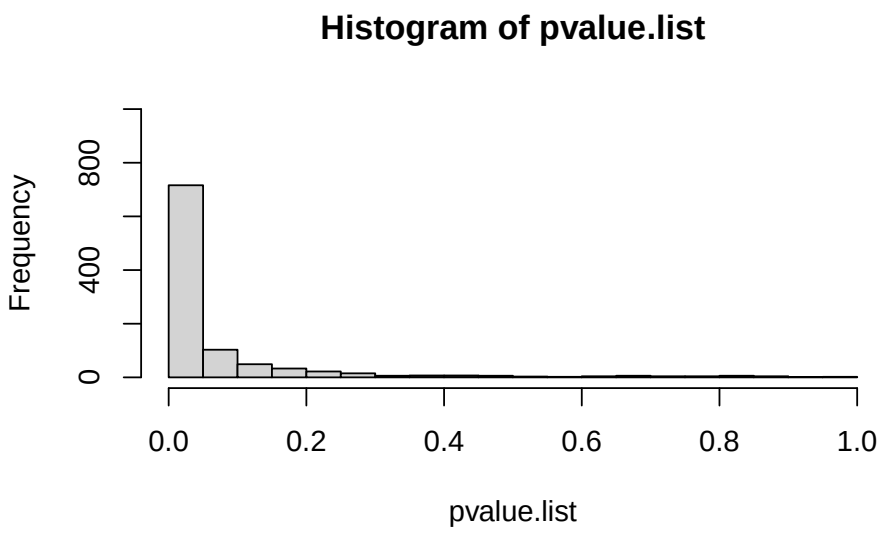
What percent of the 1000 simulated samples give a p-value less than 0.05? From the graph we see that about 500 out of 1000 simulated samples give $p < 0.05$. So, the probability that our experiment will give us a significant result is 500/1000, or 50%. The simulation indicates that we have about 50% power.

If we increase sample size, we increase power. What is the probability that we will detect a significant difference ($p < 0.05$) if we increase the sample size to n=50? Do a simulation of 1000 samples and t-tests and look at the distribution of p-values.

```
set.seed(12345)
n = 50
pvalue.list = c()
for (i in 1:1000)
{
  placebo.sample = sample(placebo, size=n)
  drug.sample = sample(drug, size=n)
  pvalue.list[i] = t.test(placebo.sample, drug.sample,
var.equal=TRUE)$p.value
}
```

Plot the pvalue.list.

```
hist(pvalue.list, xlim= c(0, 1), breaks=seq(0,1,.05),
ylim=c(0,1000))
```



What percent of the 1000 simulated samples give a p-value less than 0.05? From the graph, approximately 700 out of 1000 simulated samples give $p < 0.05$. So, the probability that our experiment will give us a significant result is 700/1000, or 70%. The simulation indicates that we have 70% power. Even after a fairly big increase in sample size we still have unsatisfactory power.

3.1.2 Decrease the unexplained variance

If we decrease the unexplained variance, we increase power. Suppose that we know that other factors affect our response variable. For example, age, sex, medical history, and baseline disease severity may all affect the response variable, regardless of treatment. We may know that different instruments give higher or lower values, so we can control for those differences. A good strategy to reduce the unexplained variance is to include variables that affect the response in a multiple regression or ANOVA model. Here's a simulation.

Create a population of 10,000 patients who receive a placebo. We'll generate these to have approximately mean = 150, standard deviation = 10.

```
set.seed(1234)
placebo= rnorm(10000, 150, 10)
```

```
mean(placebo)
```

```
[1] 150.0612
```

```
sd(placebo)
```

```
[1] 9.875294
```

Create a population of 10,000 patients who receive a drug to reduce blood pressure. We'll generate these to have approximately mean = 140, standard deviation = 10.

```
set.seed(3456)
drug = rnorm(10000, 140, 10)
mean(drug)
```

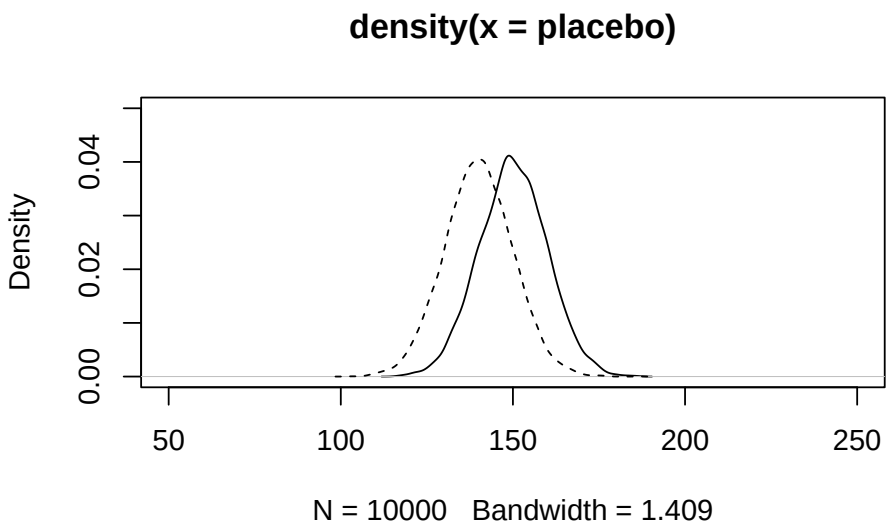
```
[1] 140.0273
```

```
sd(drug)
```

```
[1] 9.95501
```

Plot the two populations.

```
plot(density(placebo), xlim= c(50, 250), ylim=c(0, .05))
lines(density(drug), lty=2)
```

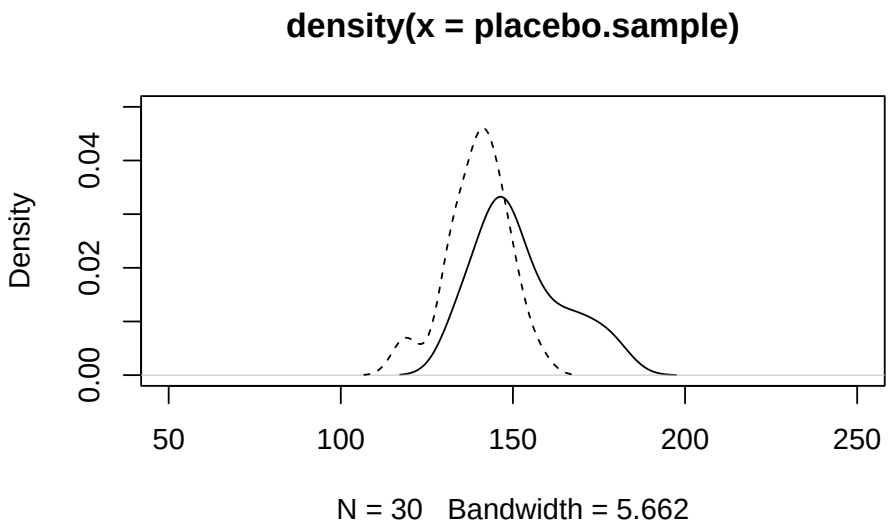


Take sample of size n = 30 from each group.

```
set.seed(4321)
placebo.sample = sample(placebo, size=30)
drug.sample = sample(drug, size=30)
```

Plot the two samples.

```
plot(density(placebo.sample), xlim= c(50, 250),ylim=c(0, .05))
lines(density(drug.sample), lty=2)
```



Perform a t-test to compare the two samples.

```
t.test(placebo.sample, drug.sample, var.equal=TRUE)
```

```
    Two Sample t-test

data:  placebo.sample and drug.sample
t = 4.2261, df = 58, p-value = 8.519e-05
alternative hypothesis: true difference in means is not equal
to 0
95 percent confidence interval:
  6.399623 17.917451
sample estimates:
```

```
mean of x mean of y
 151.9098  139.7513
```

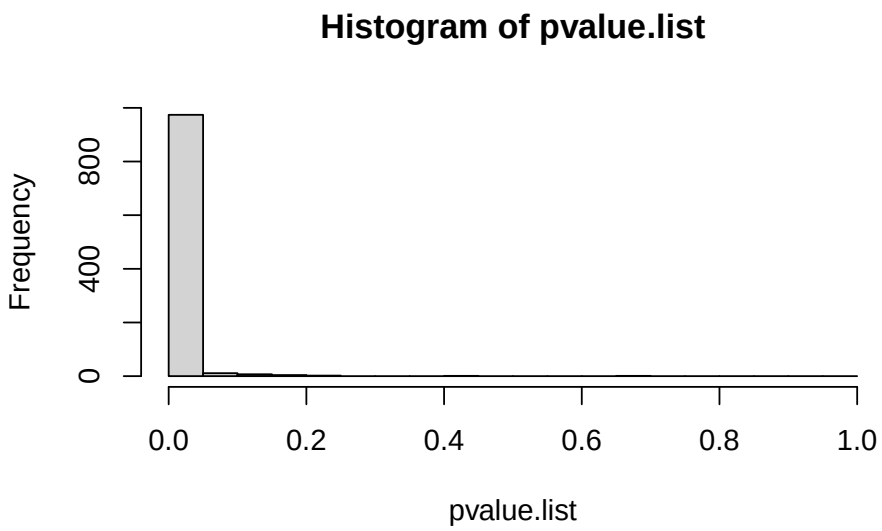
After reducing the variability within group we get a good p-value, p = 0.000085, for this sample.

What is the probability that we will detect a significant difference ($p < 0.05$) if we take many samples of size n=30, with this reduced variability? Do a simulation of 1000 samples and t-tests and look at the distribution of p-values.

```
set.seed(4321)
n = 30
pvalue.list = c()
for (i in 1:1000)
{
  placebo.sample = sample(placebo, size=n)
  drug.sample = sample(drug, size=n)
  pvalue.list[i] = t.test(placebo.sample, drug.sample,
var.equal=TRUE)$p.value
}
```

Plot the pvalue.list.

```
hist(pvalue.list, xlim= c(0, 1), breaks=seq(0,1,.05),
ylim=c(0,1000))
```



What percent of the 1000 simulated samples give a p-value less than 0.05?

```
sum(pvalue.list < 0.05)/1000
```

```
[1] 0.974
```

We see that 974 out of 1000 simulated samples give $p < 0.05$. So, the probability that our experiment will give us a significant result is 974/1000, or 97.4%. The simulation indicates that we have 97.4% power.

Which would you prefer to do: increase the sample size (spending more time and money), or decrease the unexplained variance by controlling for other variables that affect the response?

3.2 Which of the four factors that affect power should you change to best increase power?

- Difference between group means: In many experiments and clinical trials, for example testing a drug or testing the effect of a genetic mutation, it is difficult to increase the difference between the group means. Usually this is not something you can change quickly. You may be able to select subjects (such as patients or animals with more severe disease) that may have a bigger response to the treatment.

- Variability within treatment group: There are many ways that you can reduce the variability within treatment groups. For example:
  - Use multiple regression or multi-way ANOVA to include additional variables that affect the response. If variables such as age or sex affect the response, it is less work to collect that information for each subject than it is to get more subjects.
  - Use rank methods or robust methods that reduce the effect of outliers on variability.
  - Identify and control process variables, such as plate, batch, day, or operator, that increase unexplained variability. See the chapter on fractional factorial experiment designs for very efficient ways to identify such variables.
- Sample size: When you want to increase power, increasing sample size is less effective than reducing variability for two reasons.
  - Reducing the standard deviation directly increases power. Unfortunately, increasing the sample size only increases power as the square root of n, rather than directly with n. We also get diminishing returns: the bigger the sample size, the less benefit we get from further increasing n.
  - If you increase power by increasing sample size, you haven't fixed the problem of large variability within a group. You will still have to increase sample size in all your future experiments. In contrast, suppose you do a little work and find out what variables contribute most of the unexplained variability in the experimental setup or system you use. Then, in all your subsequent experiments, you can use the same methods to reduce variability, increase power and reduce sample size. Reducing unexplained variability is much more effective than increasing sample size.
- Change alpha: Most scientific and clinical journals expect alpha of 0.05, so you have little choice about using a different alpha. However, see the chapter "Use two primary analyses" to see how using alpha of 0.025 for two analyses can increase the overall power of a study.

3.3 Definition of effect size and standardized effect size

The effect size for a t-test can be defined as the difference between the means of the two groups.

The standardized effect size for a t-test is the difference between the means of the two groups divided by the within-group standard deviation.

Some statisticians use the term “effect size” to mean the “standardized effect size”.

4 If you have normally distributed data with outliers: Robust tests

If your data are approximately normally distributed, but you have a few extreme outliers, the outliers can cause much worse p-values and cause large errors in the estimated effect of the explanatory variables. In this chapter we examine robust methods, which reduce the influence of extreme outliers.

Install the RobStatTM package if you haven't already done so.

```
# install.packages("RobStatTM")
```

Load the library, using the command library(RobStatTM).

```
library(RobStatTM)
```

Load the ggplot library.

```
library(ggplot2)
```

4.1 Robust methods: Reduce the influence of extreme observations

Robust methods are designed to give good power when most of the observations in a data set are from a normal distribution with a particular mean and standard deviation, while a small portion of the observations are from a different distribution. The different distribution will likely have a different mean and standard deviation and might not be normal. This small portion of the observations from the different distribution are often identified as outliers.

The essential idea of robust methods is to down weight the outliers so that they have less effect on the statistical model and tests. If there are no outliers, the robust tests give results very similar to a classical t-test, analysis of variance, or linear regression. If there are outliers, the robust test will

usually give better p-values than the classical tests, by giving most weight to the majority of observations.

Robust methods are available in R, JMP, SAS, Stata, S-plus and other software. The R package RobStatTM presented here accompanies the textbook "Robust Statistics Theory and Methods (with R)" by Ricardo Maronna et al. (Maronna et al. 2018). It includes recent research advances not included in other software.

4.1.1 Example with one explanatory variable: Robust alternative to t-tests

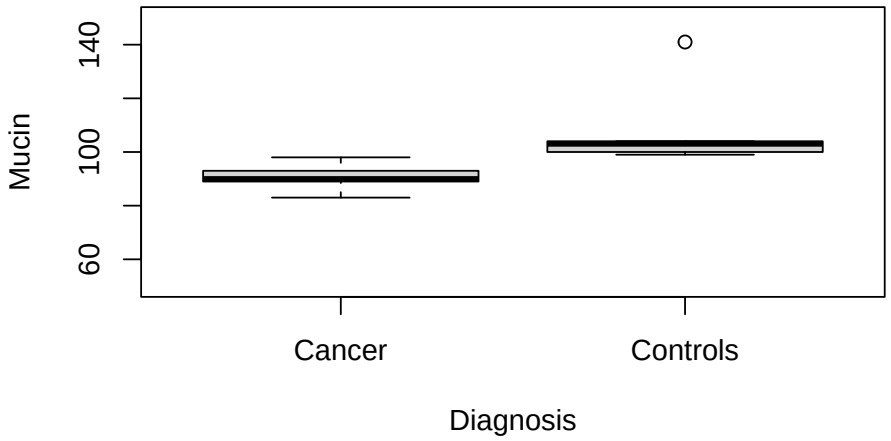
Let's start with an example where we use a t-test for the question, do colon cancer patients have elevated levels of the mucin protein in their blood? In this synthetic data, we have the level of the protein mucin in two groups: (1) patients with colon cancer and (2) healthy controls. The two-sample t-test is commonly used for such data.

Here is R code with the data and a graph.

```
cancer=c(83, 89, 90, 93, 98)
controls=c(99, 100, 103, 104,  141)
Mucin=c(cancer,controls)
Diagnosis=c(rep("Cancer",5), rep("Controls",5))
colon.data=data.frame(Diagnosis, Mucin)

colon.data
```

```
   Diagnosis Mucin
1     Cancer    83
2     Cancer    89
3     Cancer    90
4     Cancer    93
5     Cancer    98
6   Controls    99
7   Controls   100
8   Controls   103
9   Controls   104
10  Controls   141
```

```
with(colon.data, boxplot(Mucin ~ Diagnosis, ylab="Mucin",
ylim=c(50,150)))
```



The boxplot and data show that there is complete separation between the two groups. In fact, all the cancer subjects have lower mucin values (98 or less) than any of the controls (99 or greater). But notice the large outlier in the control group, where one subject has a mucin value of 141.

Here are the means and medians of the two groups. Notice that the mean of the control group, 109.4, is larger than the median, 103. This difference is due to the outlier.

```
aggregate(Mucin ~ Diagnosis, colon.data, mean)
```

```
  Diagnosis Mucin
1    Cancer  90.6
2  Controls 109.4
```

```
aggregate(Mucin ~ Diagnosis, colon.data, median)
```

```
  Diagnosis Mucin
1    Cancer    90
2  Controls   103
```

Here is the ordinary two-sample t-test, using the R function t.test.

```
with(colon.data, t.test(Mucin ~ Diagnosis, var.equal=TRUE))
```

```
    Two Sample t-test

data:  Mucin by Diagnosis
t = -2.258, df = 8, p-value = 0.05389
alternative hypothesis: true difference in means between group
Cancer and group Controls is not equal to 0
95 percent confidence interval:
 -37.9994752   0.3994752
sample estimates:
  mean in group Cancer mean in group Controls
                  90.6                   109.4
```

The boxplot and the data show that there is complete separation of the two groups, but the p-value for the t-test is not significant ($p = 0.05389$). The reason that the t-test is not significant is the large outlier. The outlier increases the standard deviation, which increases the standard error, which gives the non-significant p-value.

The output from the t-test includes the means of the two groups (90.6 and 109.4), and the 95% confidence interval for difference between the means of the two groups (-37.99 to 0.399). Because the 95% confidence interval includes 0, we get $p > 0.05$.

Perform the analysis using the function lmrobM from the RobStatTM package.

```
summary(with(colon.data, lmrobM(Mucin ~ Diagnosis)))
```

```
Call:
lmrobM(formula = Mucin ~ Diagnosis)
Residuals:
    Min      1Q  Median      3Q     Max
-7.6069 -1.5801  0.4466  2.4733 39.5000

Coefficients:
                 Estimate Std. Error t value Pr(>|t|)
(Intercept)        90.607      1.999  45.336 6.19e-11 ***
DiagnosisControls  10.893      2.993   3.639   0.0066 **
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
Robust residual standard error: 4.984
Multiple R-squared:  0.4481,    Adjusted R-squared:  0.3792
Convergence in 5 IRWLS iterations
```

The lmrobM robust test gives a p-value of 0.0066. In contrast to the non-significant t-test, the outlier had little effect on the robust test.

4.1.2 Example robust analysis of a data set with two explanatory variables

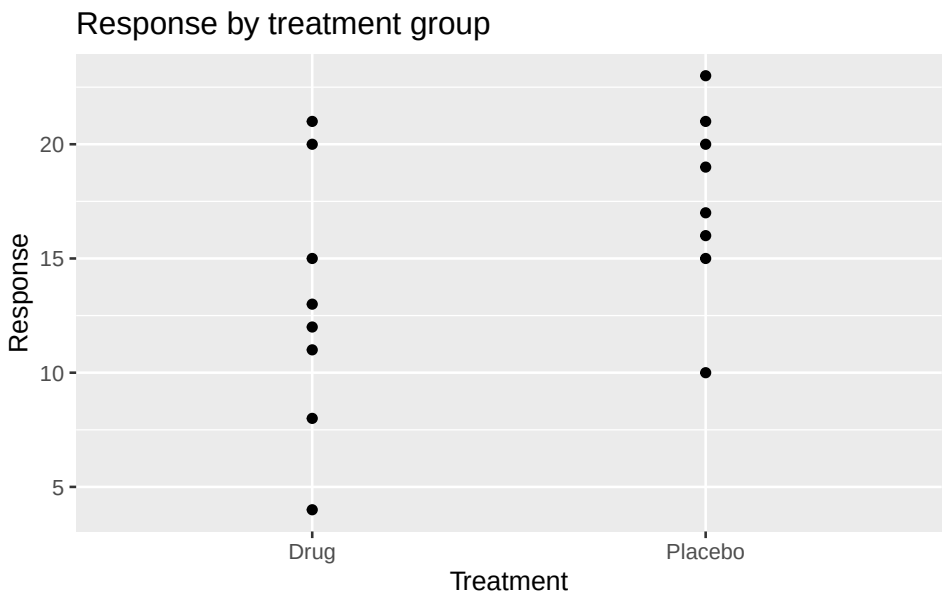
Now let's look at an example that illustrates robust analysis of a data set with two explanatory variables. I'll start with a dataset that has no outliers. I'll then modify the data set to have outliers. I'll show the analysis with and without outliers for an ordinary ANOVA and for the robust analysis.

We have the following (synthetic) data for an experiment with mice, where treatment was either drug or placebo, and there were 4 mice from each litter. Within each litter, 2 mice were randomly assigned to drug and 2 were randomly assigned to placebo.

```
anova.2way.data = tibble::tribble(
~Treatment, ~Litter, ~Response,
  "Drug", "Litter.1", 4,
  "Drug", "Litter.1", 8,
  "Drug", "Litter.2", 11,
  "Drug", "Litter.2", 12,
  "Drug", "Litter.3", 13,
  "Drug", "Litter.3", 15,
  "Drug", "Litter.4", 20,
  "Drug", "Litter.4", 21,
  "Placebo", "Litter.1", 10,
  "Placebo", "Litter.1", 15,
  "Placebo", "Litter.2", 16,
  "Placebo", "Litter.2", 17,
  "Placebo", "Litter.3", 19,
  "Placebo", "Litter.3", 20,
  "Placebo", "Litter.4", 21,
  "Placebo", "Litter.4", 23)
```

Here is a graph of the response by treatment group.

```
ggplot(anova.2way.data,
aes(y=Response, x=Treatment)) +
geom_point() + ggtitle("Response by treatment group")
```



Here is the t-test for response versus treatment group.

```
t.test(Response ~ Treatment, var.equal=TRUE, data =
anova.2way.data)
```

```
    Two Sample t-test

data:  Response by Treatment
t = -1.8664, df = 14, p-value = 0.08307
alternative hypothesis: true difference in means between group
Drug and group Placebo is not equal to 0
95 percent confidence interval:
 -9.9398424  0.6898424
sample estimates:
   mean in group Drug mean in group Placebo
               13.000                17.625
```

The t-test is not significant, with p = 0.083. We would conclude that the drug has no significant effect. The t-test analysis can also be done using the R function lm(), which performs a linear regression.

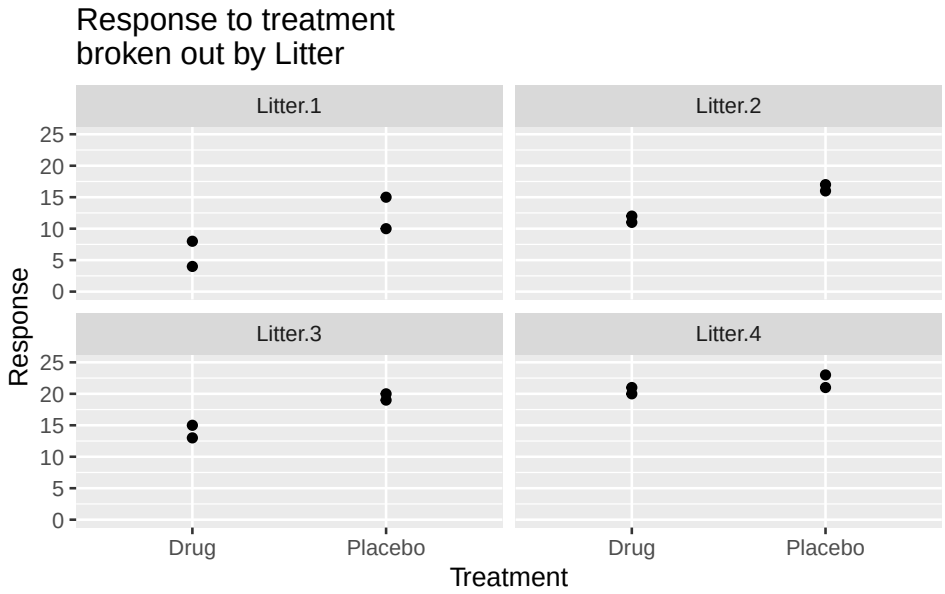
```
summary(lm (Response ~ Treatment, data = anova.2way.data))
```

```

Call:
lm(formula = Response ~ Treatment, data = anova.2way.data)

Residuals:
    Min      1Q  Median      3Q     Max
-9.0000 -2.1563 -0.3125  2.6250  8.0000

Coefficients:
                 Estimate Std. Error t value Pr(>|t|)
(Intercept)        13.000      1.752   7.419 3.26e-06 ***
TreatmentPlacebo    4.625      2.478   1.866   0.0831 .
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 4.956 on 14 degrees of freedom
Multiple R-squared:  0.1992,    Adjusted R-squared:  0.142
F-statistic: 3.483 on 1 and 14 DF,  p-value: 0.08307
```

The linear regression gives the same result as the t-test (as it should) with p = 0.083.

However, the researcher thought that mice from different litters might have different responses. Mice from a litter with few kits tend to be larger, while mice from a litter with many kits tend to be smaller. This might affect their response to the drug. Here is a plot of response versus treatment, broken out by litter.

```
ggplot(anova.2way.data, aes(y=Response, x=Treatment)) +
geom_point() +
  facet_wrap(~Litter) +
        ggtitle("Response to treatment  \nbroken out by
        Litter") + ylim(0,25)
```



The graph suggests that there is some difference among the 4 litters. Litter 1 has low values for both drug and placebo, while litter 4 has high values for both.

Run a two-way ANOVA including both treatment and litter as explanatory variables.

```
lm.2way = lm(Response ~ Litter + Treatment, data=
anova.2way.data)
anova(lm.2way)
```

```
Analysis of Variance Table

Response: Response
          Df  Sum Sq Mean Sq F value    Pr(>F)
Litter     3 303.187 101.062  27.323 2.135e-05 ***
Treatment  1  85.562  85.562  23.132 0.0005449 ***
Residuals 11  40.688   3.699
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

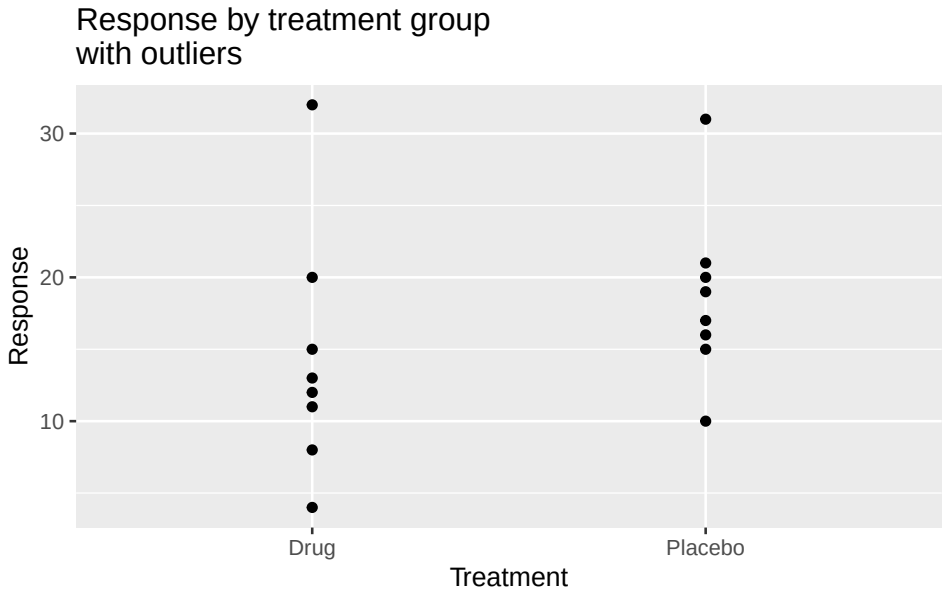
The two-way ANOVA shows that both litter ($p = 0.000021$) and treatment ($p = 0.00054$) have a significant effect on the response. Analyses that included only treatment as the explanatory variable would lead to the wrong conclusions. What if, in addition, there were outliers in the data?

Here is the modified data set that has outliers. I've changed the highest values in litter 4 to be 32 for the mouse receiving drug and 31 for the mouse receiving placebo. These are rows 8 and 16 in the table below.

```
anova.2way.data.outlier = tibble::tribble(
~Treatment, ~Litter, ~Response,
  "Drug", "Litter.1", 4,
  "Drug", "Litter.1", 8,
  "Drug", "Litter.2", 11,
  "Drug", "Litter.2", 12,
  "Drug", "Litter.3", 13,
  "Drug", "Litter.3", 15,
  "Drug", "Litter.4", 20,
  "Drug", "Litter.4", 32,
  "Placebo", "Litter.1", 10,
  "Placebo", "Litter.1", 15,
  "Placebo", "Litter.2", 16,
  "Placebo", "Litter.2", 17,
  "Placebo", "Litter.3", 19,
  "Placebo", "Litter.3", 20,
  "Placebo", "Litter.4", 21,
  "Placebo", "Litter.4", 31)
```

Here is the graph of response by treatment group for the data with outliers.

```
ggplot(anova.2way.data.outlier, aes(y=Response, x=Treatment))
+ geom_point() +
ggtitle("Response by treatment group \nwith outliers")
```



Here is the t-test for the data with the outliers, run first using the t.test function and then using the lm function.

```
t.test(Response ~ Treatment, var.equal=TRUE, data =
anova.2way.data.outlier)
```

```
    Two Sample t-test

data:  Response by Treatment
t = -1.1478, df = 14, p-value = 0.2703
alternative hypothesis: true difference in means between group
Drug and group Placebo is not equal to 0
95 percent confidence interval:
 -12.191454   3.691454
sample estimates:
   mean in group Drug mean in group Placebo
               14.375                 18.625
```

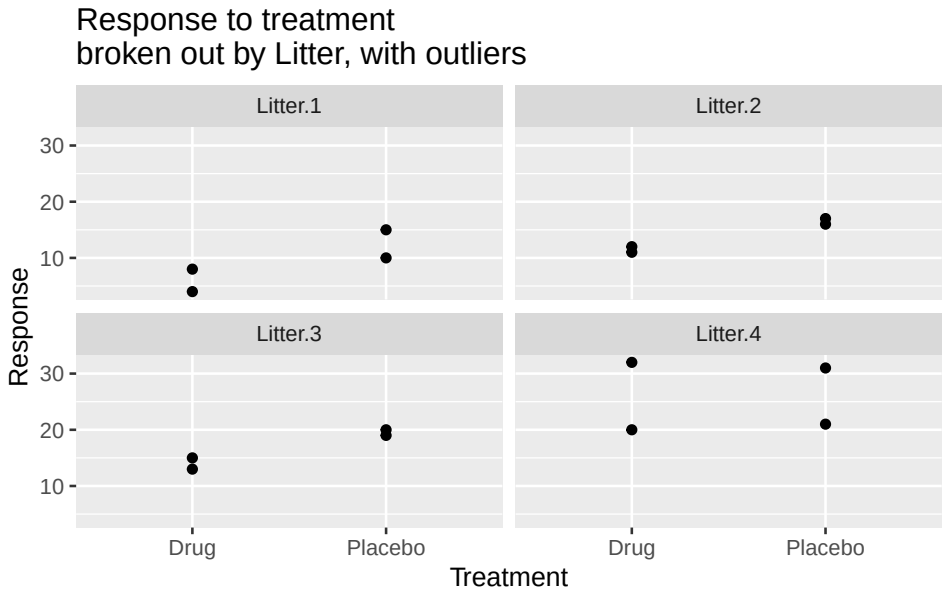
```
summary(lm (Response ~ Treatment, data =
anova.2way.data.outlier))
```

```
Call:
```

```
lm(formula = Response ~ Treatment, data =
anova.2way.data.outlier)

Residuals:
    Min      1Q  Median      3Q     Max
-10.375  -3.438  -1.500   1.625  17.625

Coefficients:
                 Estimate Std. Error t value Pr(>|t|)
(Intercept)        14.375      2.618   5.490 7.96e-05 ***
TreatmentPlacebo    4.250      3.703   1.148     0.27
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 7.405 on 14 degrees of freedom
Multiple R-squared:  0.08601,   Adjusted R-squared:  0.02073
F-statistic: 1.317 on 1 and 14 DF,  p-value: 0.2703
```

The t-test and linear model both show that the drug effect is not significant, p = 0.27. Let's look at the results broken out by litter.

```
ggplot(anova.2way.data.outlier, aes(y=Response, x=Treatment))
+ geom_point() +
  facet_wrap(~Litter) +
ggtitle("Response to treatment  \nbroken out by Litter, with
outliers")
```



The outliers in litter 4 are evident, as is the large difference in response between the litters.

We'll first try an ordinary two-way ANOVA for this data with outliers.

```
lm.2way.outlier = lm(Response ~ Litter + Treatment,
  data= anova.2way.data.outlier)
anova(lm.2way.outlier)
```

```
Analysis of Variance Table

Response: Response
          Df Sum Sq Mean Sq F value    Pr(>F)
Litter     3 596.50 198.833 12.7718 0.0006627 ***
Treatment  1  72.25  72.250  4.6409 0.0542426 .
Residuals 11 171.25  15.568
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

The ordinary two-way ANOVA confirms the additional variability due to litter ($p = 0.00066$) but indicates that the treatment is not significant ($p = 0.054$).

If the treatment has an effect in one litter, but a (significantly) different effect in another litter, there is an interaction. An interaction can occur if,

for example, the effect of treatment differs significantly by sex, by age, or by other variables. Examples of interactions are in the chapter "Look (out) for interactions" and in the chapter "Avoid one-variable-at-a-time experiments. Use factorial experiment design".

Here is the test for an interaction between treatment and litter. The formula "Response ~ Litter * Treatment" specifies the interaction test.

```
anova(lm(Response ~ Litter * Treatment, data=
anova.2way.data.outlier))
```

```
Analysis of Variance Table

Response: Response
                 Df Sum Sq Mean Sq F value   Pr(>F)
Litter            3 596.50 198.833 10.8950 0.003377 **
Treatment         1  72.25  72.250  3.9589 0.081811 .
Litter:Treatment  3  25.25   8.417  0.4612 0.717073
Residuals         8 146.00  18.250
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

The interaction test is not significant ($p = 0.717$), indicating that there is no interaction between treatment and litter. So, we can drop the interaction and use the results for the test of litter + treatment shown above.

4.1.3 Robust regression using the RobStatTM package

Now we'll do a robust analysis of the same data using the RobStatTM package.

Here is the analysis of the outlier data, with litter and treatment as the two explanatory variables. The function lmrobM() performs the robust regression analysis. (The two-way ANOVA can be implemented as a linear regression.)

```
anova.2way.data.outlier.lmrobM = lmrobM(Response ~ Litter +
Treatment,
  data = anova.2way.data.outlier)

summary(anova.2way.data.outlier.lmrobM)
```

```
Call:
lmrobM(formula = Response ~ Litter + Treatment, data =
anova.2way.data.outlier)
Residuals:
    Min      1Q  Median      3Q     Max
-2.7439 -0.6934  0.5000  1.4406 13.9942

Coefficients:
                 Estimate Std. Error t value Pr(>|t|)
(Intercept)         6.744      1.124   6.000 8.92e-05 ***
LitterLitter.2      4.762      1.403   3.393 0.006000 **
LitterLitter.3      7.512      1.403   5.354 0.000233 ***
LitterLitter.4     11.262      1.714   6.569 4.03e-05 ***
TreatmentPlacebo    4.988      1.060   4.704 0.000646 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Robust residual standard error: 2.986
Multiple R-squared:  0.6808,    Adjusted R-squared:  0.5647
Convergence in 8 IRWLS iterations
```

The robust regression indicates that treatment has a significant effect ($p = 0.0006$). Compare this result to the conventional two-way ANOVA result of $p = 0.054$ for the outlier data, which indicated that treatment was not significant.

We can also examine the results for the robust analysis for the data with no outliers.

```
anova.2way.data.lmrobM = lmrobM(Response ~ Litter + Treatment,
  data = anova.2way.data) # control=cont)

summary(anova.2way.data.lmrobM)
```

```
Call:
lmrobM(formula = Response ~ Litter + Treatment, data =
anova.2way.data)
Residuals:
    Min      1Q  Median      3Q     Max
-2.9258 -0.8813  0.1246  0.9704  3.4615
```

```
Coefficients:
                 Estimate Std. Error t value Pr(>|t|)
(Intercept)        6.9258     1.0767   6.433 4.86e-05 ***
LitterLitter.2     4.7678     1.3628   3.498 0.004984 **
LitterLitter.3     7.5186     1.3727   5.477 0.000193 ***
LitterLitter.4    12.0215     1.3599   8.840 2.50e-06 ***
TreatmentPlacebo   4.6127     0.9664   4.773 0.000578 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Robust residual standard error: 2.265
Multiple R-squared:  0.926, Adjusted R-squared:  0.8991
Convergence in 7 IRWLS iterations
```

We see that, for the data with no outliers, the p-value for the robust method is similar to the p-value for the classical analysis of variance ($p = 0.000545$). These results indicate that, for data with no outliers, the robust method gives results very similar to the results for a classical analysis of variance. But when there are outliers, the robust method will usually give much better p-values.

You can also analyse data with outliers using rank methods, as described in the next two chapters.

4.2 Which method should you use?

There is no single method that will always work best for all data. Robust and rank-based methods will work well in many, perhaps most, situations.

The robust methods are designed to work well when most of the data are from a normal distribution and a small portion of the data is outliers, possibly from a normal distribution with a different mean. In these situations, I find that the robust methods have very good power and usually give the best p-values.

The rank-based methods described in the next two chapters are designed to work well regardless of the distribution of the data. So, when the data have unusual or irregular distributions, the rank-based methods may give better power.

The answer also depends, in part, on the software you have available and your experience with the methods.

For detailed comparisons of the strengths and weaknesses of many rank and robust methods see the textbooks by Hettmansperger (Hettmansperger and McKean 2011) and by Wilcox (Wilcox 2022). However, be forewarned that these texts require a relatively advanced knowledge of statistics.

The robust methods and rank-based methods have the advantages of available software and textbooks and perform well in many data sets. However, many other methods are available. This is an active area of research in statistics.

5 If you have non-normal data and one explanatory variable: Wilcoxon and Kruskal-Wallis

If you have outliers in your data, or the data within each treatment group are not normally distributed, t-tests are more likely to give non-significant p-values. Outliers and non-normal distributions reduce the power of t-tests to detect significant effects. Even when graphs of your data show that there is a difference among the groups, you may get non-significant p-values from t-tests as a result of the outliers or non-normal distributions.

In these cases, the Wilcoxon rank tests and Kruskal-Wallis tests are alternatives to the t-test and anova. However, none of these methods allow you to have more than one explanatory variable. For this reason, I suggest that, instead of using these methods, you consider the methods that allow for more than one explanatory variable, described in the following chapter.

If your sample size is large, deviations from normal distribution may be less of a problem for t-tests and ANOVA. However, even with a large sample size, you can often increase power by using a rank method.

As an example of what we mean by ranks, here are observations from the sleep data frame from R with the original values of the variable extra and the values of extra converted to ranks. You can use the help(sleep) command to read a description of the data set.

```
ranked.data  = data.frame(extra = sort(sleep$extra),
  ranked.extra = rank(sort(sleep$extra)))

ranked.data
```

```
   extra ranked.extra
1   -1.6          1.0
2   -1.2          2.0
3   -0.2          3.0
```

```
4   -0.1          4.5
5   -0.1          4.5
6    0.0          6.0
7    0.1          7.0
8    0.7          8.0
9    0.8          9.5
10   0.8          9.5
11   1.1         11.0
12   1.6         12.0
13   1.9         13.0
14   2.0         14.0
15   3.4         15.5
16   3.4         15.5
17   3.7         17.0
18   4.4         18.0
19   4.6         19.0
20   5.5         20.0
```

5.1 Alternative to two-sample t-test: Wilcoxon rank sum for two independent samples

Let's look at the colon cancer example from the previous chapter, where we use a t-test for the question, do colon cancer patients have elevated levels of the mucin protein in their blood?

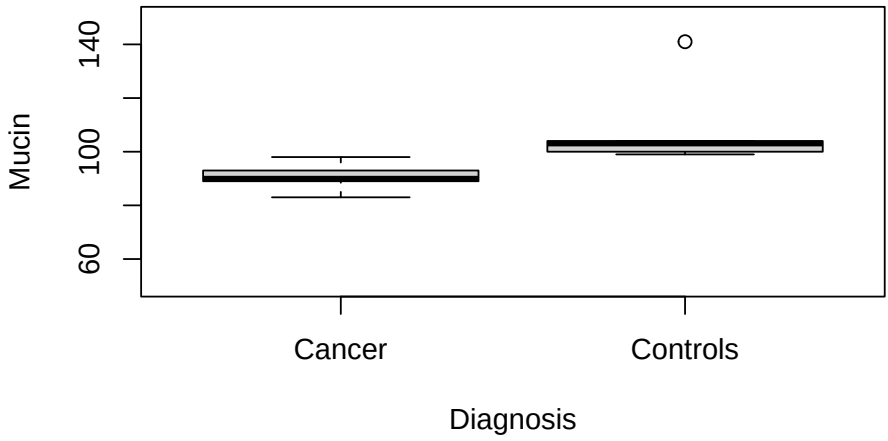
Here is R code with the data and a graph.

```
cancer=c(83, 89, 90, 93, 98)
controls=c(99, 100, 103, 104,  141)
Mucin=c(cancer,controls)
Diagnosis=c(rep("Cancer",5), rep("Controls",5))
colon.data=data.frame(Diagnosis, Mucin)

colon.data
```

```
   Diagnosis Mucin
1     Cancer    83
2     Cancer    89
3     Cancer    90
4     Cancer    93
```

```
5     Cancer    98
6   Controls    99
7   Controls   100
8   Controls   103
9   Controls   104
10  Controls   141
```

```
with(colon.data, boxplot(Mucin ~ Diagnosis, ylab="Mucin",
ylim=c(50,150)))
```



The boxplot and data show that there is complete separation between the two groups. In fact, all the cancer subjects have lower mucin values (98 or less) than any of the controls (99 or greater). But notice the large outlier in the control group, where one subject has a mucin value of 141.

Here are the means and medians of the two groups. Notice that the mean of the control group, 109.4, is larger than the median, 103. This difference is due to the outlier.

```
aggregate(Mucin ~ Diagnosis, colon.data, mean)
```

```
  Diagnosis Mucin
1    Cancer  90.6
2  Controls 109.4
```

```
aggregate(Mucin ~ Diagnosis, colon.data, median)
```

```
  Diagnosis Mucin
1    Cancer    90
2  Controls   103
```

Here is the ordinary two-sample t-test, using the R function t.test.

```
with(colon.data, t.test(Mucin ~ Diagnosis, var.equal=TRUE))
```

```

    Two Sample t-test

data:  Mucin by Diagnosis
t = -2.258, df = 8, p-value = 0.05389
alternative hypothesis: true difference in means between group
Cancer and group Controls is not equal to 0
95 percent confidence interval:
 -37.9994752   0.3994752
sample estimates:
  mean in group Cancer mean in group Controls
                  90.6                  109.4
```

The output from the t-test includes the means of the two groups (90.6 and 109.4), and the 95% confidence interval for difference between the means of the two groups (-37.99 to 0.399). Because the 95% confidence interval includes 0, we get $p > 0.05$.

The Wilcoxon rank sum test is an alternative to the t-test. The Wilcoxon rank sum test uses the rank value of each observation, rather than the actual value, so it is not affected by the outlier. Here are the rank values of the original observations.

Here is the Wilcoxon test using the R function wilcox.test.

```
with(colon.data, wilcox.test(Mucin ~ Diagnosis,
conf.int=TRUE))
```

```

    Wilcoxon rank sum exact test

data:  Mucin by Diagnosis
W = 0, p-value = 0.007937
alternative hypothesis: true location shift is not equal to 0
```

```
95 percent confidence interval:
 -51  -5
sample estimates:
difference in location
                   -13
```

Using the rank values, the Wilcoxon rank sum test yields a p-value of p = 0.0079. We reject the null hypothesis and conclude that colon cancer patients have mucin levels different from those of healthy controls.

The output from the Wilcoxon test includes the sample estimate of the difference in location, which is -13 for these data. The difference in location, the Hodges-Lehmann estimator, is the median of all pairwise differences between the two groups. That is, we calculate the 5*5 = 25 pairs of differences between each value in the cancer group and each value in the control group, and then take the median of those 25 differences.

The 95% confidence interval for the difference in location for these data is -51 to -5 (the Moses confidence interval). Because the 95% confidence interval does not include 0, we get $p < 0.05$. The Hodges-Lehmann estimator is not necessarily equal to the difference between the medians of the two groups. As described below, there may be no difference between the group medians. But if one group tends to have higher (or lower) values than the other group (stochastic dominance), the Wilcoxon test can give $p < 0.05$ even if the medians are identical.

5.2 How do you quantify the difference between two treatment groups using ranks? The Hodges-Lehmann estimator

When you compare two treatment groups using a t-test, the output tells you the means of the two group and the difference between the means.

When we compare the ranks of two treatment groups, the most commonly used descriptor of differences is the Hodges-Lehmann (HL) estimator. The HL estimator is the median of all paired differences between the two treatment groups. It's easier to understand with a simple example.

Suppose we have the following data on two groups.

```
treatment = c(5, 7, 8, 10, 12, 15, 18)
control = c(9, 11, 13, 14, 16, 20, 25)
```

We calculate all the pairwise (treatment - control) differences.

```
# Calculate all pairwise treatment - control differences
cross_diffs = outer(treatment, control, "-")
cross_diffs
```

```
     [,1] [,2] [,3] [,4] [,5] [,6] [,7]
[1,]   -4   -6   -8   -9  -11  -15  -20
[2,]   -2   -4   -6   -7   -9  -13  -18
[3,]   -1   -3   -5   -6   -8  -12  -17
[4,]    1   -1   -3   -4   -6  -10  -15
[5,]    3    1   -1   -2   -4   -8  -13
[6,]    6    4    2    1   -1   -5  -10
[7,]    9    7    5    4    2   -2   -7
```

Sort the pairwise differences.

```
sort(cross_diffs)
```

```
 [1] -20 -18 -17 -15 -15 -13 -13 -12 -11 -10 -10  -9  -9  -8
 -8  -8  -7  -7  -6
[20]  -6  -6  -6  -5  -5  -4  -4  -4  -4  -3  -3  -2  -2  -2
-1  -1  -1  -1   1
[39]   1   1   2   2   3   4   4   5   6   7   9
```

The Hodges-Lehmann (HL) estimator is then the median of all the pairwise differences.

```
median(cross_diffs)
```

```
[1] -4
```

The Hodges-Lehmann (HL) estimator tells us that the treatment group is shifted by -4 units compared to the control group.

Notice that the HL estimator is an estimate of a shift parameter, that is, the amount by which the entire distribution of outcomes in one group is displaced relative to the other. This is not the same as the difference in means or the difference in medians. Instead, it is based on the differences between all the points in each group.

In this particular example, the median of the differences (the HL estimator) happens to be equal to the difference of the medians, but this need not always be true.

```
median(treatment)
```

```
[1] 10
```

```
median(control)
```

```
[1] 14
```

```
median(treatment) - median(control)
```

```
[1] -4
```

5.3 Sample size for a t-test to give power of 0.8 for the colon data

What sample size would we need if we wanted to get a significant p-value for the t-test for the colon data? The difference between the group means is delta = 109.4-90.6 = 18.8. The standard deviation in the control group is sd = 17.8. We'll use the usual significance level (alpha) of sig.level = 0.05, and desired power of 0.8. The function power.t.test gives the required sample size for these assumptions:

```
power.t.test(delta = 18.8, sd = 17.8, sig.level = 0.05, power
= 0.8)
```

```
     Two-sample t test power calculation

              n = 15.09557
          delta = 18.8
             sd = 17.8
      sig.level = 0.05
          power = 0.8
    alternative = two.sided

NOTE: n is number in *each* group
```

We would need a sample size of 15 per group, 30 total, to have an 80% chance of getting $p < 0.05$ using a t-test. The chapters on sample size and power provide more explanation.

5.4 Alternative to paired t-test: Wilcoxon signed rank test

The paired t-test is often used to test for difference in paired (matched) samples, such as the difference before and after treatment. The Wilcoxon signed rank test is a rank-based analog of the paired t-test. Here's an example.

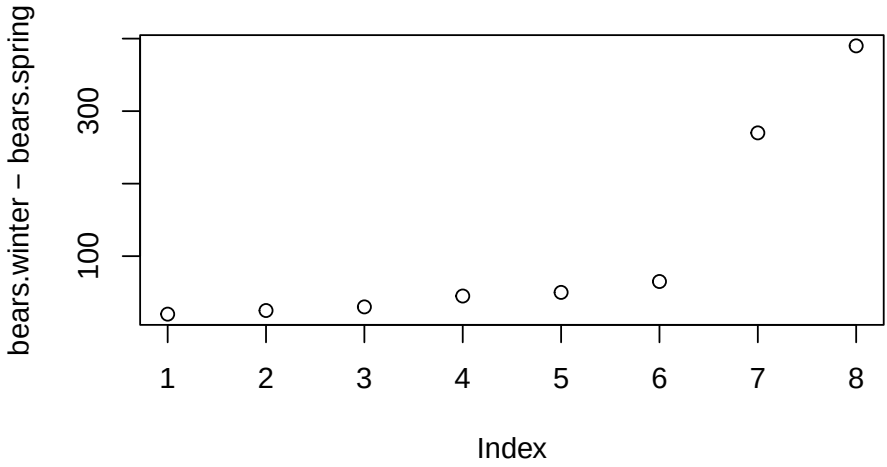
Suppose we would like to know if bears lose weight between winter and spring, while they are hibernating. We collect the following data.

```
bears.winter=c(300,470,550,760,800,955,1260,1290)
bears.spring=c(280,445,520,715,750,890,990,900)
Weight=c(bears.winter, bears.spring)
Season=c(rep("Winter",8), rep("Spring",8))
bear.data=data.frame(Season,Weight)

bear.data
```

```
   Season Weight
1  Winter    300
2  Winter    470
3  Winter    550
4  Winter    760
5  Winter    800
6  Winter    955
7  Winter   1260
8  Winter   1290
9  Spring    280
10 Spring    445
11 Spring    520
12 Spring    715
13 Spring    750
14 Spring    890
15 Spring    990
16 Spring    900
```

The graph of the weight change shows that there are two large outliers.

```
plot(bears.winter-bears.spring)
```



We'll first use the paired t-test to examine the change in weight of bears, where the same bears were weighed in winter and in spring. Then we'll analyze the data using the Wilcoxon signed rank test.

```
t.test(bears.winter, bears.spring, paired=TRUE)
```

```
    Paired t-test

data:  bears.winter and bears.spring
t = 2.274, df = 7, p-value = 0.05714
alternative hypothesis: true mean difference is not equal to 0
95 percent confidence interval:
  -4.460666 228.210666
sample estimates:
mean difference
        111.875
```

The paired t-test gives p = 0.057, indicating that bears do not lose weight while hibernating.

```
wilcox.test(bears.winter, bears.spring, paired=TRUE)
```

```
    Wilcoxon signed rank exact test
```

```
data:  bears.winter and bears.spring
V = 36, p-value = 0.007813
alternative hypothesis: true location shift is not equal to 0
```

The Wilcoxon signed rank test gives a significant result, p = 0.0078, indicating that bears lose weight during winter hibernation.

Notice that all the bears actually lose weight. Using the paired t-test, the p-value is not significant, and we conclude that bears don't lose weight. The Wilcoxon signed rank test gives us p =0.0078, so we reject the null hypothesis, and conclude that bears do lose weight.

5.5 Rank alternatives to one-way ANOVA: Kruskal-Wallis and Rfit

One-way ANOVA allows us to compare two or more treatment groups. The Kruskal-Wallis test is one rank alternative. Another alternative is the rank-based oneway.rfit function from the package Rfit.

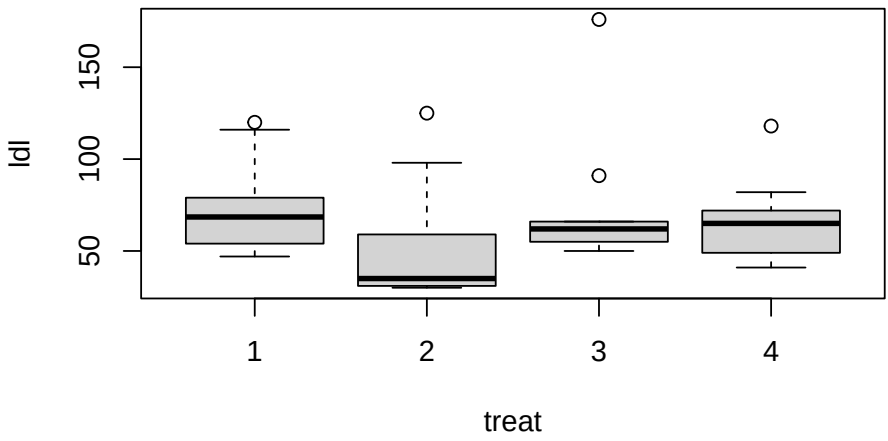
We'll use the quail data set from the Rfit package. Thirty-nine quail were randomized to one of 4 treatments for lowering low density lipoprotein (ldl). The variable "treat" is a factor. The variable ldl is the low density lipoprotein.

```
# install.packages("Rfit") # if not already installed
library(Rfit)
```

```
head(quail, 10) # display the first 10 rows of the quail data
set
```

```
   treat ldl
1      1  52
2      1  67
3      1  54
4      1  69
5      1 116
6      1  79
7      1  68
8      1  47
9      1 120
10     1  73
```

```
with(quail, boxplot(ldl ~ treat))
```



The boxplots suggest that there are differences among the 4 treatment groups. But there are large outliers that will adversely affect t-tests or ANOVA.

Here are the median values for each treatment.

```
aggregate(ldl~treat, quail, median)
```

```
  treat  ldl
1     1 68.5
2     2 35.0
3     3 62.0
4     4 65.0
```

We'll begin with a standard one-way analysis of variance.

```
quail.lm.fit = with(quail, lm(ldl ~ treat))
anova(quail.lm.fit)
```

```
Analysis of Variance Table

Response: ldl
          Df Sum Sq Mean Sq F value Pr(>F)
treat      3   3189 1062.85  1.1433 0.3451
Residuals 35  32536  929.61
```

The one-way analysis of variance indicates that there is no difference among the 4 treatments ($p = 0.345$).

Next try the Kruskal-Wallis test.

```
with(quail, kruskal.test(ldl, treat))
```

```
    Kruskal-Wallis rank sum test

data:  ldl and treat
Kruskal-Wallis chi-squared = 7.1879, df = 3, p-value = 0.06614
```

The Kruskal-Wallis test approaches significance, with $p = 0.066$.

Alternatively, we can use the function oneway.rfit, from the Rfit package, to perform a rank-based analysis of the quail data.

```
quail.oneway.rfit = with(quail, oneway.rfit(ldl, treat))
quail.oneway.rfit
```

```
Call:
oneway.rfit(y = ldl, g = treat)

Overall Test of All Locations Equal

Drop in Dispersion Test
F-Statistic     p-value
   3.916944    0.016394


    Pairwise comparisons using Rfit

data:  ldl and treat

  1      2      3
2 0.0046 -      -
3 0.6315 0.0157 -
4 0.5599 0.0243 0.9069

P value adjustment method: none
```

The oneway.rfit function indicates that there is a significant difference among the quail treatment groups p = 0.016394. The oneway.rfit output also shows pairwise comparisons that test for differences among the 4 treatment groups. By default, there is no p-value adjustment for multiple hypothesis testing. We can request estimates of the differences between each pair of treatments with confidence intervals adjusted for multiple comparisons by specifying method = "tukey".

```
summary(quail.oneway.rfit, method="tukey")
```

```
Multiple Comparisons
Method Used  tukey

  I J Estimate  St Err Lower Bound CI Upper Bound CI
1 1 2      -25 8.26704      -47.29541       -2.70459
2 1 3       -4 8.26704      -26.29541       18.29541
3 1 4       -5 8.49358      -27.90636       17.90636
4 2 3      -21 8.26704      -43.29541        1.29541
5 2 4      -20 8.49358      -42.90636        2.90636
6 3 4        1 8.49358      -21.90636       23.90636
```

For the pairwise comparisons, if the lower and upper bound CI do not include zero then the pair of treatments differ after Tukey correction for multiple hypothesis testing. Thus, in the first row, which compares treatments 1 and 2, the lower and upper confidence bounds are -47.29 to -.27, which does not include zero, so the difference is significant.

5.6 Do the Wilcoxon tests and Kruskal-Wallis test compare the medians of groups?

If the treatment groups do not differ in shape, but differ only by a shift in their location, then the Wilcoxon tests and Kruskal-Wallis compare the medians of the treatment groups. If the treatment groups differ in shape and location, these rank tests compare the entire distributions of the groups (that is, the ranks of all the observations, not just the median).

You can construct two treatment groups that have identical medians, but that differ in their rank sums, and will give significant differences in the Wilcoxon or Kruskal-Wallis tests.

For the following data, the medians of the groups are both zero but the standard deviations (shape) are different.

```
Group.A=c(0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
          0,0,0,0,0,0,0,0,0,0,0,0,1,1,1,1,1,1,1,1,1,1,1,1,
          1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1)

Group.B=c(0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
          0,0,0,0,0,0,0,0,0,0,0,0,2,2,2,2,2,2,2,2,2,2,2,2,
          2,2,2,2,2,2,2,2,2,2,2,2,2,2,2,2,2,2,2,2)

median(Group.A) # 0
```

```
[1] 0
```

```
median(Group.B) # 0
```

```
[1] 0
```

```
sd(Group.A) # 0.5
```

```
[1] 0.5028453
```

```
sd(Group.B) # 1.0
```

```
[1] 1.005691
```

```
wilcox.test(Group.A, Group.B)
```

```

    Wilcoxon rank sum test with continuity correction

data:  Group.A and Group.B
W = 1800, p-value = 0.01428
alternative hypothesis: true location shift is not equal to 0
```

The Wilcoxon rank sum test gives p = 0.014, indicating that the two groups are significantly different. This result shows that, although group A and group B have identical medians, they differ in scale (standard deviation), and differ significantly according to the Wilcoxon test. In this situation, the Wilcoxon test is said to be a test for "stochastic dominance", which means that one group tends to have higher values than the other group. A caveat is that the calculation of the p-value for the Wilcoxon test assumes treatment groups do not differ in scale but differ only by a shift in their location.

5.7 When should I use t-tests and ANOVA versus rank methods?

T-tests and the classical rank methods we looked at in this chapter may sometimes be sufficient to get good results. Their weakness is that these tests only allow for one explanatory variable. For most analyses, you are more likely to get $p < 0.05$ if you include additional explanatory variables in an analysis of variance, or if you use the robust and rank-based methods described in the previous and next chapters that allow for more than one explanatory variable.

The Wilcoxon, Kruskal-Wallis tests don't require the assumption of normal distributions, and are not affected by outliers, so why don't we always use them instead of t-tests and one-way ANOVA? There are several considerations.

If the data are exactly normally distributed, then ANOVA and t-tests have slightly more power to detect differences than do the standard versions of the rank tests. However, even with small or moderate deviations from normality, the rank tests will usually have power equal to or greater than the power of the tests that assume normality. When the data within treatment groups is substantially non-normal, the rank tests will pretty reliably have power equal to or greater than the tests that assume normality.

When you have a very small sample size (5 or less per group) it may be mathematically nearly impossible to get a significant p-value using a rank test. But it may be possible to get significance using a t-test or other parametric test.

One criticism of using rank methods used to be the difficulty of calculating confidence intervals. Before modern computers existed, getting confidence intervals for rank methods was harder than for methods that assume a normal distribution. But with today's computers it is easy to get confidence intervals for rank methods.

Another criticism of rank methods is the difficulty in understanding the effect size, particularly among readers who are not familiar with the Hodges-Lehmann estimator.

6 If you have non-normal data and more than one explanatory variable: Rank methods for multiple explanatory variables

Data that are not normally distributed can cause loss of power and poor p-values for t-test, analysis of variance and multiple regression. The Wilcoxon rank sum test and Kruskal-Wallis help if you only have one explanatory variable, but what if you have more than one variable that affects the outcome? In this chapter we'll look at rank methods that allow for multiple explanatory variables.

Load these libraries to run the examples in this chapter.

```
library(tidyverse)
library(coin)
library(Rfit)
library(rankFD)
```

6.1 Overview of rank methods for multiple explanatory variables

Rank analysis is an active area of statistics research. Several R packages and tools are available. We'll focus on these two:

- Van Elteren stratified Wilcoxon test (coin package)
- Rank-based regression (Rfit package)

6.1.1 Van Elteren stratified Wilcoxon test with coin

The van Elteren stratified Wilcoxon test was first published in 1960. Among its advantages are the facts that it is fairly widely known and has been used for FDA submissions and in articles published in the major medical and scientific journals. It has some disadvantages that we'll note shortly. It is available in standard statistics packages such as SAS and R. We'll use the van Elteren test from the R package coin.

6.1.2 Rank-based regression with Rfit

Rfit is built around rank-based regression as an alternative to ordinary least squares regression. The Rfit package accompanies the textbook "Nonparametric Statistical Methods Using R" by John Kloke and Joseph W. McKean (J. Kloke and McKean 2015). Think of it as a rank-based alternative to lm(). The appendix to this chapter gives a brief comparison of rfit versus traditional least-squares regression.

6.1.3 rankFD for Rank-based analysis of variance and factorial design

The rankFD package accompanies the textbook "Rank and Pseudo-Rank Procedures for Independent Observations in Factorial Designs" by Brunner (Brunner et al. 2018).

rankFD is built around nonparametric hypothesis testing in factorial designs. It is particulalry useful if you are looking for a rank-based alternative for factorial analysis of variance and experiment design.

6.2 Example data set with outliers

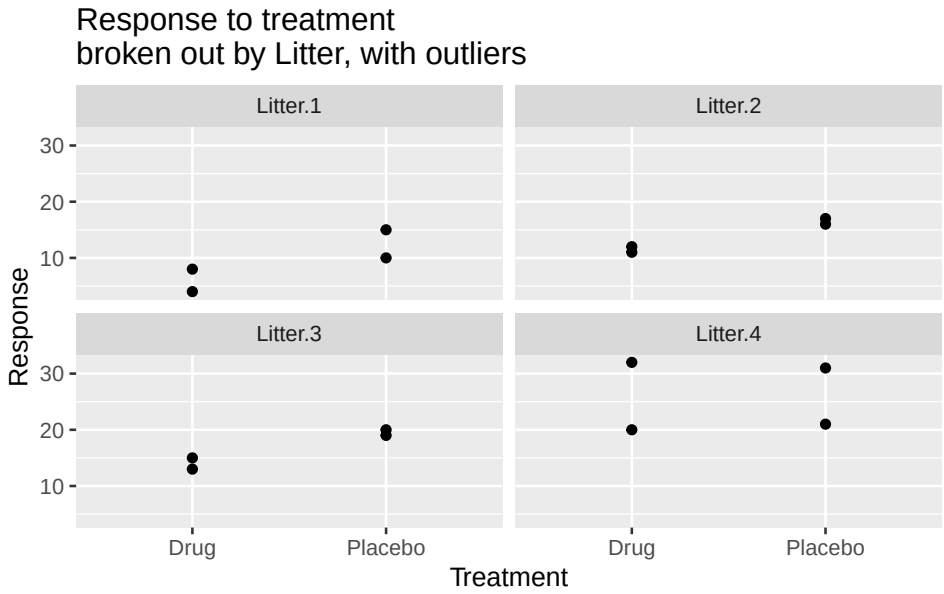
We'll use this anova.2way.data.outlier data set for some of the examples.

```
anova.2way.data.outlier = tibble::tribble(
~Treatment, ~Litter, ~Response,
  "Drug", "Litter.1", 4,
  "Drug", "Litter.1", 8,
  "Drug", "Litter.2", 11,
  "Drug", "Litter.2", 12,
```

```
  "Drug", "Litter.3", 13,
  "Drug", "Litter.3", 15,
  "Drug", "Litter.4", 20,
  "Drug", "Litter.4", 32,
  "Placebo", "Litter.1", 10,
  "Placebo", "Litter.1", 15,
  "Placebo", "Litter.2", 16,
  "Placebo", "Litter.2", 17,
  "Placebo", "Litter.3", 19,
  "Placebo", "Litter.3", 20,
  "Placebo", "Litter.4", 21,
  "Placebo", "Litter.4", 31
)

# van Elteren stratified Wilcoxon test from the coin R package

# requires that covariates are defined to be factors
anova.2way.data.outlier$Litter =
as.factor(anova.2way.data.outlier$Litter)
anova.2way.data.outlier$Treatment =
as.factor(anova.2way.data.outlier$Treatment)

str(anova.2way.data.outlier)
```

```
tibble [16 x 3] (S3: tbl_df/tbl/data.frame)
 $ Treatment: Factor w/ 2 levels "Drug","Placebo": 1 1 1 1 1 1
 1 1 2 2 ...
 $ Litter   : Factor w/ 4 levels "Litter.1","Litter.2",..: 1 1
 2 2 3 3 4 4 1 1 ...
 $ Response : num [1:16] 4 8 11 12 13 15 20 32 10 15 ...
```

```
ggplot(anova.2way.data.outlier, aes(y=Response, x=Treatment))
+ geom_point() +
  facet_wrap(~Litter) +
ggtitle("Response to treatment  \nbroken out by Litter, with
outliers")
```



The outliers in litter 4 are evident, as is the large difference in response between the litters.

6.3 Analysis using standard anova

Start with an ordinary two-way ANOVA for this data.

```
lm.2way.outlier = lm(Response ~ Litter + Treatment,
  data= anova.2way.data.outlier)
anova(lm.2way.outlier)
```

```
Analysis of Variance Table

Response: Response
          Df Sum Sq Mean Sq F value    Pr(>F)
Litter     3 596.50 198.833 12.7718 0.0006627 ***
Treatment  1  72.25  72.250  4.6409 0.0542426 .
Residuals 11 171.25  15.568
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

The ordinary two-way ANOVA confirms the additional variability due to litter ($p = 0.00066$) but treatment is not significant ($p = 0.054$).

6.4 Rank-based analysis using the van Elteren stratified Wilcoxon test

The syntax for the van Elteren stratified Wilcoxon test is to specify the usual formula "Response ~ Treatment" followed by a vertical bar "|" and the stratification variable, in this case "Litter". As the code below shows, this gives "Response ~ Treatment | Litter". Note also that the van Elteren stratified Wilcoxon test from the R package coin requires that the explanatory variables are defined to be factors.

```
# van Elteren stratified Wilcoxon test
wilcox_test(Response ~ Treatment | Litter,
                  data         = anova.2way.data.outlier,
                  distribution = "asymptotic")
```

```
    Asymptotic Wilcoxon-Mann-Whitney Test

data:  Response by
     Treatment (Drug, Placebo)
     stratified by Litter
Z = -2.636, p-value = 0.00839
alternative hypothesis: true mu is not equal to 0
```

The van Elteren stratified Wilcoxon test gives p = 0.008 for treatment, stratifying for Litter, which is an improvement over the non-significant result for the two-way anova.

6.5 Rank-based analysis using the Rfit package

Another alternative for rank-based analysis is the Rfit package. To run these examples, install and load the package with this code:

```
# install.packages("Rfit") # if not already installed
library(Rfit)
```

Here is the analysis of the outlier data, with litter and treatment as the two explanatory variables. The function rfit() performs the analysis.

```
anova.2way.data.outlier.rfit = with(anova.2way.data.outlier,
  rfit(Response ~ Litter + Treatment))
summary(anova.2way.data.outlier.rfit)
```

```
Call:
rfit.default(formula = Response ~ Litter + Treatment)

Coefficients:
                 Estimate Std. Error t.value   p.value
(Intercept)        7.2500     2.0287  3.5737  0.004366 **
LitterLitter.2     4.2500     2.5213  1.6857  0.119987
LitterLitter.3     7.2500     2.5213  2.8755  0.015092 *
LitterLitter.4    16.0000     2.5213  6.3460 5.475e-05 ***
TreatmentPlacebo   5.0000     1.7828  2.8046  0.017135 *
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Multiple R-squared (Robust): 0.7677992
Reduction in Dispersion Test: 9.0932 p-value: 0.0017
```

The rank-based rfit test indicates that treatment has a significant effect ($p = 0.017$). Compare this result to the conventional two-way ANOVA result of $p = 0.054$, which indicated that treatment was not significant.

The output also indicates that there are significant differences among the litters.

We could have started the analysis with an interaction test, using the Rfit function raov. The formula "Response ~ Litter * Treatment" specifies the interaction test.

```
with(anova.2way.data.outlier, raov(Response ~ Litter *
Treatment))
```

```

Robust ANOVA Table
                 DF       RD  Mean RD       F p-value
Litter            3 55.90170 18.63390 6.69423 0.01423
Treatment         1  7.37902  7.37902 2.65092 0.14214
Litter:Treatment  3  1.11803  0.37268 0.13388 0.93711
```

The interaction test is not significant ($p = 0.937$), indicating that there is no interaction between treatment and litter. So, the model without interaction was suitable.

We can also examine the results for rfit for the data with no outliers.

```
anova.2way.data = tibble::tribble(
~Treatment, ~Litter, ~Response,
  "Drug", "Litter.1", 4,
  "Drug", "Litter.1", 8,
  "Drug", "Litter.2", 11,
  "Drug", "Litter.2", 12,
  "Drug", "Litter.3", 13,
  "Drug", "Litter.3", 15,
  "Drug", "Litter.4", 20,
  "Drug", "Litter.4", 21,
  "Placebo", "Litter.1", 10,
  "Placebo", "Litter.1", 15,
  "Placebo", "Litter.2", 16,
  "Placebo", "Litter.2", 17,
  "Placebo", "Litter.3", 19,
  "Placebo", "Litter.3", 20,
  "Placebo", "Litter.4", 21,
  "Placebo", "Litter.4", 23
)
```

```
summary(rfit(Response ~ Litter + Treatment, data =
anova.2way.data))
```

```
Call:
rfit.default(formula = Response ~ Litter + Treatment, data =
anova.2way.data)

Coefficients:
                 Estimate Std. Error t.value   p.value
(Intercept)        7.2500     1.4574  4.9746  0.000419 ***
LitterLitter.2     4.5000     1.7165  2.6216  0.023761 *
LitterLitter.3     7.5000     1.7165  4.3693  0.001119 **
LitterLitter.4    12.0000     1.7165  6.9909 2.297e-05 ***
TreatmentPlacebo   4.5000     1.2138  3.7075  0.003456 **
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Multiple R-squared (Robust): 0.8141111
Reduction in Dispersion Test: 12.04378 p-value: 0.00052
```

We see that, for the data with no outliers, the p-value for the rfit method (p = 0.003456) is similar to the p-value for the classical analysis of variance (p = 0.000545), although not quite so good.

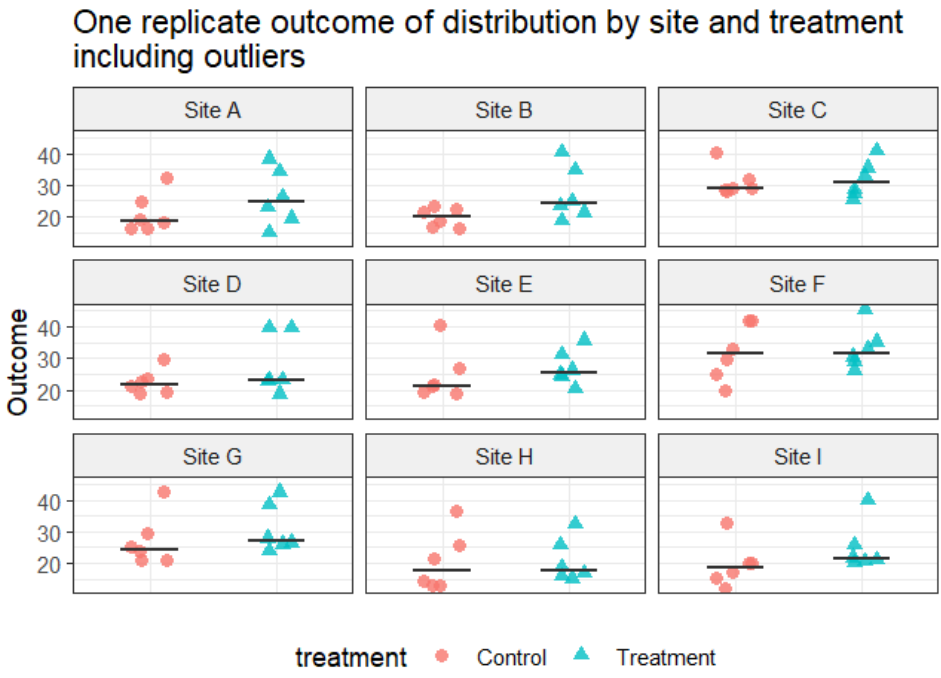
6.6 Compare power for Wilcoxon, stratified Wilcoxon and rfit: data with outliers

Let's run some simulations to compare power for these rank methods. We'll start with a data set that has outliers. Code for the simulations is on the book website.

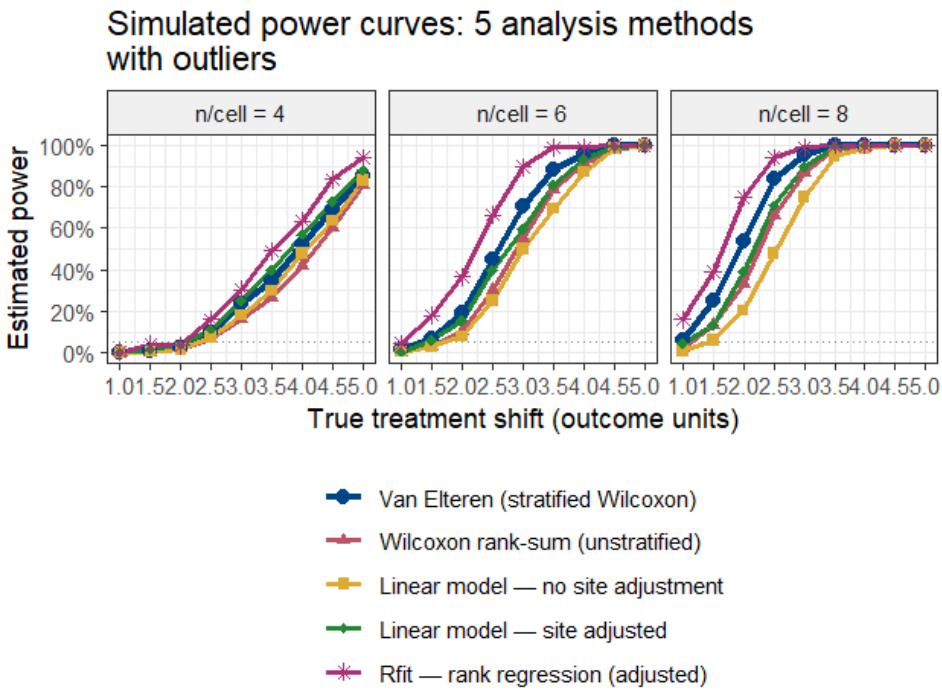
For this first simulation, we'll assume the following.

- Two group: "Control" and "Treatment"
- Response (y) variable is "Outcome"
- Study is performed at 9 sites. Sites are a source of variation.
- The number of observations per site in each group is 4, 6 or 8.
- Within each site, within each treatment group, data is randomly sampled from a normal distribution (mean = 20, sd = 4), with the addition of 2 outliers each to the "Control" and "Treatment" groups within each site.
- Within site the Treatment group is shifted relative to the Control group. We use a range of shift values from 1 to 5 by 0.5

The graph below shows the data for one simulation. Notice both the differences among the sites and the differences between treatment groups within each site.



The graph below shows the power curves comparing the analysis methods over many simulations.



For this simulated data set with an important covariate (site) and outliers, the methods that ignore the covariate (t-test and Wilcoxon rank sum) have the lowest power. The linear model that adjusts for the site covariate has more power, but lower power than the stratified Wilcoxon and rfit, both of which adjust for the site covariate and handle the outliers without inflating the variance.

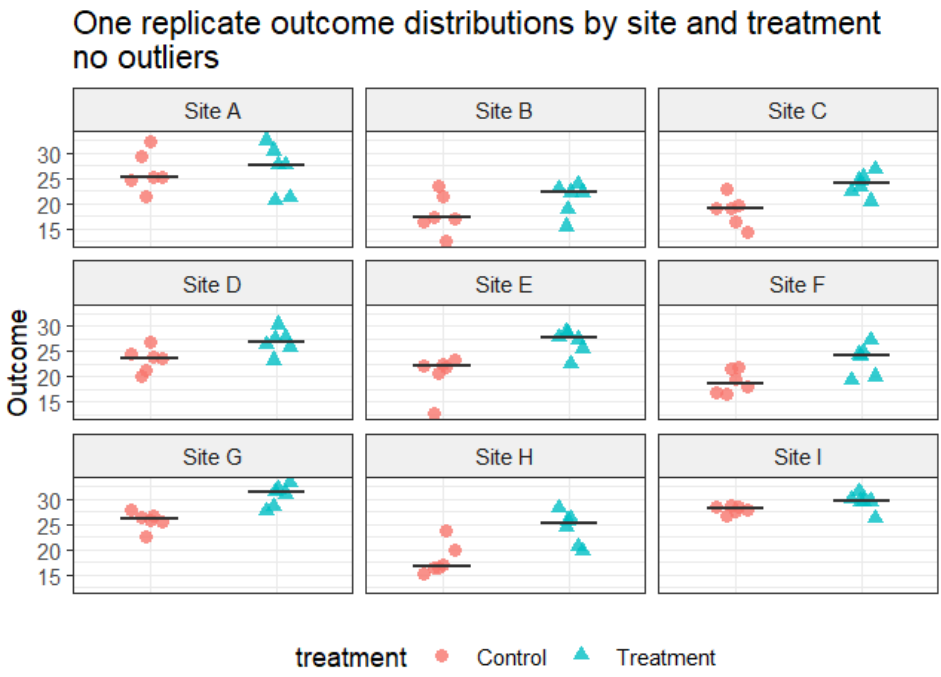
How does the power compare for data without outliers? We'll look at that next.

6.7 Compare power for Wilcoxon, stratified Wilcoxon and rfit: no outliers

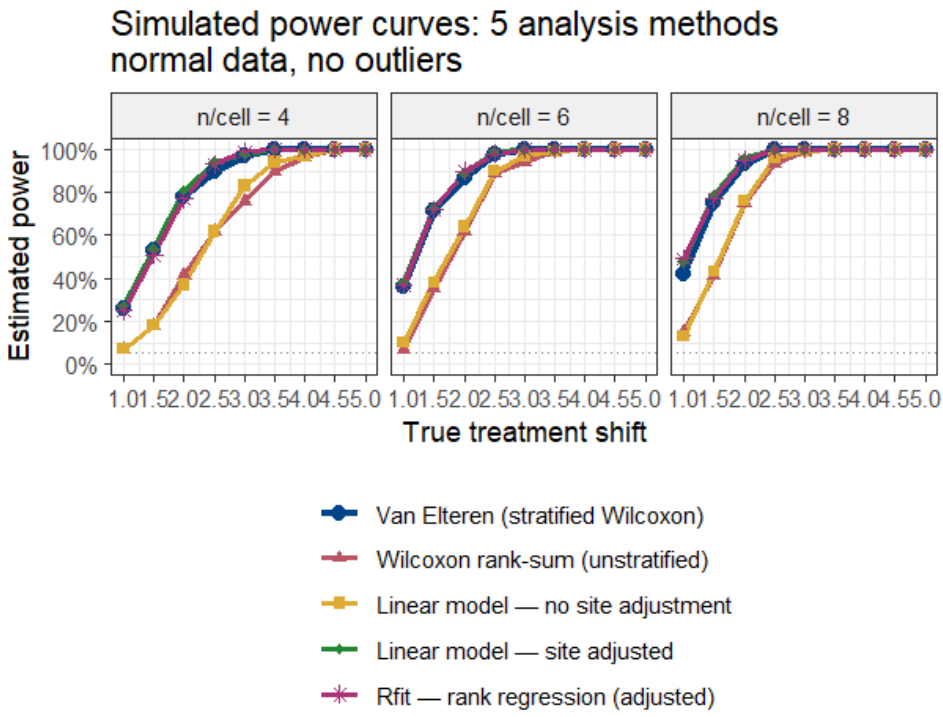
Now we'll compare the power for the 5 methods for data with no outliers. The data is the same as for the simulation above, except that no outliers are added.

The graph below shows the data for one simulation. Notice both the differences among the sites and the differences between treatment groups within

each site.



The graph below shows the power curves comparing the analysis methods over many simulations.



For this simulated data set with an important covariate (site) but no outliers, the two methods that ignore the site covariate (linear model with no site adjustment and Wilcoxon rank sum) have the lowest power. The stratified Wilcoxon, rfit, and linear model that adjust for the site covariate all have similar power. For data that are normally distributed with no outliers, rfit is expected to have 95% of the power of the linear model. So the results seen here are not surprising.

6.8 Limitations of the rank methods

Every method (including t-tests, anova, and linear regression) has assumptions and limitations. Here we note limitations of the rank methods considered in this chapter.

6.8.1 Limitations of the van Elteren stratified Wilcoxon test

The van Elteren test (1960) is a stratified extension of the Wilcoxon–Mann–Whitney rank-sum test. It combines within-stratum rank statistics

into a single weighted statistic, controlling one or more stratification factors. While useful, the procedure carries several important limitations. These are described in some detail in the textbook by Brunner (Brunner et al. 2018).

Assumes uniform effect cross strata

The test combines stratum-level rank statistics by weighting them, implicitly assuming a common treatment effect across all strata. If the true effect varies considerably from stratum to stratum (there is a treatment-by-stratum interaction), the single combined p-value can be misleading. It is a good idea to look at the within-stratum results before relying solely on the overall test.

Reduced power with many small strata

When the number of strata is large relative to the total sample size, so that many strata contain only a handful of observations, the within-stratum rank information becomes very sparse. The test can lose power compared to an unstratified Wilcoxon test, or even compared to a parametric approach such as a mixed model. A common guideline is to avoid strata with fewer than five observations per treatment arm.

Ties require careful handling

Like all rank-based methods, the van Elteren test requires an adjustment when tied observations are present. Standard mid-rank tie-breaking is incorporated into the usual variance formula, but with heavy ties (for example, an ordinal outcome with only a few levels) the normal approximation to the null distribution can be poor, inflating the Type I error rate. In such cases a permutation or exact version of the test should be considered.

No built-in effect size estimate

The test produces a p-value, but it does not directly provide an effect-size estimate or confidence interval. To quantify the magnitude of the treatment difference, a companion measure such as the stratified Hodges–Lehmann location shift must be computed separately. Reporting a p-value alone may be insufficient for clinical or scientific interpretation.

Asymptotic validity requires adequate sample size

The standard test relies on a large-sample normal approximation. With very small per-stratum sample sizes the approximation may be unreliable. Exact conditional permutation tests are available but are computationally intensive, and software implementations vary in whether they offer this option.

Sensitive to stratum definition

The stratification scheme should be defined before the analysis and should reflect genuine a priori prognostic factors. Post-hoc choice of strata or collapsing / splitting strata after seeing the data inflates the Type I error. Regulatory guidance (such as ICH E9) requires pre-specification of the stratification factors in the statistical analysis plan.

Practical guidance

None of these limitations invalidates the van Elteren test. It remains a widely-used, nonparametric option for stratified two-group comparisons in clinical trials and observational studies. The key safeguards are to pre-specify the stratification factors and weighting scheme, verify the homogeneity-of-effect assumption, ensure strata are not too small, and supplement the p-value with a suitable effect-size estimate and confidence interval.

6.8.2 Limitations of Rfit

The rfit package (J. D. Kloke and McKean 2012) implements rank-based estimation and inference for linear models in R. It fits regression coefficients by minimizing a rank-based dispersion function derived from Jaeckel (Jaeckel 1972), producing estimates that are robust to non-normality and heavy-tailed errors while retaining high efficiency relative to ordinary least squares. Despite these advantages, the approach has a number of limitations that practitioners should understand before adopting it as a primary analytical strategy.

Relies on asymptotic theory. May be inaccurate with small samples.

The standard errors, t-statistics, and F-statistics produced by rfit are justified by large-sample asymptotic arguments. In small datasets the normal and chi-squared approximations underpinning these quantities can be unreliable, leading to poorly calibrated p-values and confidence intervals.

Choice of score function can affect results

The method requires selection of a score function (e.g., Wilcoxon, sign, normal, Bent scores) that determines what distributional shape is being targeted. The default Wilcoxon scores are optimal when the error distribution is logistic. Choosing an inappropriate score function can reduce efficiency relative to least squares or relative to the correctly specified rank estimator.

Estimates are location shift, not conditional means

Rank-based regression via rfit estimates shift parameters in a location model rather than conditional means. Location shift as an effect descriptor is less familiar to many users than effect sizes based on means.

Ties in the outcome degrade performance

Like all rank-based methods, rfit uses mid-rank tie-breaking by default. When the outcome contains many ties, as is common with ordinal scales, rounded measurements, or clumped continuous data, the dispersion function becomes less sensitive to differences among tied values and efficiency is reduced

Limited availability and regulatory acceptance

The Rfit package (Kloke & McKean, 2012) is available in R but has no direct equivalent in SAS, Stata, or Python. Moreover, rank-based linear regression has little presence in FDA or EMA guidance documents, so its use as a primary analysis in confirmatory clinical trials requires additional scientific justification compared with OLS or prespecified nonparametric alternatives.

Practical guidance

Rfit is best suited to continuous, approximately symmetric outcomes in moderately large samples where robustness to heavy tails or outliers in the response is the primary concern. For small samples, sparse or ordinal outcomes, or settings requiring regulatory acceptance, alternative approaches should be considered alongside or instead of rfit.

6.9 Appendix: How does Rfit rank regression work?

The core idea

Most statistical analyses compare groups by calculating averages and asking how far apart those averages are. This works well when the data are well-behaved, but averages are can be greatly affected by a small number of unusually high or low outlier values.

Rfit takes a different approach. Instead of working with the raw measurements, it first converts every observation into its rank, that is, its position in the ordered list of all values. The lowest observation gets rank 1, the next gets rank 2, and so on. The analysis is then carried out on these ranks rather than the original numbers.

Because a rank only records where a value sits relative to the others, not how extreme it actually is, an outlier that is, say, ten times larger than the next highest value has no more influence than if it were just slightly larger. The ranks are far less sensitive to those extreme observations.

How Rfit differs from ordinary least squares regression

Standard linear regression finds the line of best fit by minimizing the sum of squared distances between each data point and the line. Squaring those distances means large errors caused by outliers count disproportionately and can distort the fitted line substantially.

Rfit uses the same idea of fitting a line through the data, but it minimizes a rank-based criterion instead of squared distances. In practical terms this means: - Outliers are handled gracefully. A single extreme value cannot pull the fitted line away from where the bulk of the data sit. - Covariates are still adjusted for. Just like standard regression, Rfit can include additional variables — such as study site or patient age — to account for factors that are not the primary focus of interest. - The output is familiar. Rfit produces coefficient estimates, standard errors, and p-values in the same format as ordinary regression, making results straightforward to interpret. - Efficiency is preserved. When data are perfectly normally Rfit is about 95% as efficient, meaning very little statistical power is sacrificed for the added robustness.

Why does this matter?

In many real-world datasets, such as biology experiments, clinical measurements, and laboratory assays, a small proportion of observations fall far outside the typical range. These might reflect genuine biological variation, data-entry errors, or rare events. Whatever the cause, they can mislead standard methods. Rfit provides a middle ground: it is more resistant to those unusual values than ordinary regression, yet unlike a simple non-parametric test it can still adjust for other variables and handle complex study designs. When data contain outliers, simulations show Rfit detecting real treatment effects at higher rates than either standard regression or unadjusted rank tests.

In summary

Rfit is a robust regression method that replaces the raw data with ranks before fitting a standard regression model. It combines the interpretability and flexibility of regression — including the ability to adjust for other variables — with the outlier-resistance of rank-based methods. It is most valuable when

data contain skewed distributions or extreme observations, and it sacrifices little when data are well-behaved.

How to interpret the rfit coefficients

The coefficients produced by Rfit are interpreted in the same way as those from ordinary linear regression. Each coefficient represents the estimated shift in the outcome associated with a one-unit increase in that predictor, after holding all other predictors constant.

For example, in a clinical trial model of the form "rfit(outcome ~ treatment + site)", the coefficient for treatment estimates the typical difference in outcome between the treatment and control arms, adjusted for site. A positive value means the treatment group tended to score higher; a negative value means lower. The site coefficients capture how much each site's typical outcome differs from the reference site, after accounting for treatment.

One subtlety is worth noting. Because Rfit works on ranks internally, the coefficient is technically an estimate of a shift parameter, that is, the amount by which the entire distribution of outcomes in one group is displaced relative to the other. It is not just a comparison of the means or a comparison of the medians.

The p-value and confidence interval accompanying each coefficient are interpreted in the usual way: the p-value tests whether the true shift is zero, and the confidence interval gives a plausible range for the size of the effect. Because Rfit is robust, these inferential summaries are more reliable than those from ordinary regression when the data contain outliers or are non-normally distributed.

7 Survival Analysis

In this chapter we'll look at ways to increase power and get better p-values in survival analyses:

- Avoid log rank tests. Use Cox proportional hazards regression to include covariates that affect survival.
- Omitting important covariates shrinks hazard ratios towards the null hypothesis value and reduces power.
- Balancing covariates across treatment arms is not sufficient. Include covariates in the analysis.
- Avoid testing for a difference in percent surviving at a specific timepoint. Keep the time information and use survival analysis to get better power.

Load these R libraries which are required for the examples.

```
library(survival)
library(ggplot2)
library(tidyverse)
library(survminer)
```

7.1 Avoid log rank tests. Use Cox proportional hazards regression to include additional variables that affect survival, which will increase power.

The standard version of the log rank test has several disadvantages for analysis of survival data compared to Cox proportional hazards regression.

- Has lower power
- Cannot include quantitative variables (such as age, weight, cholesterol)
- Does not provide a quantitative measure of the effect of treatment on survival

While there are versions of the log rank test that address these weaknesses, a simpler solution is to use Cox proportional hazards.

7.1.1 Example 1: Get better p-values by including a covariate that affects survival

Suppose we examine the effect of caloric restriction (CR) on the lifespan of a particular strain of mice and get the following data. Does treatment (CR or control) have a significant effect on survival? We'll first use a log rank test for treatment without covariates. Then we'll use Cox proportional hazards regression to include the covariate sex.

Here is the R code to generate the data.

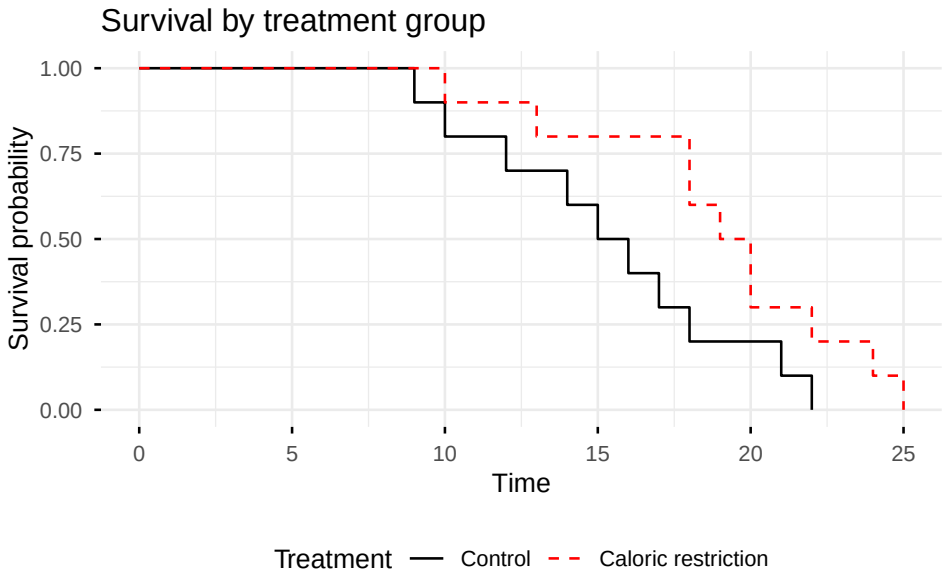
```
# mouse.lifespan.data
mouse.lifespan.data = data.frame(
  treatment = c("CR", "CR", "CR", "CR", "CR", "CR", "CR",
  "CR", "CR", "CR",
                "Control", "Control", "Control", "Control",
                "Control",
                "Control", "Control", "Control", "Control",
                "Control"),
  sex = c("M", "M", "M", "M", "M", "F", "F", "F", "F", "F",
          "M", "M", "M", "M", "M", "F", "F", "F", "F", "F"),
  months = c(10, 13, 18, 20, 22, 18, 19, 20, 24, 25,
             9, 10, 14, 17, 18, 12, 15, 16, 21, 22),
  event = c(1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
            1, 1, 1, 1, 1, 1, 1, 1, 1, 1))

mouse.lifespan.data
```

```
   treatment sex months event
1         CR   M     10     1
2         CR   M     13     1
3         CR   M     18     1
4         CR   M     20     1
5         CR   M     22     1
6         CR   F     18     1
7         CR   F     19     1
8         CR   F     20     1
9         CR   F     24     1
```

```
10         CR   F     25     1
11    Control   M      9     1
12    Control   M     10     1
13    Control   M     14     1
14    Control   M     17     1
15    Control   M     18     1
16    Control   F     12     1
17    Control   F     15     1
18    Control   F     16     1
19    Control   F     21     1
20    Control   F     22     1
```

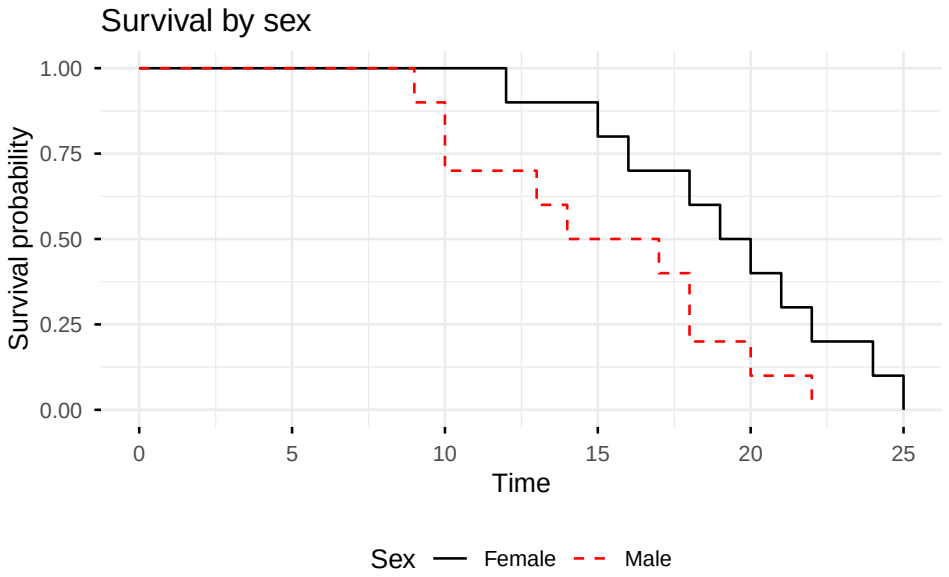
Examine the Kaplan-Meier plots by treatment.

```
# Create life table estimates broken out by treatment
mouse.lifespan.survfit.by.treatment = survfit(Surv(months,
event == 1) ~ treatment, data = mouse.lifespan.data)

# Plot using ggsurvplot
ggsurvplot(
  mouse.lifespan.survfit.by.treatment,
  data = mouse.lifespan.data,
  title = "Survival by treatment group",
  legend.title = "Treatment",
  legend = "bottom",
  conf.int = FALSE,
  pval = FALSE,
  risk.table = FALSE,
  legend.labs = c("Control", "Caloric restriction"),
  palette = c("black", "red"),
  linetype = c("solid", "dashed"),
  size = 0.5,
  ggtheme = theme_minimal()
)
```



Examine the Kaplan-Meier plots by sex.

```
# Create life table estimates broken out by sex
mouse.lifespan.survfit.by.sex = survfit(Surv(months, event ==
1) ~ sex,
                                        data =
                                        mouse.lifespan.data)

# Plot using ggsurvplot
ggsurvplot(
  mouse.lifespan.survfit.by.sex,
  data = mouse.lifespan.data,
  title = "Survival by sex",
  legend.title = "Sex",
  legend = "bottom",
  conf.int = FALSE,
  pval = FALSE,
  risk.table = FALSE,
  legend.labs = c("Female", "Male"),
  palette = c("black", "red"),
  linetype = c("solid", "dashed"),
  size = 0.5,
  ggtheme = theme_minimal()
)
```



The Kaplan-Meier plots suggest that both treatment and sex influence survival.

Here is the log rank test for the effect of treatment.

```
# Log rank test using survdiff
surv.diff.mouse.rx = survdiff(Surv(months, event == 1) ~
treatment,
                              data = mouse.lifespan.data)
surv.diff.mouse.rx
```

```
Call:
survdiff(formula = Surv(months, event == 1) ~ treatment, data
= mouse.lifespan.data)

                    N Observed Expected (O-E)^2/E (O-E)^2/V
treatment=Control 10       10     6.51     1.873      3.33
treatment=CR      10       10    13.49     0.904      3.33

 Chisq= 3.3  on 1 degrees of freedom, p= 0.07
```

The log rank test indicates that treatment is not significant, with p = 0.07.

Notice that the log rank test does not give us a measure of the effect size, such as a hazard ratio.

Here is the log rank test for the effect of sex.

```
surv.diff.mouse.sex = survdiff(Surv(months, event == 1) ~ sex,
                               data = mouse.lifespan.data)
surv.diff.mouse.sex
```

```
Call:
survdiff(formula = Surv(months, event == 1) ~ sex, data =
mouse.lifespan.data)

       N Observed Expected (O-E)^2/E (O-E)^2/V
sex=F 10       10    13.53     0.922      3.42
sex=M 10       10     6.47     1.930      3.42

 Chisq= 3.4  on 1 degrees of freedom, p= 0.06
```

The log rank test indicates that sex is not significant, with p = 0.06.

Now run the Cox regression, using the R function `coxph` from the survival package.

```
# Cox PH for treatment + sex
surv.diff.mouse.ph = coxph(Surv(months, event == 1) ~
treatment + sex,
                           data = mouse.lifespan.data)
summary(surv.diff.mouse.ph)
```

```
Call:
coxph(formula = Surv(months, event == 1) ~ treatment + sex,
data = mouse.lifespan.data)

  n= 20, number of events= 20

               coef exp(coef) se(coef)      z Pr(>|z|)
treatmentCR -1.0499    0.3500   0.5107 -2.056   0.0398 *
sexM         1.1047    3.0184   0.5141  2.149   0.0316 *
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

            exp(coef) exp(-coef) lower .95 upper .95
treatmentCR     0.350     2.8574    0.1286    0.9522
sexM            3.018     0.3313    1.1021    8.2673
```

```
Concordance= 0.701  (se = 0.051 )
Likelihood ratio test= 7.75  on 2 df,   p=0.02
Wald test            = 7.16  on 2 df,   p=0.03
Score (logrank) test = 7.78  on 2 df,   p=0.02
```

The Cox regression gives p = 0.0398 for treatment and p = 0.0316 for sex. Both are significant.

What do you prefer? Analyses with log rank tests for one variable at a time, showing that neither treatment nor sex is statistically significant? Or the analysis with the Cox regression showing that both treatment and sex are statistically significant?

Notice that the Cox regression also provides the hazard ratios for treatment and sex, which describe the magnitude of their effect on survival.

7.1.2 Example 2: Get better p-values by including two covariates that affect survival.

Suppose that we run a similar experiment on mouse survival, but now we account for possible effects of litter on lifespan.

Here is the R code to generate the data set.

```
# mouse.lifespan.litter.data
mouse.lifespan.litter.data = data.frame(
  treatment = c("CR", "CR", "CR", "CR", "CR", "CR", "CR",
  "CR", "CR", "CR",
                "Control", "Control", "Control", "Control",
                "Control",
                "Control", "Control", "Control", "Control",
                "Control"),
  sex = c("M", "M", "M", "M", "M", "F", "F", "F", "F", "F",
          "M", "M", "M", "M", "M", "F", "F", "F", "F", "F"),
  litter = c("A", "B", "C", "D", "E", "A", "B", "C", "D", "E",
             "A", "B", "C", "D", "E", "A", "B", "C", "D",
             "E"),
  months = c(13, 16, 17, 23, 20, 21, 19, 22, 22, 24,
             9, 13, 11, 14, 16, 19, 16, 17, 21, 23),
  event = c(1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
            1, 1, 1, 1, 1, 1, 1, 1, 1, 1))
```

Here are the data.

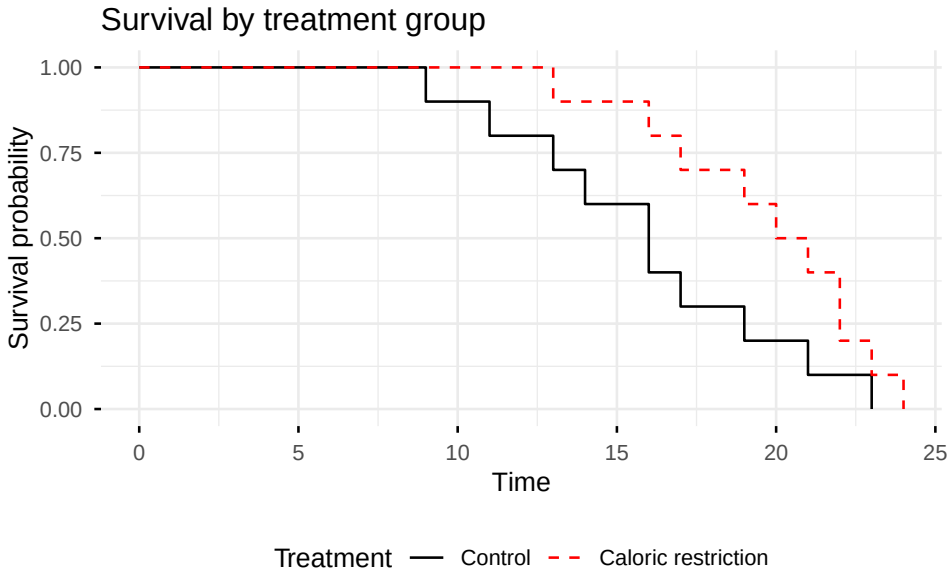
```
mouse.lifespan.litter.data
```

```
   treatment sex litter months event
1         CR   M      A     13     1
2         CR   M      B     16     1
3         CR   M      C     17     1
4         CR   M      D     23     1
5         CR   M      E     20     1
6         CR   F      A     21     1
7         CR   F      B     19     1
8         CR   F      C     22     1
9         CR   F      D     22     1
10        CR   F      E     24     1
11   Control   M      A      9     1
12   Control   M      B     13     1
13   Control   M      C     11     1
14   Control   M      D     14     1
15   Control   M      E     16     1
16   Control   F      A     19     1
17   Control   F      B     16     1
18   Control   F      C     17     1
19   Control   F      D     21     1
20   Control   F      E     23     1
```

Here is R code to create the Kaplan-Meier plots.

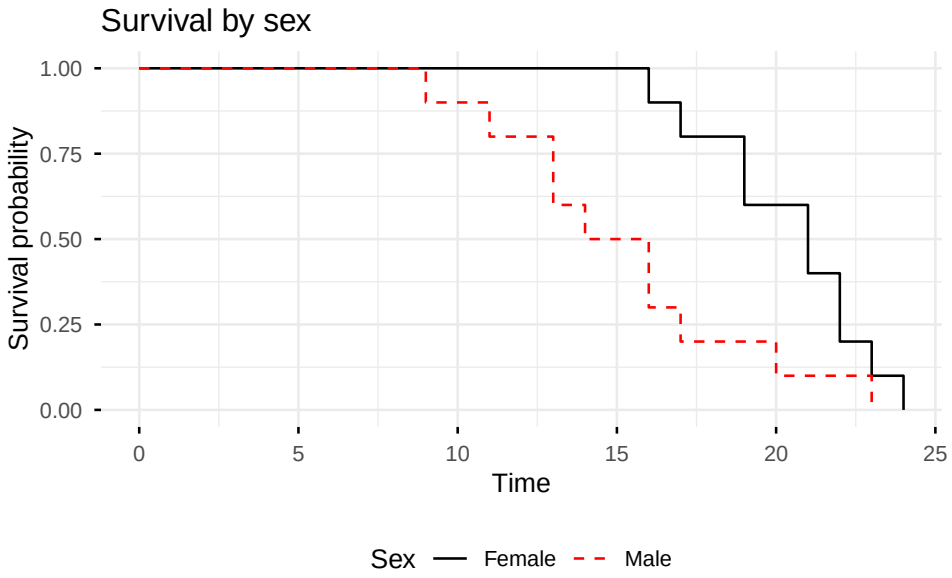
```
## KM plots

# Create life table estimates broken out by treatment
mouse.lifespan.litter.survfit.by.treatment = survfit(
  Surv(months, event == 1) ~ treatment, data =
  mouse.lifespan.litter.data)

# Plot using ggsurvplot
ggsurvplot(
  mouse.lifespan.litter.survfit.by.treatment,
  data = mouse.lifespan.litter.data,
  title = "Survival by treatment group",
  legend.title = "Treatment",
  legend = "bottom",
  conf.int = FALSE,
  pval = FALSE,
```

```
  risk.table = FALSE,
  legend.labs = c("Control", "Caloric restriction"),
  palette = c("black", "red"),
  linetype = c("solid", "dashed"),
  size = 0.5,
  ggtheme = theme_minimal()
)
```



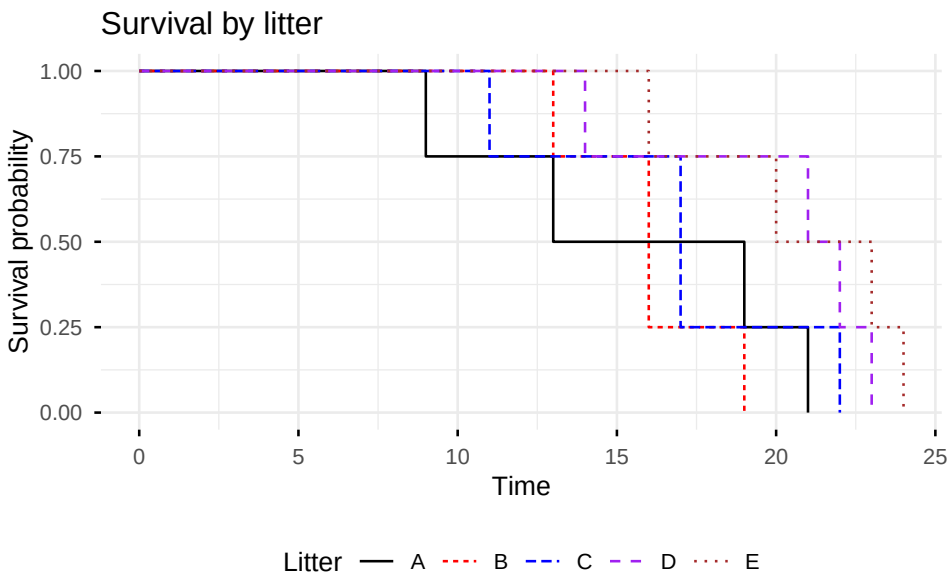

```
# Create life table estimates broken out by sex
mouse.lifespan.litter.survfit.by.sex = survfit(
  Surv(months, event == 1) ~ sex, data =
  mouse.lifespan.litter.data)

# Plot using ggsurvplot
ggsurvplot(
  mouse.lifespan.litter.survfit.by.sex,
  data = mouse.lifespan.litter.data,
  title = "Survival by sex",
  legend.title = "Sex",
  legend = "bottom",
  conf.int = FALSE,
  pval = FALSE,
  risk.table = FALSE,
```

```
  legend.labs = c("Female", "Male"),
  palette = c("black", "red"),
  linetype = c("solid", "dashed"),
  size = 0.5,
  ggtheme = theme_minimal()
)
```




```
# Create life table estimates broken out by litter
mouse.lifespan.litter.survfit.by.litter = survfit(
  Surv(months, event == 1) ~ litter, data =
  mouse.lifespan.litter.data)

# Plot using ggsurvplot
ggsurvplot(
  mouse.lifespan.litter.survfit.by.litter,
  data = mouse.lifespan.litter.data,
  title = "Survival by litter",
  legend.title = "Litter",
  legend = "bottom",
  conf.int = FALSE,
  pval = FALSE,
  risk.table = FALSE,
  legend.labs = c("A", "B", "C", "D", "E"),
```

```
  palette = c("black", "red", "blue", "purple", "brown"),
  linetype = "strata",
  size = 0.5,
  ggtheme = theme_minimal()
)
```



The Kaplan-Meier plots suggest that there are significant differences due to treatment and sex. There also appears to be some variation in lifespan among the litters, with litter A possibly having shorter lifespans than litters D and E.

Here is the log rank test for the effect of treatment.

```
# Log rank test using survdiff
surv.diff.mouse.rx = survdiff(Surv(months, event == 1) ~
treatment,
                              data =
                              mouse.lifespan.litter.data)
surv.diff.mouse.rx # p = 0.08
```

```
Call:
survdiff(formula = Surv(months, event == 1) ~ treatment, data
= mouse.lifespan.litter.data)

                    N Observed Expected (O-E)^2/E (O-E)^2/V
```

```
treatment=Control 10       10     6.65     1.682      3.11
treatment=CR      10       10    13.35     0.839      3.11

 Chisq= 3.1  on 1 degrees of freedom, p= 0.08
```

The log rank test indicates that treatment is not significant, with p = 0.08.

Here is the log rank test for the effect of sex.

```
surv.diff.mouse.sex = survdiff(Surv(months, event == 1) ~ sex,
                               data =
                               mouse.lifespan.litter.data)
surv.diff.mouse.sex # p = 0.02
```

```
Call:
survdiff(formula = Surv(months, event == 1) ~ sex, data =
mouse.lifespan.litter.data)

       N Observed Expected (O-E)^2/E (O-E)^2/V
sex=F 10       10    14.26      1.27      5.59
sex=M 10       10     5.74      3.16      5.59

 Chisq= 5.6  on 1 degrees of freedom, p= 0.02
```

The log rank test indicates that sex is significant, with p = 0.02.

Here is the log rank test for the effect of litter.

```
surv.diff.mouse.litter = survdiff(Surv(months, event == 1) ~
litter,
                                  data =
                                  mouse.lifespan.litter.data)
surv.diff.mouse.litter # p = 0.1
```

```
Call:
survdiff(formula = Surv(months, event == 1) ~ litter, data =
mouse.lifespan.litter.data)

         N Observed Expected (O-E)^2/E (O-E)^2/V
litter=A 4        4     2.36     1.139     1.484
litter=B 4        4     2.01     1.973     2.618
litter=C 4        4     3.04     0.302     0.416
litter=D 4        4     5.50     0.411     0.709
litter=E 4        4     7.08     1.343     2.962
```

```
 Chisq= 7.1  on 4 degrees of freedom, p= 0.1
```

The log rank test indicates that litter is not significant, with p = 0.1.

Now run the Cox regression, first including just treatment and sex.

```
# Cox PH for treatment + sex
surv.diff.mouse.ph = coxph(Surv(months, event == 1) ~
treatment + sex,
                           data = mouse.lifespan.litter.data)
summary(surv.diff.mouse.ph)
```

```
Call:
coxph(formula = Surv(months, event == 1) ~ treatment + sex,
data = mouse.lifespan.litter.data)

  n= 20, number of events= 20

               coef exp(coef) se(coef)      z Pr(>|z|)
treatmentCR -1.4091    0.2444   0.5597 -2.518  0.01181 *
sexM         1.6038    4.9719   0.5652  2.838  0.00455 **
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

            exp(coef) exp(-coef) lower .95 upper .95
treatmentCR    0.2444     4.0922   0.08159    0.7319
sexM           4.9719     0.2011   1.64216   15.0535

Concordance= 0.782  (se = 0.048 )
Likelihood ratio test= 11.34  on 2 df,   p=0.003
Wald test            = 9.61  on 2 df,   p=0.008
Score (logrank) test = 10.86  on 2 df,   p=0.004
```

The Cox regression gives p = 0.01181 for treatment and p = 0.00455 for sex. That is an improvement in the p-values for both variables compared to the log rank test, which gave a non-significant p = 0.08 for treatment and p = 0.02 for sex.

What if we also include litter in the Cox model? Here are the results.

```
# treatment + sex + litter
surv.diff.mouse.ph = coxph(Surv(months, event == 1) ~
treatment + sex + litter,
```

```
                            data = mouse.lifespan.litter.data)
summary(surv.diff.mouse.ph)
```

```
Call:
coxph(formula = Surv(months, event == 1) ~ treatment + sex +
    litter, data = mouse.lifespan.litter.data)

  n= 20, number of events= 20

                coef exp(coef) se(coef)      z Pr(>|z|)
treatmentCR -2.43139   0.08791  0.74825 -3.249 0.001156 **
sexM         3.00016  20.08872  0.84776  3.539 0.000402 ***
litterB      0.10214   1.10753  0.88230  0.116 0.907842
litterC     -0.39908   0.67094  0.80267 -0.497 0.619055
litterD     -3.02950   0.04834  1.10927 -2.731 0.006313 **
litterE     -3.02888   0.04837  0.98430 -3.077 0.002090 **
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

            exp(coef) exp(-coef) lower .95 upper .95
treatmentCR   0.08791   11.37470  0.020284    0.3810
sexM         20.08872    0.04978  3.813629  105.8196
litterB       1.10753    0.90291  0.196489    6.2428
litterC       0.67094    1.49045  0.139139    3.2353
litterD       0.04834   20.68693  0.005497    0.4251
litterE       0.04837   20.67403  0.007026    0.3330

Concordance= 0.89  (se = 0.028 )
Likelihood ratio test= 29.66  on 6 df,   p=5e-05
Wald test            = 18.07  on 6 df,   p=0.006
Score (logrank) test = 24.03  on 6 df,   p=5e-04
```

The Cox regression including both sex and litter as covariates gives p = 0.0012 for treatment, compared to p = 0.0118 for the Cox model with only sex as a covariate, and the non-significant p = 0.08 for treatment from the log rank test that did not include sex or litter as covariates.

What do you prefer? Analysis with the log rank test, showing that treatment is not statistically significant, p = 0.08? Or the analysis with the Cox regression including sex and litter as covariates which shows that treatment is statistically significant, p = 0.0012? Notice that we got this better result

without increasing the sample size or making any other change to the experiment. The only thing we had to do was include the covariates in the analysis.

7.2 Omitting important predictors shrinks hazard ratios towards the null hypothesis value and reduces power.

What happens when you run a survival analysis using treatment as the only variable, and you omit a covariate that is an important predictor of survival? Doing so will shrink the coefficient for treatment towards the null hypothesis value of beta = 0. It will shrink the hazard ratio for treatment towards the null hypothesis value, that is, towards hazard ratio = exp(beta) = exp(0) = 1. If you shrink the treatment coefficient beta closer to 0 and make the hazard ratio closer to 1.0 you reduce power and get worse p-values. Let's see an example to demonstrate what happens.

We'll examine the effect of treatment and age on time to an event. We generate data by sampling from an exponential distribution with the following known coefficients.

- Coefficient for treatment = beta1 = 1.0.
- Coefficient for age = beta2 = 0.5.
- Treatment takes values 0 or 1.
- Age is uniformly distributed between 3 and 18.

Here is R code to generate 10,000 observations from the distribution. The large sample size of 10,000 observations will provide accurate values for the estimates of the coefficients.

```
# Generate simulated data
set.seed(1)
n = 10000
treatment = c(rep(0, n/2), rep(1, n/2))
age = runif(n, min = 3, max = 18)  # Important continuous
predictor

beta1 = 1.0
beta2 = 0.2
```

```
# Hazard: h(t) = h0(t) * exp(beta1*treatment + beta2*age)
hazard = exp(beta1 * treatment + beta2 * age)
time = rexp(n, rate = hazard)
event = rep(1, n)      # No censoring

omit.predictor.data = data.frame(time, event, treatment, age)
```

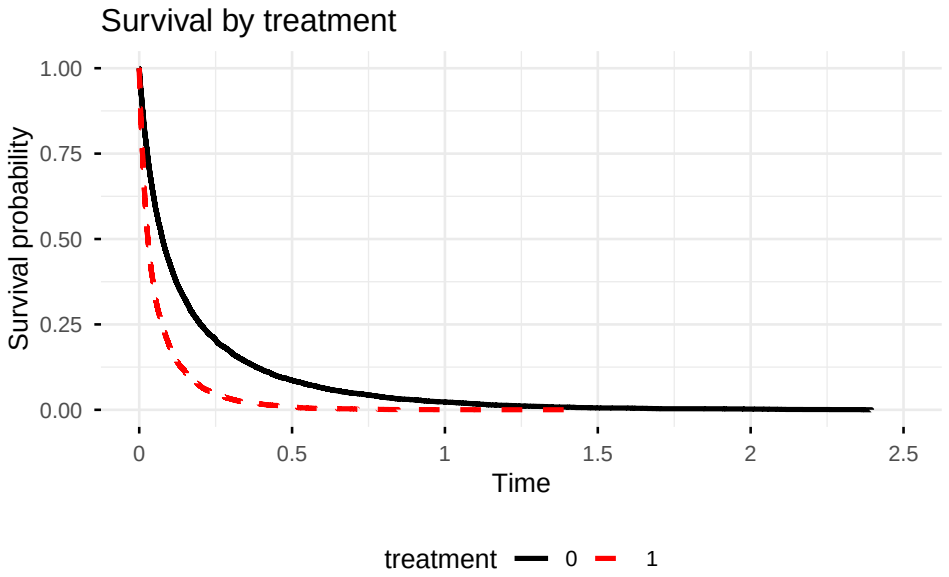
Here are the first few rows in the data set.

```
head(omit.predictor.data)
```

```
        time event treatment       age
1 0.52332837     1         0  6.982630
2 0.06348051     1         0  8.581858
3 0.04632830     1         0 11.592800
4 0.07599466     1         0 16.623117
5 0.17442875     1         0  6.025229
6 0.10528667     1         0 16.475845
```

Let's look at Kaplan-Meier plots for treatment and for age.

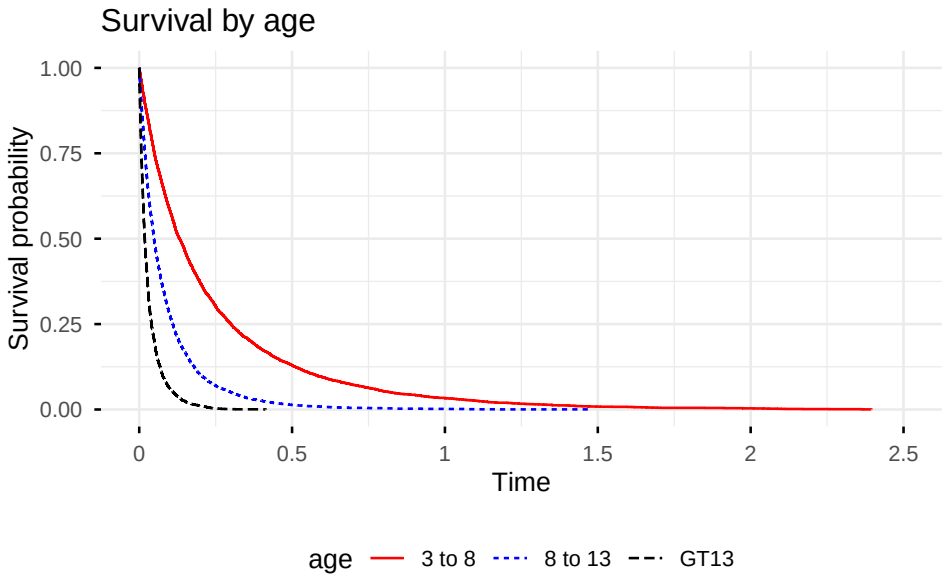
```
### KM plots

# Create life table estimates broken out by treatment
omit.predictor.survfit.by.treatment = survfit(Surv(time, event
== 1) ~ treatment, data = omit.predictor.data)

# Plot using ggsurvplot
ggsurvplot(
  omit.predictor.survfit.by.treatment,
  data = omit.predictor.data,
  title = "Survival by treatment",
  legend.title = "treatment",
  legend = "bottom",
  legend.labs = c(0, 1),
  palette = c("black", "red"),
  linetype = c("solid", "dashed"),
  linewidth = 0.5,
  ggtheme = theme_minimal()
)
```



We convert age to a categorical variable just for the Kaplan-Meier plot. Cox regression will use age as a continuous variable.

```
# Create age categories to use in the KM plot
omit.predictor.data = omit.predictor.data %>%
  mutate(age.categorical = case_when(
    age >= 3  & age < 8  ~ "3 to 8",
    age >= 8  & age < 13 ~ "8 to 13",
    age >= 13 & age < 18 ~ "GT13"))

# Create life table estimates broken out by age category
omit.predictor.survfit.by.age = survfit(Surv(time, event == 1)
~ age.categorical, data = omit.predictor.data)

# Plot using ggsurvplot
ggsurvplot(
  omit.predictor.survfit.by.age,
  data = omit.predictor.data,
  title = "Survival by age",
  legend.title = "age",
  legend = "bottom",
  legend.labs = c("3 to 8", "8 to 13", "GT13"),
  palette = c("red", "blue", "black"),
  linetype = "strata",
```

```
  size = 0.5,
  ggtheme = theme_minimal()
)
```



Now the Cox regression analyses.

First, the full model (both treatment and age).

```
# Fit Full Model (Correct)
model_full = coxph(Surv(time, event) ~ treatment + age, data =
omit.predictor.data)
summary(model_full)$coefficients["treatment", ]
```

```
      coef    exp(coef)     se(coef)           z     Pr(>|z|)
 0.98138095  2.66813826  0.02182259 44.97088160  0.00000000
```

In the full model (both treatment and age), the estimated coefficient is 0.98. HR is $\exp(0.98) = 2.668$. These values are very close to the true coefficient of 1.0, and the true hazard ratio of $\exp(1.0) = 2.718$.

Next, the reduced model (only treatment).

```
model_reduced = coxph(Surv(time, event) ~ treatment, data =
omit.predictor.data)
round(summary(model_reduced)$coefficients["treatment", ],
digits = 4)
```

```
    coef exp(coef)  se(coef)         z  Pr(>|z|)
  0.7025    2.0188    0.0210   33.5074    0.0000
```

In the reduced model (only treatment), the estimated coefficient is 0.70. HR is 2.02.

The reduced model gives values far from the true coefficient of 1.0 and the true hazard ratio of 2.718. The reduced model, omitting the important predictor variable age, shrinks the coefficient for treatment closer to 0 (from 0.98 to 0.70). It shrinks the hazard ratio closer to 1 (from 2.67 to 2.02).

What happens if we run the simulation with a sample size of 60 rather than 10,000? Here are the results from one such run. The R code is the same as above, with n = 60.

Here are results for n = 60 for the full model (both treatment and age).

```
# Create sample of 60 observations from omit.predictor.data
set.seed(123)
omit.predictor.data.60 =
omit.predictor.data[sample(nrow(omit.predictor.data), size =
60),]
```

```
# Fit Full Model (Correct)
model_full_60 = coxph(Surv(time, event) ~ treatment + age,
data = omit.predictor.data.60)
summary(model_full_60)$coefficients["treatment", ]
```

```
       coef    exp(coef)     se(coef)           z     Pr(>|z|)
0.870408354 2.387885758 0.286545640 3.037590645 0.002384777
```

In the full model (both treatment and age), the estimated coefficient is 0.87. HR is 2.39.

Next, the reduced model (only treatment).

```
model_reduced_60 = coxph(Surv(time, event) ~ treatment, data =
omit.predictor.data.60)
round(summary(model_reduced_60)$coefficients["treatment", ],
digits = 4)
```

```
    coef exp(coef)  se(coef)         z  Pr(>|z|)
  0.4159    1.5158    0.2629    1.5818    0.1137
```

In the reduced model (only treatment), the estimated coefficient is 0.42. HR is 1.51.

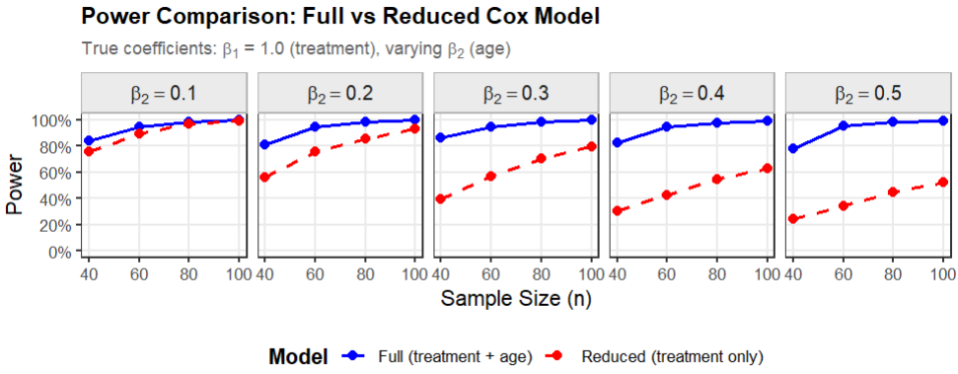
Compare these to the true coefficient of 1.0 and the true HR of 2.718. The model omitting age has greatly attenuated the treatment coefficient and HR.

Let's take a thousand samples of size n = 60 and see how often the full model gives p < 0.05 versus how often the reduced model gives p < 0.05. This will tell us how much effect omitting age has on the power for this analysis. The true treatment coefficient is still beta1 = 1.0. Here are results of running 1000 simulations.

Model	True coefficient of treatment beta1	True treatment coefficient of age beta2	Sample size	Power
Full	1.0	0.2	60	0.963
Reduced	1.0	0.2	60	0.739

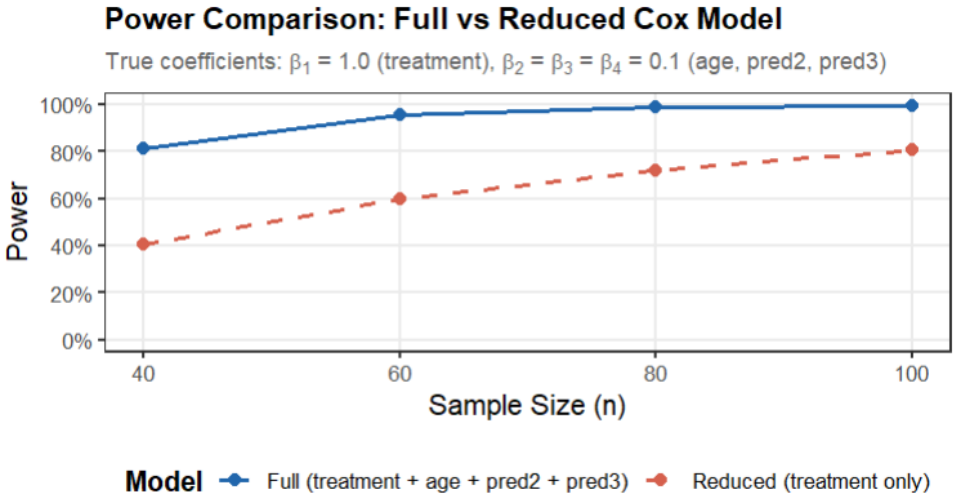
With a sample size of n = 60, the model including age has power = 0.963, while the model omitting age has power = 0.74. Which do you prefer for your research?

The degree to which omitting an important variable affects the coefficient for treatment depends on both the coefficient for the omitted variable and the sample size. Let's look at the power when we vary beta2, the coefficient for age, from 0.1 to 0.5 and vary sample size from n = 40 to 100. Here are the simulation results in a graph.



The graph shows that the loss of power caused by omitting an important predictor is larger when the (absolute) value of the omitted variable's coefficient is larger and when the sample size is smaller.

If you have only one omitted variable and it is not an important predictor (coefficient near 0), then it has less impact on the power for the treatment effect. However, if you have multiple omitted variables that each have a small effect, then they may collectively have a large impact on power for the treatment effect, as shown in the graph below. For the simulation shown in the graph, we assumed 3 predictor variables each with a (relatively small) beta of 0.1.



7.3 Balancing covariates across treatment arms is not sufficient. Include covariates in the analysis.

You may have been told that, if a covariate is balanced across the treatment arms, you do not need to include it in the analysis. This advice is wrong. Omitting an important predictor, even if it is perfectly balanced across treatment arms, will reduce power, give worse p-values, and shrink the hazard ratio towards the null hypothesis value. Here is an example.

Suppose we have the following data.

```
# balanced.data
balanced.data = data.frame(
```

```
  treatment = c("Drug", "Drug", "Drug", "Drug", "Drug",
  "Drug", "Drug", "Drug", "Drug", "Drug",
                "Control", "Control", "Control", "Control",
                "Control",
                "Control", "Control", "Control", "Control",
                "Control"),
  age = c("40", "50", "60", "70", "80", "40", "50", "60",
  "70", "80",
          "40", "50", "60", "70", "80", "40", "50", "60",
          "70", "80"),
  months = c(10, 13, 18, 20, 22, 18, 19, 20, 24, 25,
             9, 10, 14, 17, 18, 12, 15, 16, 21, 22),
  event = c(1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
            1, 1, 1, 1, 1, 1, 1, 1, 1, 1))

balanced.data
```

```
   treatment age months event
1       Drug  40     10     1
2       Drug  50     13     1
3       Drug  60     18     1
4       Drug  70     20     1
5       Drug  80     22     1
6       Drug  40     18     1
7       Drug  50     19     1
8       Drug  60     20     1
9       Drug  70     24     1
10      Drug  80     25     1
11   Control  40      9     1
12   Control  50     10     1
13   Control  60     14     1
14   Control  70     17     1
15   Control  80     18     1
16   Control  40     12     1
17   Control  50     15     1
18   Control  60     16     1
19   Control  70     21     1
20   Control  80     22     1
```

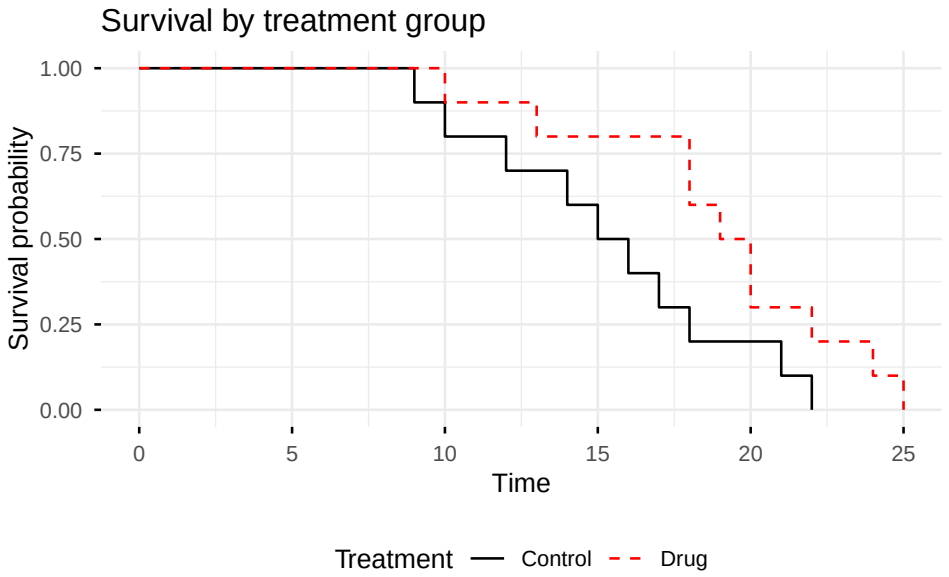
There are 10 control subjects and 10 drug subjects. Within each treatment group there are 2 subjects age 40, 2 age 50, and so on, so that age is perfectly balanced between the treatment arms.

```
with(balanced.data, table(age, treatment))
```

```
    treatment
age  Control Drug
  40       2    2
  50       2    2
  60       2    2
  70       2    2
  80       2    2
```

Here is the Kaplan-Meier plot.

```
## KM plot by treatment

# Create life table estimates broken out by treatment
balanced.survfit.by.treatment = survfit(Surv(months, event ==
1) ~ treatment, data = balanced.data)

# KM plot using ggsurvplot
ggsurvplot(
  balanced.survfit.by.treatment,
  data = balanced.data,
  title = "Survival by treatment group",
  legend.title = "Treatment",
  legend = "bottom",
  legend.labs = c("Control", "Drug"),
  palette = c("black", "red"),
  linetype = c("solid", "dashed"),
  size = 0.5,
  ggtheme = theme_minimal()
)
```



Here is the Cox analysis including only treatment in the model.

```
# Cox PH with only treatment
surv.diff.balanced.ph.treatment = coxph(Surv(months, event ==
1) ~ treatment,
                                        data = balanced.data)
summary(surv.diff.balanced.ph.treatment)
```

```
Call:
coxph(formula = Surv(months, event == 1) ~ treatment, data =
balanced.data)

  n= 20, number of events= 20

                coef exp(coef) se(coef)      z Pr(>|z|)
treatmentDrug -0.8448    0.4296   0.4863 -1.737   0.0823 .
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

              exp(coef) exp(-coef) lower .95 upper .95
treatmentDrug    0.4296      2.327    0.1657     1.114

Concordance= 0.625  (se = 0.062 )
Likelihood ratio test= 3.03  on 1 df,   p=0.08
```

```
Wald test            = 3.02  on 1 df,   p=0.08
Score (logrank) test = 3.18  on 1 df,   p=0.07
```

In the model including only treatment, treatment is not significant, p = 0.0823.

Here is the Cox analysis including age as well as treatment in the model.

```
# Cox PH with treatment + age
surv.diff.balanced.ph = coxph(Surv(months, event == 1) ~
treatment + age, data = balanced.data)
summary(surv.diff.balanced.ph)
```

```
Call:
coxph(formula = Surv(months, event == 1) ~ treatment + age,
data = balanced.data)

  n= 20, number of events= 20

                  coef exp(coef) se(coef)      z Pr(>|z|)
treatmentDrug -1.89972   0.14961  0.64949 -2.925 0.003445 **
age50         -0.78969   0.45398  0.75462 -1.046 0.295340
age60         -1.47309   0.22922  0.77571 -1.899 0.057561 .
age70         -3.60871   0.02709  1.05680 -3.415 0.000638 ***
age80         -4.49220   0.01120  1.22045 -3.681 0.000233 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

              exp(coef) exp(-coef) lower .95 upper .95
treatmentDrug   0.14961      6.684  0.041891    0.5343
age50           0.45398      2.203  0.103445    1.9924
age60           0.22922      4.363  0.050114    1.0484
age70           0.02709     36.918  0.003414    0.2149
age80           0.01120     89.318  0.001024    0.1224

Concordance= 0.859  (se = 0.029 )
Likelihood ratio test= 24.9  on 5 df,   p=1e-04
Wald test            = 16.37  on 5 df,   p=0.006
Score (logrank) test = 22.96  on 5 df,   p=3e-04
```

In the model that includes age and treatment, treatment is significant p = 0.003.

Notice also how omitting age affects the coefficients and hazard ratios, summarized in the table below.

	Omit age	Include age as covariate	Consequence
Treatment coefficient	-0.8	-1.9	Omitting age shrinks the treatment coefficient towards the null hypothesis value of 0.
Treatment hazard ratio	0.43	0.15	Omitting age shrinks the treatment hazard ratio towards the null hypothesis value of 1.0.
Treatment p-value	0.082	0.003	Omitting age causes the treatment p-value to be non-significant.

What do you prefer? Omit a covariate because it is balanced across the treatment results, giving a worse p-value and a hazard ratio shrunk toward the null hypothesis? Or include the covariate giving a better p-value and a better hazard ratio?

7.4 Avoid testing for a difference in percent surviving at a specific timepoint. Keep the time information and use survival analysis to get better power.

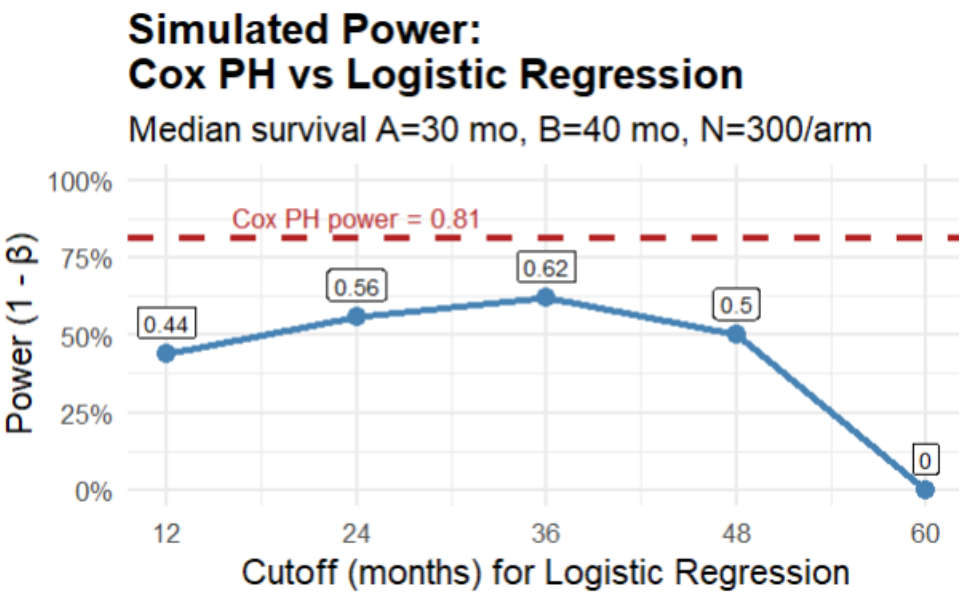
As an alternative to survival analysis, you might have thought about simply asking, "What percent of each treatment group survives to, say, 24 months." You could do this analysis using a Chi-squared test or a logistic regression (to include important covariates). While using such a cutoff may be attractive as an easy way to present results, it greatly reduces power and gives worse p-values than a Cox proportional hazards regression. Here are results of a simulation to illustrate the loss of power. The R code for the simulation is at the end of this chapter.

For this simulation, we assume the following.

```
n_sim      = 100  # Number of Monte Carlo replications
n_per_arm  = 300  # patients per treatment arm
median_A   = 30   # median survival in treatment group A
(months)
```

```
median_B   = 40   # median survival in treatment group B
(months)
max_follow = 60   # administrative censoring (months)
# logistic regression cutpoints:
cutoffs    = c(12, 24, 36, 48, 60)
alpha      = 0.05
```

The graph shows the power for the Cox PH regression versus the logistic regression, where the logistic regression used cutoffs of 12, 24, 36, 48, or 60 months. In these simulations, power for Cox regression is 0.81, which is greater than the power at any cutoff for a logistic regression.



Here are some of the weaknesses of using cutoffs.

- Using cutoffs throws away information on the actual time to the events.
- Subjects censored before the cutoff are typically excluded. This can introduce bias if censoring is informative.
- Power depends heavily on the chosen cutoff; a poor choice can dramatically reduce power.
- Multiple cutoffs inflate Type I (false positive) error unless corrected. Correction for multiple hypothesis testing further reduces power.

8 For count outcomes consider negative binomial regression and Wilcoxon tests

Here are some examples of count data in biology and medicine.

- The number of baby mice in a litter
- The number of bacteria colonies on a culture plate
- The number of days a newborn baby spends in the hospital
- The number of migraine headaches a patient has in one month
- The number of plants of a particular species found in a particular area

If you have count data, you might first think of analyzing it using a t-test or linear regression (to allow covariates). Two alternatives, negative binomial regression, and Wilcoxon rank tests, will sometimes give better power and better p-values for count data, under conditions that we will describe.

Load these libraries to run the examples in this chapter.

```
library(MASS) # use the glm.nb function from the MASS library
library(tidyverse)
```

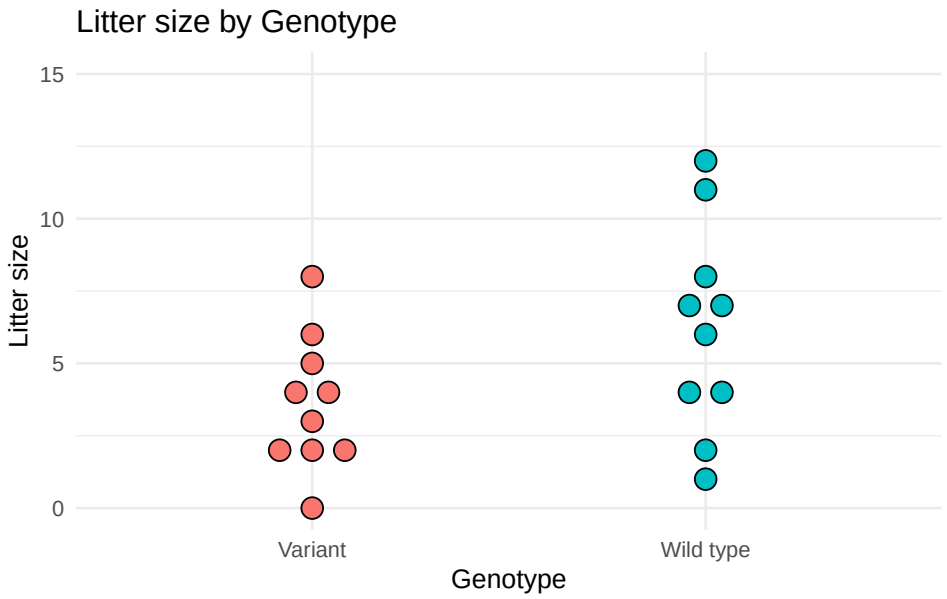
8.1 Example: Do mice with a genetic variant have smaller litters?

Let's start with a small example to demonstrate negative binomial regression, along with a t-test and a Wilcoxon rank sum test.

A researcher believes that a particular genetic variant may reduce the average litter size in the mouse strain she studies. The response variable is the number of baby mice in the litter. The researcher determines the litter size for 10 wild type mothers and 10 mothers with the genetic variant. For mothers that do not give birth after breeding, the litter size is set to 0.

The following R code sets up the data frame and creates the graph.

```
## Create data set
n.0 = 10
n.1 = 10
litter.sizes.data = data.frame(Genotype = c(rep("Wild type",
n.0), rep("Variant", n.1)),
  litter.size = c(1,2,4,4,6,7,7,8,11,12, 0,2,2,2,3,4,4,5,6,8))

# Graph the litter size by genotype
ggplot(litter.sizes.data, aes(y=litter.size,
x=as.factor(Genotype), fill=Genotype)) +
  geom_dotplot(binaxis='y', stackdir='center',
               stackratio=1.5, dotsize=1.5) + theme_minimal()
               +
  theme(legend.position="none") + ylab("Litter size") +
  xlab("Genotype") + ylim(0,15) +
  ggtitle("Litter size by Genotype")
```



Calculate the mean for each genotype.

```
# Mean by Genotype
with(litter.sizes.data, aggregate(litter.size ~ Genotype,
FUN="mean"))
```

```
   Genotype litter.size
1   Variant         3.6
2 Wild type         6.2
```

In the variant genotype, the mean litter size is 3.6. In the wild type mice the mean is 6.2. We want to know if the difference between the means is statistically significant.

Let's start with a t-test. Here's the R code and output.

```
t.test(litter.size ~ Genotype, data=litter.sizes.data)
```

```
    Welch Two Sample t-test

data:  litter.size by Genotype
t = -1.9261, df = 15.412, p-value = 0.07275
alternative hypothesis: true difference in means between group
Variant and group Wild type is not equal to 0
95 percent confidence interval:
 -5.4705486  0.2705486
sample estimates:
  mean in group Variant mean in group Wild type
                    3.6                     6.2
```

The t-test gives a non-significant result, p = 0.07.

The researcher might be concerned that the data may not be normally distributed, particularly since litter size can't take negative values. She tries a Wilcoxon rank sum test. Here's the R code and output.

```
wilcox.test(litter.size ~ Genotype, data=litter.sizes.data)
```

```
    Wilcoxon rank sum test with continuity correction

data:  litter.size by Genotype
W = 28.5, p-value = 0.1093
alternative hypothesis: true location shift is not equal to 0
```

The Wilcoxon rank sum test gives a non-significant result, p = 0.11.

Now let's try the negative binomial regression model. Here's the R code and output. We use the `glm.nb` function from the MASS library.

```
# negative binomial model
litter.sizes.nb = glm.nb(litter.size ~ Genotype,
data=litter.sizes.data)
summary(litter.sizes.nb)
```

```
Call:
glm.nb(formula = litter.size ~ Genotype, data =
litter.sizes.data,
    init.theta = 6.70930258, link = log)

Coefficients:
                  Estimate Std. Error z value Pr(>|z|)
(Intercept)         1.2809     0.2066   6.200 5.64e-10 ***
GenotypeWild type   0.5436     0.2715   2.002   0.0453 *
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for Negative Binomial(6.7093) family
taken to be 1)

    Null deviance: 26.489  on 19  degrees of freedom
Residual deviance: 22.425  on 18  degrees of freedom
AIC: 102.25

Number of Fisher Scoring iterations: 1

              Theta:  6.71
          Std. Err.:  5.42

 2 x log-likelihood:  -96.25
```

The negative binomial regression gives a significant result, p = 0.045. For this particular data set, we get a better p-value using negative binomial regression. We'll examine more data sets shortly.

8.2 Why not use t-tests or ordinary linear regression for count data?

What happens if you use t-tests or ordinary linear regression to analyze count data, such as the number of migraines per month? There are several reasons why count regression models (Poisson regression or negative binomial regression) or Wilcoxon tests are sometimes better choices for count data, particularly when the number of counts is small (typically mean counts of 1 to 10).

Consider a study of the effect of treatment on the number of migraine headaches a patient has per month. An ordinary linear regression could predict that the number of migraines will be negative. Clearly, a negative number of migraines does not make sense. But that is what an ordinary linear regression may predict.

Linear regression models assume that the residuals are normally distributed. But count data residuals are often not normally distributed. Recall that the residuals for a regression model are the differences between the observed y value and the y value that the model predicts. For example, if the observed number of migraines for a particular patient is 3 per month, and the regression model predicts that they have 2 migraines per month, then the residual is $3 - 2 = 1$. If the residuals are normally distributed around the mean, then counts of (mean + x) and (mean – x) are equally likely. Taking our migraine example, with mean = 1 migraine per month, having 3 migraines $(1 + 2) = 3$ is just as likely as –1 migraine $(1 - 2) = -1$. But because these are counts, negative values are impossible, and the assumption of normally distributed residuals is violated. Because the calculation of p-values and confidence intervals for linear regression relies on the normality assumption, the p-values and confidence intervals will be incorrect.

Ordinary linear regression models assume that the residuals have constant variance, that is, the same variance for any value of the explanatory variable. Count data models assume that the data have variance that increases as the mean count increases (residuals are Poisson distributed or negative binomial distributed). The predicted values and prediction confidence intervals from an ordinary linear regression that assumes constant variance will be incorrect when variance is not constant.

If ordinary linear regression is used to make predictions of counts given particular values of the explanatory variables, the predicted counts from

the linear regression models are likely to have larger errors compared to the predicted counts from count regression models.

For count data that meet the negative binomial distribution assumption, the p-values from a negative binomial regression will usually be better (smaller) than the p-values from t-tests and linear regression (which don't meet the normality and variance assumptions).

8.3 Compare power for data from a negative binomial distribution

The example above used only one data set, one effect size and one sample size. To see if the better p-values for negative binomial regression hold up when we have different effect sizes and different sample sizes, we run a simulation study. We generate 1000 different data sets and vary the sample size per genotype. The code generates data as samples from a negative binomial distribution, which is common for count data. After that, we'll do a similar power simulation using data that is not generated from a negative binomial distribution.

The first graph shows power for the three methods when the mean of the placebo group is 2.5, 3, 4 or 5, and the mean for the drug group is 2. The effect size for a negative binomial regression is sometimes reported as the difference between the means, and is sometimes reported as the rate ratio of the drug mean to the placebo mean. For this graph we show the rate ratio. That is, dividing the drug mean of 2 by the placebo group means of 2.5, 3, 4 or 5 gives us the rate ratios $2/2.5 = 0.8$, $2/3 = 0.67$, $2/4 = 0.5$ and $2/5 = 0.4$, which are the values shown on the x-axis of the graph.

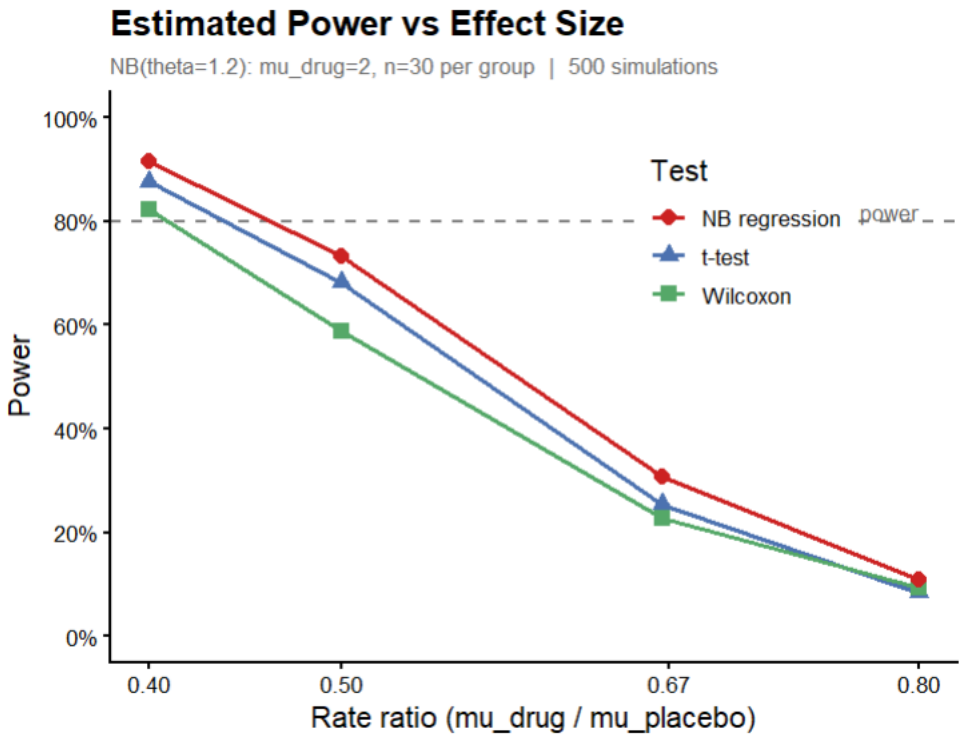


Figure 8.1: Power versus effect size (rate ratio) for negative binomial regression, t-test, and Wilcoxon rank sum test, using data generated from a negative binomial distribution.

The graph shows power versus the effect size (rate ratio). We see that the negative binomial regression has consistently greater power than either t-test or Wilcoxon rank sum test across a range of rate ratios for these data.

We can also compare the power for different sample sizes, as shown in the graph below. The graph shows power versus the sample size. We see that the negative binomial regression has consistently greater power than either t-test or Wilcoxon rank sum test across a range of sample sizes for these data.

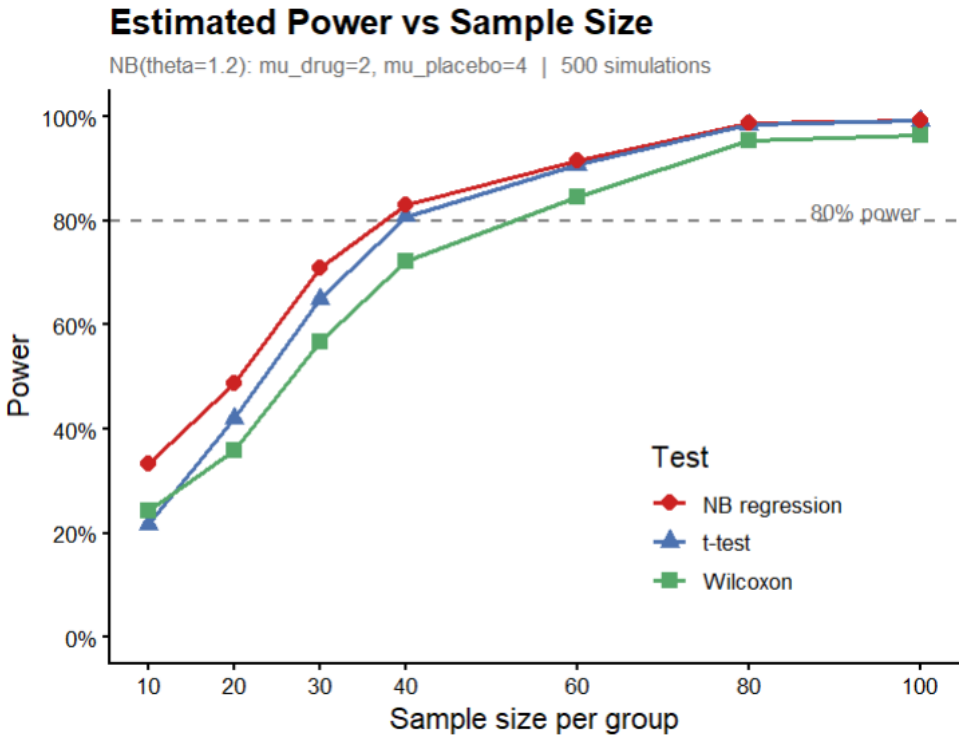


Figure 8.2: Power versus sample size for negative binomial regression, t-test, and Wilcoxon rank sum test, using data generated from a negative binomial distribution.

For this simulation we generated data from a negative binomial distribution. As we noted, this distribution is common for count data. How does power change with data that are not from a negative binomial distribution?

8.4 Compare power when data are from a mixture of distributions

One situation in which the Wilcoxon test may give better results than NB or t-test occurs when the data within at least one group is a mixture. For example, the treatment (drug) group may be a mixture of responders and non-responders.

Here is example data from one such study, where the drug group is a mixture of 70% responders and 30% non-responders. The non-responder's distribution

is essentially the same as the distribution for the placebo group. The graph below shows data from one such sample.

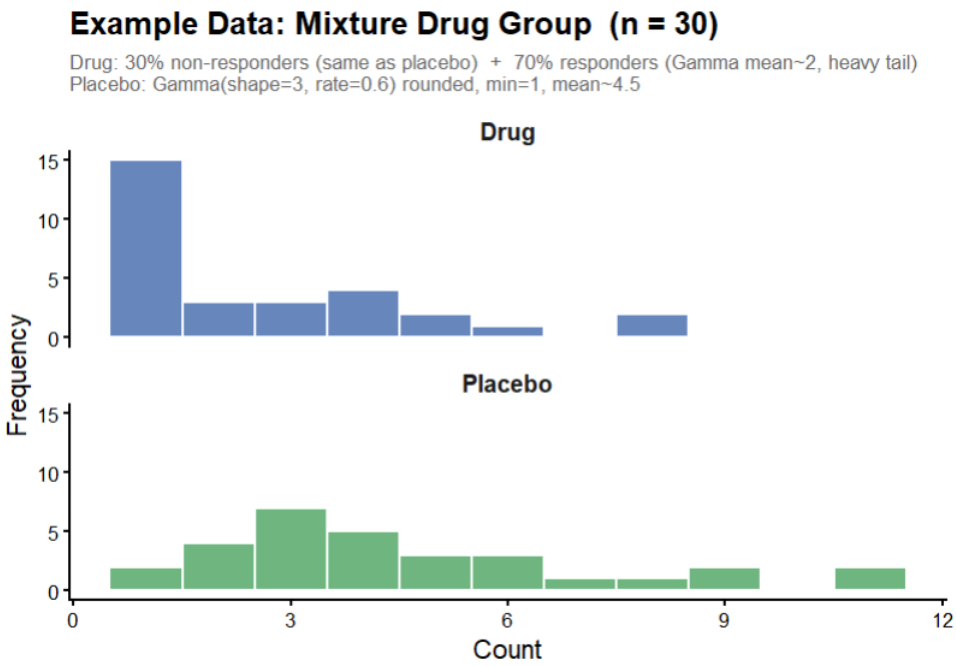


Figure 8.3: Sample data from a study where the drug group is a mixture of responders (70%) and non-responders (30%).

The mixture creates a bimodal structure within the drug group. There is a cluster of low values from responders and a cluster of moderate values from non-responders, plus occasional large outliers from the responders' heavy tail. This inflates the drug group's variance, which causes problems for t-tests and negative binomial models: the t-test suffers from increased variance, and the NB can't fit a single dispersion parameter to a bimodal mixture. The Wilcoxon simply detects that drug values tend to rank lower, which remains true regardless of the outliers and the increased variance.

Power simulations for this data set give the following results.

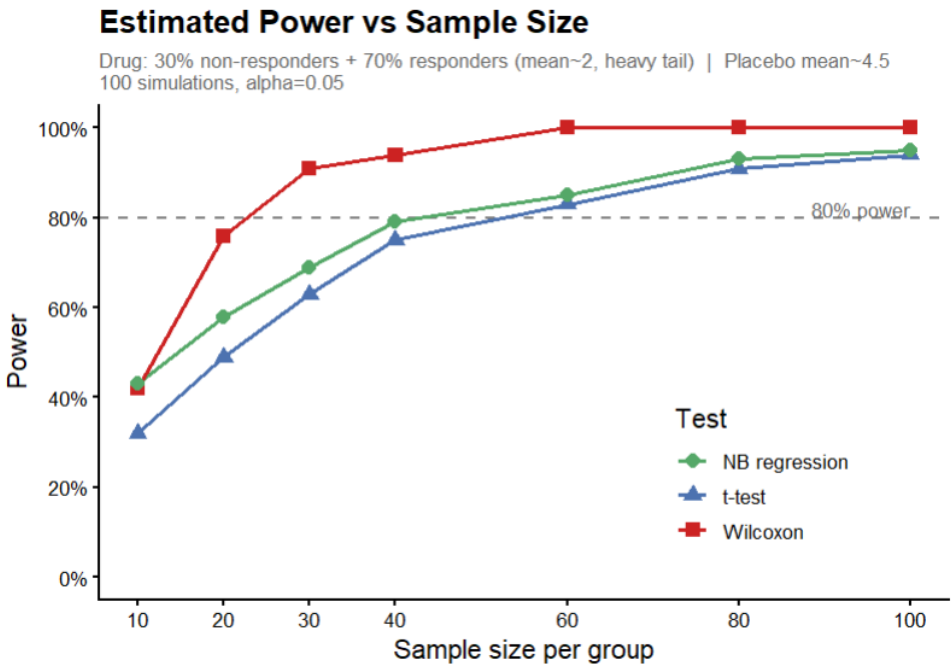


Figure 8.4: Power simulation results for the mixture distribution scenario, comparing negative binomial regression, t-test, and Wilcoxon rank sum test.

The graph shows that the Wilcoxon rank sum test has the highest power, followed by the NB regression, while the t-test has the lowest power.

There are statistical tests to determine if a particular data sample differs significantly from the NB distribution, so we might think of using them to decide which test to use. Unfortunately, those tests have quite low power for the sample sizes typical in biological data. If the number of observations is 100 or more per treatment group, then power is reasonable to detect moderate deviations from NB, while more than 500 observations per group gives high power to detect almost any deviation from NB.

9 Avoid turning quantitative variables into categorical variables

You may sometimes consider turning a quantitative response variable into a categorical response variable. Here are two examples where a collaborator may suggest creating a categorical response.

- A drug is intended to lower blood pressure. A collaborator suggests testing if there is a difference between drug and placebo in the proportion of subjects whose blood pressure decreases. This would change a quantitative measure (the actual measured change from baseline) into a categorical variable (Improved or not Improved).
- A drug is intended to reduce serum cholesterol. A collaborator suggests that the response variable should be defined as categorical: either normal cholesterol (< 200) or elevated cholesterol ($>= 200$). Let's see how using categories affects power, sample size, and p-values.

In a few cases, as we'll discuss, this can be helpful. But in most cases, turning a quantitative response variable into a categorical variable will reduce power, increase required sample size, and make your p-values worse.

We'll simulate some experimental data to see the p-values first, and then do power and sample size calculations.

9.1 Example. Creating a categorical response with 2 values

For our first example, suppose that the treatment is drug or placebo, and the response variable is defined to be either:

- Systolic blood pressure change from baseline as a quantitative variable, or

- Systolic blood pressure change as a categorical variable, with 2 categories: Improved (lower than baseline) or Not Improved (not lower than baseline).

We'll use simulated data on an experiment in which subjects received either drug or placebo, and their systolic blood pressure was measured 3 months later.

```
# SBP data
set.seed(11111)
drug   = round(rnorm(n=20, mean= -10, sd=20), 0)
placebo= round(rnorm(n=20, mean=  10, sd=20), 0)
treatment  = c(rep("Drug",    length(drug)),
               rep("Placebo", length(placebo)))
SBP.change = c(drug, placebo)
SBP.data   = data.frame(treatment, SBP.change)

SBP.data = within(SBP.data, {
  SBP.change.category = NA
  SBP.change.category[SBP.change <  0] = "Improved"
  SBP.change.category[SBP.change >= 0] = "Not Improved"
})

SBP.change.category.table = with(SBP.data,
                                 table(SBP.change.category,
                                 treatment))
```

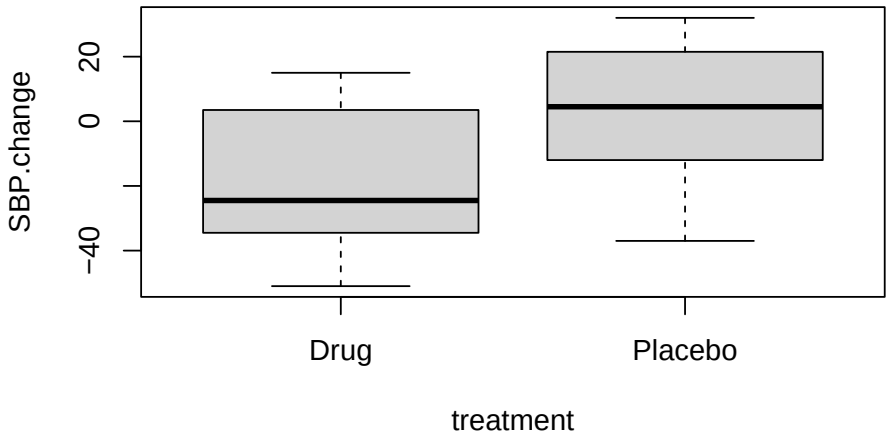
Here are the first 5 rows of SBP.data, which has the quantitative values.

```
head(SBP.data)
```

```
  treatment SBP.change SBP.change.category
1      Drug        -10            Improved
2      Drug          6        Not Improved
3      Drug          8        Not Improved
4      Drug        -34            Improved
5      Drug        -35            Improved
6      Drug         -6            Improved
```

Here are boxplots of the data.

```
with(SBP.data, boxplot(SBP.change ~ treatment))
```



The graphs suggest there is a large difference between the drug and placebo.

Here is the t-test.

```
with(SBP.data, t.test(SBP.change ~ treatment))
```

```
    Welch Two Sample t-test

data:  SBP.change by treatment
t = -3.0365, df = 37.859, p-value = 0.004316
alternative hypothesis: true difference in means between group
Drug and group Placebo is not equal to 0
95 percent confidence interval:
 -33.418666  -6.681334
sample estimates:
   mean in group Drug mean in group Placebo
              -17.60                  2.45
```

The t-test indicates that there is a significant difference between drug and placebo ($p = 0.004$).

Here is the table, where we have turned the quantitative SPD into categories.

```
SBP.change.category.table
```

```
                    treatment
SBP.change.category Drug Placebo
       Improved       14       8
       Not Improved    6      12
```

Now use a Fisher Exact test to compare the proportion Improved in each treatment arm. The Fisher test requires a table as input

```
fisher.test(SBP.change.category.table)
```

```

    Fisher's Exact Test for Count Data

data:  SBP.change.category.table
p-value = 0.111
alternative hypothesis: true odds ratio is not equal to 1
95 percent confidence interval:
  0.7972704 15.9709807
sample estimates:
odds ratio
  3.385212
```

The Fisher Exact test gives p = 0.11, indicating that there is no significant difference between drug and placebo in their effect on blood pressure. Contrast that to the t-test p = 0.004. It is evident that changing the quantitative variable into a categorical variable has changed a significant p-value into a non-significant p-value.

Let's compare the sample size required for a t-test using the measured values to the sample size required for a test using the categories. I generated the data for this example using a normal distribution with a difference (delta) between drug and placebo of 20, and a within-group standard deviation of 20. Assume the desired power is 0.8 (80%) and the default alpha of 0.05. We can calculate the required sample size using the R function power.t.test() as shown.

```
power.t.test(delta = 20, sd=20, power=0.8)
```

```

     Two-sample t test power calculation
```

```
              n = 16.71477
          delta = 20
             sd = 20
      sig.level = 0.05
          power = 0.8
    alternative = two.sided

NOTE: n is number in *each* group
```

Rounding up, we get a required sample size of 17 per treatment group for the t-test.

To calculate sample size for the test using categories, we need the expected proportion of Improved or not Improved in each treatment group. We assumed an effect size of delta = 20 for the t-test. For the next calculation, we'll assume the group means are -10 and 10, to give a difference between the means delta = 20 (which we used above for power.t.test).

The following code determines the proportion of the normal distribution for drug subjects that would be below 0, assuming a mean of -10 and standard deviation of 20.

```
# proportion of drug group below 0
pnorm(q=0, mean=-10, sd=20) # 0.69
```

```
[1] 0.6914625
```

Running this code, we find that 0.69 or 69% of drug subjects will have values below zero (that is, improved from baseline).

```
# proportion of placebo group below 0
pnorm(q=0, mean=10, sd=20) # 0.31
```

```
[1] 0.3085375
```

Running this code, we find that 0.31 or 31% of placebo subjects will have values below zero (that is, improved from baseline).

We can calculate sample size and power for a test for a difference of proportions using the R function power.prop.test(). The two proportions, as calculated above, are p1=0.69, p2=0.31 for drug and placebo respectively. Again, the target power is 0.8.

```
power.prop.test(p1=0.69, p2=0.31, power = 0.8)
```

```
     Two-sample comparison of proportions power calculation

              n = 25.9665
             p1 = 0.69
             p2 = 0.31
      sig.level = 0.05
          power = 0.8
    alternative = two.sided

NOTE: n is number in *each* group
```

Rounding up, we get a required sample size of 26 per treatment group for the test for difference in proportions.

Which do you prefer, a required sample size of 26 per group using the categorical variable, or a required sample size of 17 per group using the quantitative variable?

9.2 If you must create a categorical variable, create more than two categories.

If for some reason you have to change a quantitative variable into a categorical variable, you can get more power, smaller sample size, and smaller p-values if you create more than two categories for the categorical variable.

For this example, suppose that the treatment is either drug or placebo.

The response variable is one of the following.

- serum cholesterol as a quantitative variable, or
- serum cholesterol as a categorical variable, with 2 categories: less than 200 versus 200 or higher, or
- serum cholesterol as a categorical variable, with 3 categories: less than 200, 200 to 240, and 240 or higher.

```
# Cholesterol data
drug.mean       = 190
placebo.mean    = 210
sd.cholesterol  = 50
n.per.group     = 100
set.seed(3)
```

```
drug    = round(rnorm(n=n.per.group, mean=drug.mean,
sd=sd.cholesterol), 0)
placebo = round(rnorm(n=n.per.group, mean=placebo.mean,
sd=sd.cholesterol), 0)
treatment   = c(rep("Drug",    length(drug)),
                rep("Placebo", length(placebo)))
cholesterol = c(drug, placebo)
cholesterol.data = data.frame(treatment, cholesterol)

cholesterol.data = within(cholesterol.data, {
  chol.LT200 = NA
  chol.LT200[cholesterol <  200] = "LT200"
  chol.LT200[cholesterol >= 200] = "GTE200"
})

cholesterol.data = within(cholesterol.data, {
  chol.category = NA
  chol.category[cholesterol <  200]                    =
"LT200"
  chol.category[cholesterol >= 200 & cholesterol < 240]=
"GTE200LT240"
  chol.category[cholesterol >= 240]                    =
"GTE240"
})

LT.200.table = with(cholesterol.data,
                    table(chol.LT200, treatment))

cholesterol.ordered.table = with(cholesterol.data,
                                 table(treatment,
                                 chol.category))
```

Here are the first 10 rows of the data set, cholesterol.data.

```
head(cholesterol.data, 10)
```

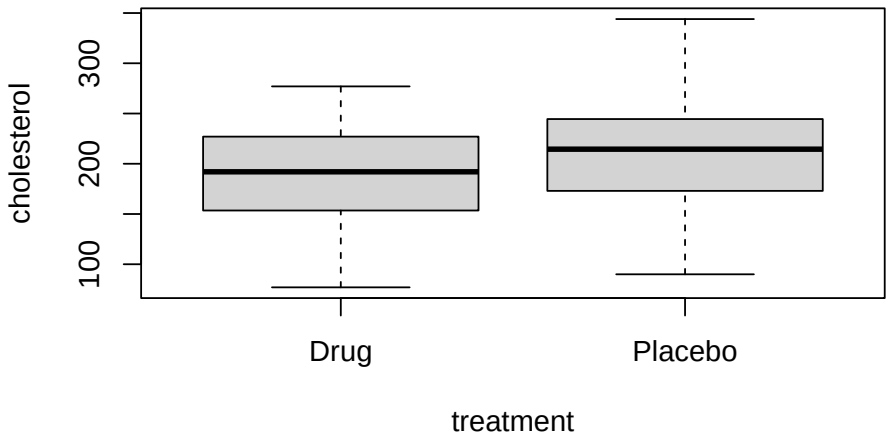
```
   treatment cholesterol chol.LT200 chol.category
1       Drug         142      LT200         LT200
2       Drug         175      LT200         LT200
3       Drug         203     GTE200   GTE200LT240
4       Drug         132      LT200         LT200
5       Drug         200     GTE200   GTE200LT240
```

```
6        Drug          192       LT200          LT200
7        Drug          194       LT200          LT200
8        Drug          246      GTE200         GTE240
9        Drug          129       LT200          LT200
10       Drug          253      GTE200         GTE240
```

The column "chol.LT200" shows serum cholesterol as a categorical variable with 2 categories: less than 200 versus 200 or higher. The column "chol.category" shows serum cholesterol as a categorical variable with 3 categories: less than 200, 200 to 240, and 240 or higher.

Create a boxplot of measured cholesterol versus treatment.

```
with(cholesterol.data, boxplot(cholesterol ~ treatment))
```



Here is the t-test for cholesterol versus treatment. The p-value of 0.0038 indicates that mean cholesterol differs between drug and placebo.

```
with(cholesterol.data, t.test(cholesterol ~ treatment))
```

```
    Welch Two Sample t-test

data:  cholesterol by treatment
t = -2.9272, df = 186.85, p-value = 0.003845
```

```
alternative hypothesis: true difference in means between group
Drug and group Placebo is not equal to 0
95 percent confidence interval:
 -34.131508  -6.648492
sample estimates:
   mean in group Drug mean in group Placebo
               190.55                210.94
```

In the cholesterol.data the column "chol.LT200" is a categorical variable with 2 categories: less than 200 versus 200 or higher. We can use a Fisher Exact test to test if there is a difference between drug and placebo using chol.LT200 as the response variable.

In the table we see that serum cholesterol is less than 200 in 54 of the drug subjects and in 44 of the placebo subjects. The drug appears to be effective.

```
LT.200.table
```

```
          treatment
chol.LT200 Drug Placebo
    GTE200   46      56
    LT200    54      44
```

Here is the Fisher Exact test using this table.

```
fisher.test(LT.200.table)
```

```

    Fisher's Exact Test for Count Data

data:  LT.200.table
p-value = 0.2029
alternative hypothesis: true odds ratio is not equal to 1
95 percent confidence interval:
 0.368984 1.213068
sample estimates:
odds ratio
 0.6706832
```

The Fisher Exact test gives p = 0.2, indicating that there is no difference between drug and placebo in the proportion of subjects with serum cholesterol less than 200. Contrast this result to the t-test, which gave p = 0.0038, indicating a significant difference between drug and placebo.

We can get a better p-value if we define serum cholesterol as a categorical variable with 3 categories: less than 200, 200 to 240, and 240 or higher, and then use a CMH test that recognizes the ordering. In the cholesterol.data, the variable chol.category has the 3 categories.

```
cholesterol.ordered.table
```

```
          chol.category
treatment GTE200LT240 GTE240 LT200
  Drug              31     15    54
  Placebo           24     32    44
```

Because we have an ordered categorical variable, we will use the Cochran-Mantel-Haenszel (CMH) test, implemented in the CMHtest() function from the vcdExtra library. For more details on this function see the chapter on the CMH test for ordered variables.

To run the CMHtest in R, install the vcdExtra package and load the library.

```
# install.packages("vcdExtra")
library(vcdExtra)
```

The CMHtest requires a table as input. We pass this table to the CMHtest() function.

```
CMHtest(cholesterol.ordered.table)
```

```
Cochran-Mantel-Haenszel Statistics for treatment by
chol.category

                 AltHypothesis    Chisq Df     Prob
cor        Nonzero correlation 0.062293  1 0.802907
rmeans  Row mean scores differ 0.062293  1 0.802907
cmeans  Col mean scores differ 8.019952  2 0.018134
general    General association 8.019952  2 0.018134
```

For this analysis, the relevant results are in the row labeled “rmeans” in the output.

- The row labeled “rmeans” (highlighted in yellow) shows the result for testing that the row mean scores differ for drug versus placebo. The CMH test gives p = 0.019, indicating that drug and placebo differ significantly in serum cholesterol.

- The row labeled "cor" is used for tables where both rows and columns are ordered.
- The row labeled "cmeans" would apply to tables in which the rows are ordered but columns are not ordered.
- The row labeled "general" is similar to the chi-squared and Fisher tests.

Let's compare the results from the three different ways of analyzing this data set.

The Fisher Exact p-value using 2 categories is not significant ($p = 0.2$). In contrast, the p-value for 3 categories using the CMH test for ordered categories is significant ($p = 0.0191$), and much closer to the p-value using the t-test with quantitative values ($p = 0.0038$).

When you create the categories, it is desirable that there be a plausible reason for selection of the thresholds that define the categories, and that there be a good number of observations in each level of the categorical variable.

9.3 Logistic regression and ordinal logistic regression for categorical response variables.

An alternative to the CMH test for categorical variables is logistic regression. If the response variable has only 2 categories, then you can use the standard version of logistic regression. If the response variable has 3 or more ordered categories, you can use ordinal logistic regression.

Ordinal logistic regression is available in the R package polr. The UCLA statistics website on ordinal logistic regression has a good introductory tutorial on polr (https://stats.oarc.ucla.edu/r/dae/ordinal-logistic-regression/). The UCLA statistics tutorial website stats.oarc.ucla.edu/other/dae is a great resource that I highly recommend. There are many very helpful and easy to understand tutorials on analyses using R and other statistical software.

9.4 When can it be helpful to use categories instead of quantitative measures?

I've occasionally encountered situations in which it was helpful to turn a quantitative variable into a categorical variable. Here are some examples.

Extremely inaccurate quantitative measurements

If the supposed "quantitative" measurements of a variable are extremely inaccurate the quantitative information may be less informative than categories. In a data set that I examined many years ago, the supposed "quantitative" measure was the expression level of a gene, based on fragments of mRNA recovered from (some quite old) paraffin-embedded tumor blocks. If you have tried this yourself, you likely know that the mRNA is highly fragmented, and the resulting gene fragments can be difficult to assemble into a complete sequence. At the time I was doing this, the human genome project had not been completed, so we had no reference sequence for many genes. I examined correlations among genes known to be in the same regulatory pathways, or known to have related biological function, but found none of the results made any sense. Eventually I replaced the "quantitative" measure with a categorical indicator that a particular gene was either found expressed or not found in a particular tissue specimen. Instead of correlation, I used a Fisher Exact test to examine the co-occurrence of genes. The analyses using the categorical indicator variable and Fisher Exact test were able to reproduce known associations and contributed to finding likely functions of many genes. So, in this case, turning a quantitative variable into a categorical variable proved useful.

Biologically meaningful categories

In some cases, an easily obtained measured variable such as age may be a proxy for a biologically meaningful category, such as pre-menopausal versus post-menopausal status. For a model of cancer versus gene expression, we might consider adding age as a covariate, as a proxy for menopausal status. If the true menopausal status is the biologically meaningful covariate, but it is not known, then an analysis may yield better results if we create a categorical variable such as, for example, age less than 50 versus age 50 or greater as a proxy. We might alternatively consider 3 ordered categories for age, such as <50, 50 to 60, and over 60, corresponding to pre-, peri-, and post-menopausal.

Non-linear response

In some cases, the relationship between two quantitative variables may not be a straight line. Testing using correlation or regression may not work well. In this case, it can be useful to split one or both variables into meaningful categories and use either ANOVA or CMH to test for association.

10 Avoid Chi-Squared tests when you have ordered categories: use CMH tests instead

When you have tables with ordered categorical variables, the Cochran-Mantel-Haenszel (CMH) test for ordered categories will usually give smaller p-values than a chi-squared or Fisher Exact test.

Here are two examples of tables with ordered categorical variables, showing counts of the number of subjects in each category.

- A clinical trial of a cancer drug in which the possible outcomes are the ordered categories "no remission", "partial remission", and "complete remission".

	No remission	Partial remission	Complete remission
Drug	1	5	3
Placebo	3	5	1

- A clinical trial of a cancer drug in which the rows are the ordered treatment categories "placebo", "low dose", "high dose", and the columns are the ordered categories "no remission", "partial remission", and "complete remission". Both rows and columns are ordered categories.

	No remission	Partial remission	Complete remission
Placebo	5	3	2
Low dose	2	3	4
High dose	1	3	6

Published articles sometimes report non-significant results when the researchers use a chi-squared tests for tables that have ordered categories. The CMH test would provide more power and better p-values for these analyses.

The Cochran-Mantel-Haenszel (CMH) test extends the chi-squared and Fisher Exact tests in two ways:

- tables with more than 2 factors (stratified analyses)
- tables with ordered rows, columns, or both

Cochran-Mantel-Haenszel (CMH) test

We'll use the CMHtest() function from vcdExtra package.

```
library(vcdExtra)
```

10.1 Example: CMH test for table with ordered columns

Consider a clinical trial of a cancer drug versus a placebo in which the possible outcomes are the ordered categories "no remission", "partial remission", and "complete remission".

Here is the data set.

```
ordered.columns.table = array(
  data = c(1, 5, 3, 3, 5, 1),
  dim = c(2, 3),
  dimnames = list(
    Treatment = c("drug", "placebo"),
    Result = c("no remission", "partial remission", "complete
    remission")
  )
)
```

Here is the resulting table.

```
ordered.columns.table
```

```
         Result
Treatment no remission partial remission complete remission
  drug               1                 3                  5
  placebo            5                 3                  1
```

The chi-squared test and Fisher Exact test ignore the ordering of the columns and return non-significant p-values for this example. Let's start with the chi-squared test, which is commonly used.

```
chisq.test(ordered.columns.table)
```

```
    Pearson's Chi-squared test

data:  ordered.columns.table
X-squared = 5.3333, df = 2, p-value = 0.06948
```

The sample size is quite small, so the Fisher test is preferred over the chi-squared test. The chi-squared output gives a warning that the chi-squared approximation may be incorrect. Next try the Fisher Exact test.

```
fisher.test(ordered.columns.table)
```

```
    Fisher's Exact Test for Count Data

data:  ordered.columns.table
p-value = 0.1135
alternative hypothesis: two.sided
```

We get a non-significant result for both the Fisher Exact test (p = 0.1135) and the chi-squared test (p = 0.069), but this is not reliable due to small sample size.

In contrast, the Cochran-Mantel-Haenszel (CMH) test uses the fact that there are ordered rows, columns, or both, which increases the power. In this example, it gives a significant result.

```
CMHtest(ordered.columns.table)
```

```
Cochran-Mantel-Haenszel Statistics for Treatment by Result

                 AltHypothesis Chisq Df     Prob
cor        Nonzero correlation 5.037  1 0.024811
rmeans  Row mean scores differ 5.037  1 0.024811
cmeans  Col mean scores differ 5.037  2 0.080579
general    General association 5.037  2 0.080579
```

Here is the interpretation of the output.

- The row labeled “cor” is used for tables where both rows and columns are ordered. We’ll look at an example shortly.

- The row labeled "rmeans" (highlighted in yellow) shows the result testing that the row mean scores differ for drug versus placebo. The CMH test gives p = 0.0248, indicating that drug and placebo differ significantly in outcomes.
- The row labeled "cmeans" would apply to tables in which the rows are ordered but columns are not ordered.
- The row labeled "general" is similar to the chi-squared and Fisher tests.

The p-value of 0.0248 for the test that row means differ indicates that there is a significant difference between the drug and placebo groups.

10.2 Example: CMH test for table with ordered rows and columns

In this example we'll see data with both ordered rows and ordered columns from a clinical trial of a cancer drug.

- Rows are the ordered treatment categories "placebo", "low dose", and "high dose".
- Columns are the ordered categories "no remission", "partial remission", and "complete remission".
- Both rows and columns are ordered categories.

```
ordered.rows.and.columns.table = array(
  data = c(5, 2, 1, 3, 3, 3, 2, 4, 6),
  dim = c(3, 3),
  dimnames = list(
    Treatment = c("placebo", "low dose", "high dose"),
    Result = c("no remission", "partial remission", "complete
    remission")
  )
)

ordered.rows.and.columns.table
```

```
           Result
Treatment   no remission partial remission complete remission
  placebo              5                 3                 2
  low dose             2                 3                 4
  high dose            1                 3                 6
```

The chi-squared test and Fisher Exact test ignore the ordering of the rows and columns and return non-significant p-values for this example.

```
chisq.test(ordered.rows.and.columns.table)
```

```
    Pearson's Chi-squared test

data:  ordered.rows.and.columns.table
X-squared = 5.0213, df = 4, p-value = 0.2851
```

```
fisher.test(ordered.rows.and.columns.table)
```

```
    Fisher's Exact Test for Count Data

data:  ordered.rows.and.columns.table
p-value = 0.3319
alternative hypothesis: two.sided
```

The Cochran-Mantel-Haenszel (CMH) test uses the fact that there are ordered rows and columns, which increases the power and gives a significant result in this example. The row labeled "cor" is used for tables where both rows and columns are ordered.

```
CMHtest(ordered.rows.and.columns.table)
```

```
Cochran-Mantel-Haenszel Statistics for Treatment by Result

                 AltHypothesis  Chisq Df     Prob
cor        Nonzero correlation 4.6071  1 0.031840
rmeans  Row mean scores differ 4.7406  2 0.093453
cmeans  Col mean scores differ 4.6667  2 0.096972
general    General association 4.8481  4 0.303236
```

The row labeled "cor" (highlighted in yellow) shows the result. The CMH test gives p = 0.032, indicating that the treatments differ significantly in outcomes.

10.3 Watch out for which CMH test your software is performing.

There are several different statistical tests that are sometimes called a Cochran-Mantel-Haenszel test and sometimes called a Mantel-Haenszel test. In some software packages, the names Cochran-Mantel-Haenszel test or Mantel-Haenszel test do not refer to the test for ordered categories, but instead refer to a stratified version of a chi-squared test. An example of a stratified test would be when we want to look at the relationship between, say, drug and placebo versus responder / non-responder, but want to stratify by, say, sex. Then there are separate tables of treatment versus response for each sex, and the CMH or MH test calculates a chi-squared statistic that essentially pools the tables together. This analysis is clearly not the same as having ordered rows or columns, but unfortunately the naming convention doesn't always make it clear which test the software actually performs. You need to read the documentation.

11 Balancing variables across treatment groups is not sufficient to get best p-values

You may have been given this advice by a well-intentioned collaborator:

"If the variables that affect the response are balanced across the treatment groups, then you don't need to include those variables in your analysis. You can do the analysis using a t-test. It won't affect your power or p-values. There is no benefit from using multiple regression or analysis of variance."

These statements are not correct. If you want good p-values and good power you should include variables that affect the response in your regression and ANOVA analyses, even if they are balanced across treatment groups.

I'll show you a couple of short examples first, skipping over some details. Then, if you want to learn more, the rest of this chapter gives details of the examples, including complete R code and the data sets.

11.1 Short example 1: Testing a treatment when the response is also affected by patient's age.

We wish to know if a drug affects a particular serum protein. The serum protein is known to change with age. Here is a data set.

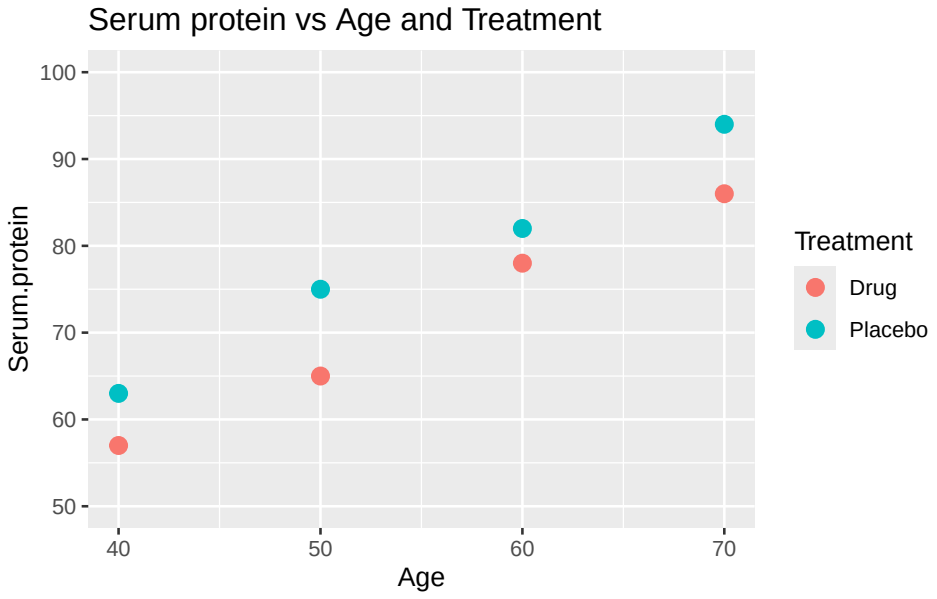
```
# Load required libraries
library(ggplot2)
library(car)
```

```
# Generate a data set "protein.data"
# Serum protein vs Age and Treatment

protein.data = tibble::tribble(
  ~Subject, ~Age, ~Treatment,   ~Serum.protein,
```

```
  1,   40, "Placebo",              63,
  2,   40, "Drug",                 57,
  3,   50, "Placebo",              75,
  4,   50, "Drug",                 65,
  5,   60, "Placebo",              82,
  6,   60, "Drug",                 78,
  7,   70, "Placebo",              94,
  8,   70, "Drug",                 86
)
```

Here is a plot showing the response (serum protein), patient age and treatment.

```
graph.1 = ggplot(protein.data, aes(y=Serum.protein, x=Age,
color=Treatment)) +
  geom_point(size=3) +
  ggtitle("Serum protein vs Age and Treatment") +
  ylim(50, 100)

plot(graph.1)
```



Notice that, at each age (40, 50, 60, and 70), one subject gets the drug, and one subject gets the placebo. So, age is perfectly balanced between the two treatment groups.

First, analyze the data using a t-test, ignoring age.

```
t.test(Serum.protein ~ Treatment, data=protein.data,
var.equal=TRUE)
```

```
    Two Sample t-test

data:  Serum.protein by Treatment
t = -0.76301, df = 6, p-value = 0.4744
alternative hypothesis: true difference in means between group
Drug and group Placebo is not equal to 0
95 percent confidence interval:
 -29.44855  15.44855
sample estimates:
   mean in group Drug mean in group Placebo
                 71.5                  78.5
```

The t-test gives us p = 0.47. According to this analysis, the drug has no effect on the serum protein. Details for this example are given below.

Next, analyze the data using a multiple regression model that includes both treatment and age as explanatory variables. The regression analysis gives us p = 0.000917 for treatment.

```
protein.lm = lm(Serum.protein ~ Treatment + Age,
data=protein.data)
summary(protein.lm)
```

```
Call:
lm(formula = Serum.protein ~ Treatment + Age, data =
protein.data)

Residuals:
   1    2    3    4    5    6    7    8
-0.5  0.5  1.5 -1.5 -1.5  1.5  0.5 -0.5

Coefficients:
                 Estimate Std. Error t value Pr(>|t|)
(Intercept)      16.50000    2.55930   6.447 0.001335 **
TreatmentPlacebo  7.00000    1.00000   7.000 0.000917 ***
Age               1.00000    0.04472  22.361 3.32e-06 ***
```

```
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 1.414 on 5 degrees of freedom
Multiple R-squared:  0.991, Adjusted R-squared:  0.9874
F-statistic: 274.5 on 2 and 5 DF,  p-value: 7.738e-06
```

The multiple regression model that includes both treatment and age as explanatory variables gives p = 0.0009 for Treatment.

Which do you prefer?

- T-test p = 0.47 for treatment, not controlling for age.
- Regression p = 0.000917 for treatment after controlling for age.

Consider again the advice:

"As long as the variables that affect the response are balanced across the treatment groups, then you don't need to include those variables in your analysis. You can do the analysis using a t-test. It won't affect your power or p-values."

Clearly, the advice is nonsense. Regardless of how famous the professor is who gives you this advice.

11.2 Short example 2: Adjusting for 2 or more variables that affect the response.

In this example we'll see how to adjust for two nuisance variables that affect the response, along with the treatment. The same method applies to adjusting for more than two explanatory variables.

We perform an experiment to test the effect of a treatment on a response. Unfortunately, we also have two nuisance variables, Operator and Instrument, that we suspect are affecting the measured response that is reported.

Here is the experiment.2.data. Notice that the two Instruments are perfectly balanced across the Treatment groups. The two Operators are also perfectly balanced across the Treatment groups.

```
set.seed(100) # set.seed makes rnorm reproducible.

Treatment = c(rep(0, 8), rep(1, 8))
Instrument = c(rep(c(0, 1),8))
Operator = c(rep(c(0, 0, 1, 1),4))
Response =  20 + Treatment + Instrument + Operator +
rnorm(n=16)
experiment.2.data = data.frame(Treatment = Treatment,
Instrument =Instrument,
                               Operator = Operator , Response=
                               Response)

experiment.2.data
```

```
   Treatment Instrument Operator Response
1          0          0        0 19.49781
2          0          1        0 21.13153
3          0          0        1 20.92108
4          0          1        1 22.88678
5          0          0        0 20.11697
6          0          1        0 21.31863
7          0          0        1 20.41821
8          0          1        1 22.71453
9          1          0        0 20.17474
10         1          1        0 21.64014
11         1          0        1 22.08989
12         1          1        1 23.09627
13         1          0        0 20.79837
14         1          1        0 22.73984
15         1          0        1 22.12338
16         1          1        1 22.97068
```

First, we'll analyze the data using a t-test, ignoring instrument and operator.

```
with(experiment.2.data, t.test(Response ~ Treatment,
var.equal=TRUE))
```

```
    Two Sample t-test

data:  Response by Treatment
```

```
t = -1.483, df = 14, p-value = 0.1602
alternative hypothesis: true difference in means between group
0 and group 1 is not equal to 0
95 percent confidence interval:
 -2.026618  0.369678
sample estimates:
mean in group 0 mean in group 1
       21.12569        21.95416
```

The t-test gives us p = 0.16. According to this analysis, the treatment has no effect on the response.

Next, we'll analyze the data using a multiple regression model that includes treatment, instrument, and operator as explanatory variables.

```
with(experiment.2.data,
  summary(lm(Response ~ Treatment + Instrument + Operator)))
```

```

Call:
lm(formula = Response ~ Treatment + Instrument + Operator)

Residuals:
     Min       1Q   Median       3Q      Max
-0.54779 -0.27434 -0.00584  0.30379  0.62598

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  19.7406     0.2003  98.574  < 2e-16 ***
Treatment     0.8285     0.2003   4.137  0.00138 **
Instrument    1.5447     0.2003   7.714 5.45e-06 ***
Operator      1.2254     0.2003   6.119 5.19e-05 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.4005 on 12 degrees of freedom
Multiple R-squared:  0.9048,    Adjusted R-squared:  0.881
F-statistic: 38.02 on 3 and 12 DF,  p-value: 2.092e-06
```

The regression analysis gives us p = 0.00138 for treatment. It also gives us $p < 0.05$ for instrument and operator, indicating that there are significant

differences in the measured response caused by variability between the operators and variability between the instruments. A detailed presentation of this example is given below.

Which do you prefer?

- T-test p = 0.16, ignoring other variables.
- Regression p = 0.00138, controlling for other variables.

If you want good p-values, include variables that affect the response in your analyses, even if those variables are perfectly balanced across your treatment groups.

If you want ways to quickly and efficiently identify nuisance variables that are affecting a response, see the chapter "Rescue a failed experiment using fractional factorial designs".

11.3 Detailed presentation for example 1

We wish to know if a drug affects a serum protein. The serum protein is known to change with age. We have the following data.

```
protein.data
```

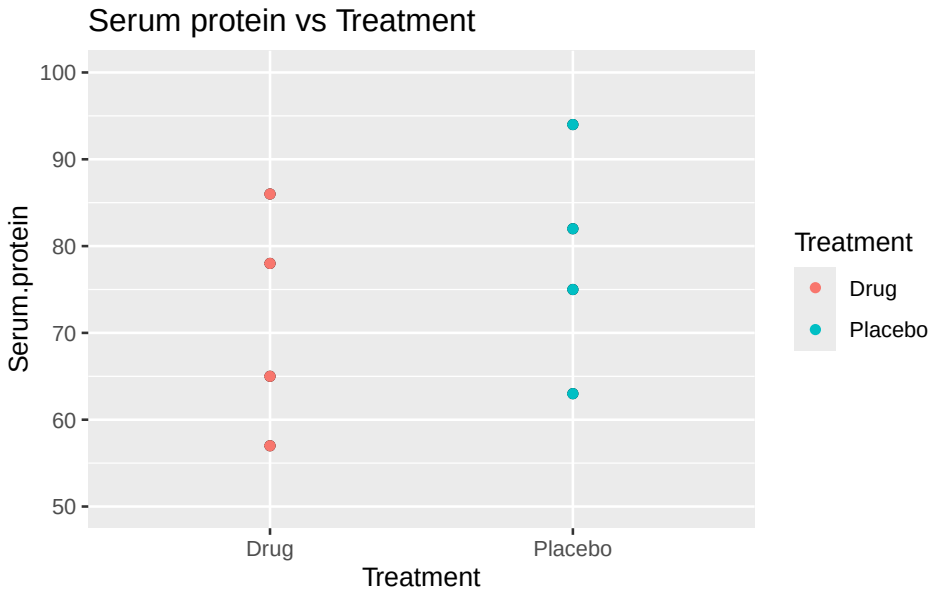
```
# A tibble: 8 x 4
  Subject   Age Treatment Serum.protein
    <dbl> <dbl> <chr>             <dbl>
1       1    40 Placebo              63
2       2    40 Drug                 57
3       3    50 Placebo              75
4       4    50 Drug                 65
5       5    60 Placebo              82
6       6    60 Drug                 78
7       7    70 Placebo              94
8       8    70 Drug                 86
```

We load the library(car) to use the Anova() function instead of the default anova() function. The car Anova() function avoids problems that can occur with correlated explanatory variables. Notice the upper case A starting the Anova() function.

```
library(car)
```

Plot the data.

```
ggplot(protein.data, aes(y=Serum.protein, x=Treatment)) +
geom_point() +
ggtitle("Serum protein vs Treatment") + ylim(50, 100) +
  geom_point(aes(color=Treatment))
```



Perform a t-test.

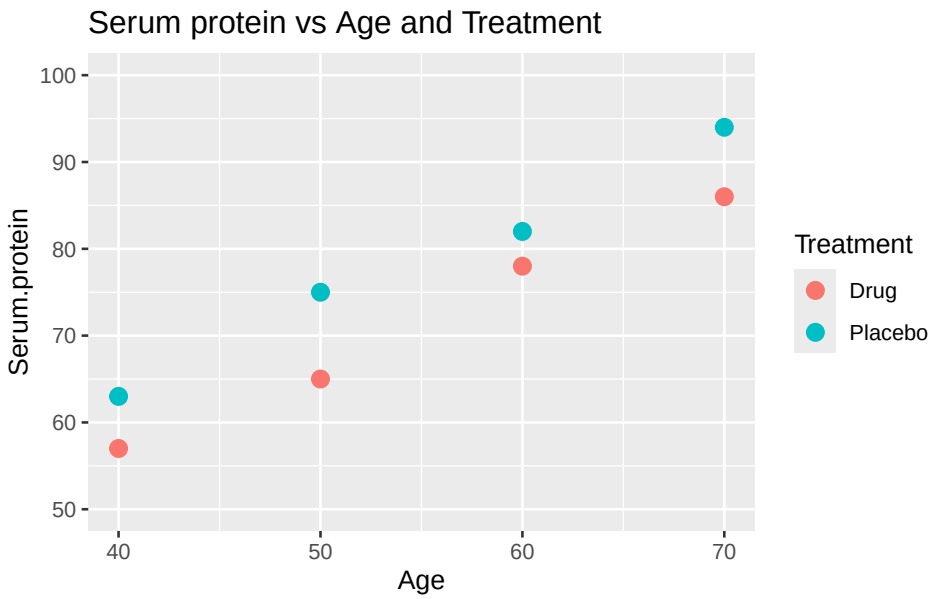
```
t.test(Serum.protein ~ Treatment, data=protein.data,
var.equal=TRUE)
```

```
    Two Sample t-test

data:  Serum.protein by Treatment
t = -0.76301, df = 6, p-value = 0.4744
alternative hypothesis: true difference in means between group
Drug and group Placebo is not equal to 0
95 percent confidence interval:
 -29.44855  15.44855
sample estimates:
   mean in group Drug mean in group Placebo
                 71.5                  78.5
```

The p-value is 0.4744, indicating that the drug has no effect on the serum protein.

However, if we plot serum protein values by age, we see an important pattern. Here is a plot showing serum protein with both age and treatment.

```
graph.1
```



The plot indicates that, within each age group, the drug is having an effect. But the t-test cannot detect that effect, because of the additional variance caused by age.

Multiple regression model including both Age and Treatment

The better method is to include both age and treatment as explanatory variables in the analysis, using either ANOVA or linear regression. Doing so removes the otherwise unexplained variability caused by age. Here is the result using lm and the Anova() function from library(car).

```
Anova(lm(Serum.protein ~ Treatment + Age, data =
protein.data))
```

```
Anova Table (Type II tests)

Response: Serum.protein
          Sum Sq Df F value    Pr(>F)
```

```
Treatment     98  1      49 0.0009167 ***
Age         1000  1     500 3.324e-06 ***
Residuals     10  5
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

The regression analysis gives us p = 0.000917 for treatment.

11.4 Detailed presentation for example 2: How to adjust for two or more variables.

In this example we adjust for two explanatory variables in addition to the treatment. The same principles apply to adjust for more than two explanatory variables.

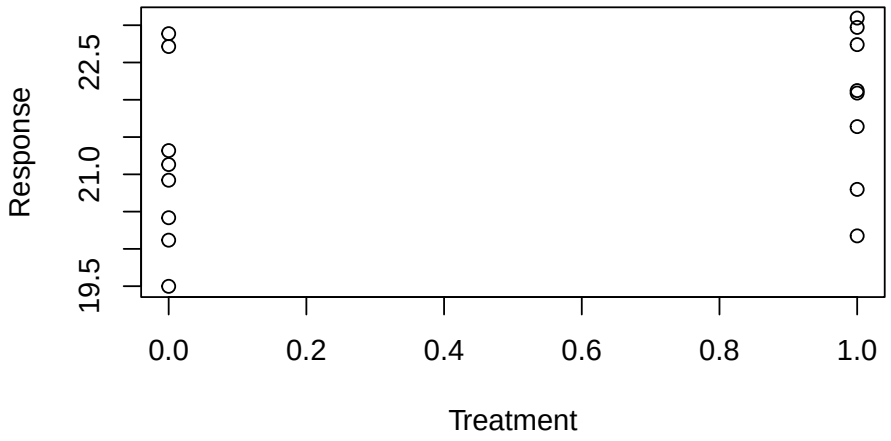
We have 4 variables in the data:

- a Response variable
- a Treatment variable
- Operator, a nuisance variable that also affects the response.
- Instrument, a nuisance variable that also affects the response.

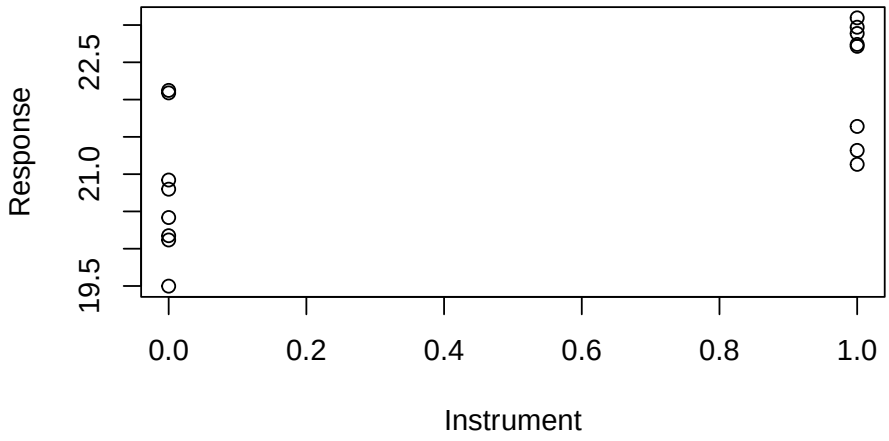
```
experiment.2.data

   Treatment Instrument Operator Response
1          0          0        0 19.49781
2          0          1        0 21.13153
3          0          0        1 20.92108
4          0          1        1 22.88678
5          0          0        0 20.11697
6          0          1        0 21.31863
7          0          0        1 20.41821
8          0          1        1 22.71453
9          1          0        0 20.17474
10         1          1        0 21.64014
11         1          0        1 22.08989
12         1          1        1 23.09627
13         1          0        0 20.79837
14         1          1        0 22.73984
15         1          0        1 22.12338
16         1          1        1 22.97068
```

Notice the perfect balance of instrument and operator across the treatment groups. Let's look at some plots of response versus Treatment, Instrument, and Operator.

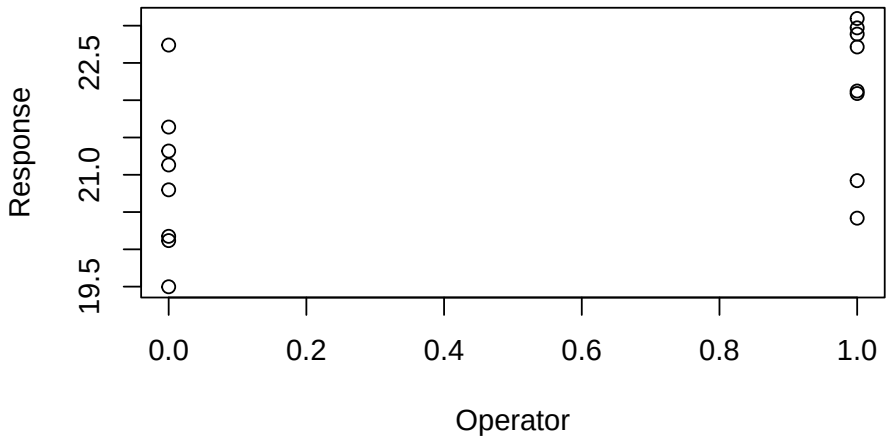
```
with(experiment.2.data, plot(Response ~ Treatment))
```




```
with(experiment.2.data, plot(Response ~ Instrument))
```




```
with(experiment.2.data, plot(Response ~ Operator))
```



The graphs suggest that Treatment, Operator, and Instrument all affect the response.

First, we'll try a t-test of Response ~ Treatment with no adjustment for Instrument or Operator.

```
with(experiment.2.data, t.test(Response ~ Treatment,
var.equal=TRUE))
```

```
    Two Sample t-test

data:  Response by Treatment
t = -1.483, df = 14, p-value = 0.1602
alternative hypothesis: true difference in means between group
0 and group 1 is not equal to 0
95 percent confidence interval:
 -2.026618  0.369678
sample estimates:
mean in group 0 mean in group 1
       21.12569        21.95416
```

The t-test gives us p = 0.16 for treatment, indicating that it has no effect on the response.

Next, let's do a multiple regression model including Response ~ Treatment + Instrument + Operator.

```
with(experiment.2.data,
  summary(lm(Response ~ Treatment + Instrument + Operator)))
```

```
Call:
lm(formula = Response ~ Treatment + Instrument + Operator)

Residuals:
     Min       1Q   Median       3Q      Max
-0.54779 -0.27434 -0.00584  0.30379  0.62598

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  19.7406     0.2003  98.574  < 2e-16 ***
Treatment     0.8285     0.2003   4.137  0.00138 **
Instrument    1.5447     0.2003   7.714 5.45e-06 ***
Operator      1.2254     0.2003   6.119 5.19e-05 ***
---
```

```
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.4005 on 12 degrees of freedom
Multiple R-squared:  0.9048,    Adjusted R-squared:  0.881
F-statistic: 38.02 on 3 and 12 DF,  p-value: 2.092e-06
```

In the output, the column labeled "Pr(>|t|)" gives the p-values. The multiple regression analysis gives p = 0.00138 for treatment. It also gives $p < 0.05$ for instrument and operator, indicating that there are significant differences in the measured response caused by variability between the operators and variability between the instruments. It would probably be a good idea to get the operators together to agree on the protocol, and to get the two instruments calibrated.

Which do you prefer?

- T-test p = 0.16, ignoring other variables.
- Regression p = 0.00138, controlling for other variables.

12 Avoid correlated explanatory variables in multiple regression and ANOVA

When the explanatory (x) variables in a regression analysis are correlated they can cause several problems, including making your p-values non-significant, and giving misleading results. If you have correlated variables in a regression model, an explanatory variable that is in fact associated with the response can get p-values much greater than 0.05. In extreme cases, even the apparent direction of the effect (positive versus negative) can be reversed.

Here are some examples of correlated explanatory variables in biology.

- Age, height, and weight
- Expression of genes in the same pathway or with related function
- Patient responses to related questions on a questionnaire
- Levels of cell cycle proteins in cancer specimens
- Expression of heavy and light chains in an immunoglobulin

```
library(tidyverse)
```

12.1 Example: How correlated explanatory variables affect linear regression

To illustrate the problems caused by correlated explanatory variables, we'll examine a response variable as a function of the drug dose, age, and weight in mice aged 3 to 8 weeks.

Here is the data set.

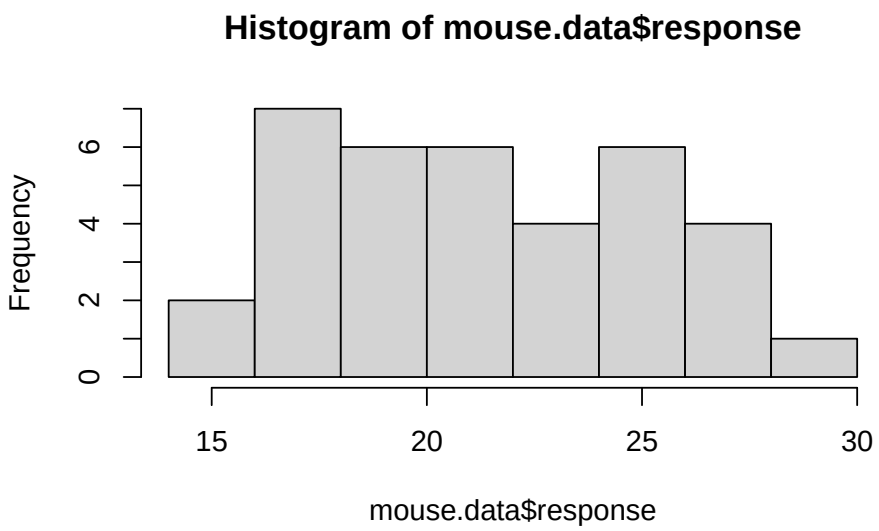
```
response =
c(25,22,18,16,19,18,22,17,18,15,20,23,19,26,22,24,25,21,27,24,27,
19,22,26,27,29,25,22,18,19,27,25,23,18,19,17)
```

```
weight =
c(9,10,12,19,20,24,7,11,12,14,17,22,9,11,12,15,18,25,9,10,13,13,17,
20,8,10,11,16,19,25,10,10,12,15,21,24)

dose=c(rep(0,6), rep(1,6), rep(2,6), rep(3,6), rep(4,6),
rep(5,6))
age = rep(3:8, 6)

mouse.data = data.frame(response, dose, age, weight)

head(mouse.data)
```

```
  response dose age weight
1       25    0   3      9
2       22    0   4     10
3       18    0   5     12
4       16    0   6     19
5       19    0   7     20
6       18    0   8     24
```

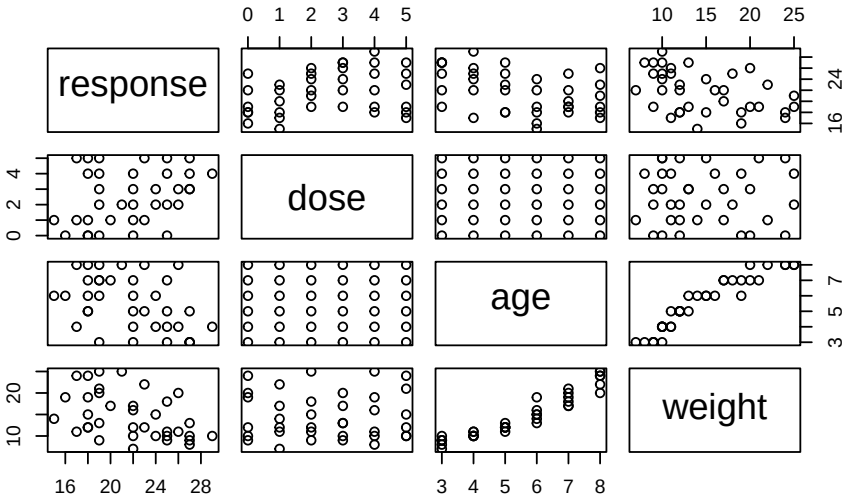
Let's do some plots to examine the data.

```
hist(mouse.data$response)
```



The response variable looks reasonably symmetric without large outliers.

```
pairs(mouse.data)
```



The plots don't indicate a lot of outliers or greatly non-normal distributions, so for the moment we'll use standard analysis methods. There does appear to be a strong correlation of age with weight, as you would expect in a population of mice from age 3 to 8 weeks.

Let's look at the correlation coefficients for each pair of variables, rounded to three decimal places.

```
round(cor(mouse.data), 3)
```

```
         response  dose    age weight
response    1.000 0.304 -0.428 -0.430
dose        0.304 1.000  0.000  0.000
age        -0.428 0.000  1.000  0.949
weight     -0.430 0.000  0.949  1.000
```

Notice that weight and age are highly correlated (R=0.949). Correlated explanatory variables can cause a variety of problems in regression analysis, as we'll see shortly. Dose is correlated with response, which suggests the drug is having an effect. In addition, age and weight are both negatively correlated with response, indicating that the response goes down as age or weight go up. This negative correlation might occur if the drug dose is effectively diluted in the older heavier animals.

Now, let's see how these correlations affect what we might conclude from some regression analyses.

MODEL 1: RESPONSE = DOSE + AGE + WEIGHT

First, we'll build a full model that includes all three variables dose + age + weight.

```
lm.3vars= lm(response ~ dose +  age + weight, data=
mouse.data)
summary(lm.3vars)
```

```

Call:
lm(formula = response ~ dose + age + weight, data =
mouse.data)

Residuals:
    Min      1Q  Median      3Q     Max
-5.6982 -2.9042  0.3242  2.4815  5.8674

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  25.0037     2.0768  12.039 2.01e-13 ***
dose          0.6571     0.3239   2.029   0.0509 .
age          -0.4323     1.0298  -0.420   0.6775
weight       -0.1692     0.3365  -0.503   0.6185
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 3.319 on 32 degrees of freedom
Multiple R-squared:  0.281, Adjusted R-squared:  0.2136
F-statistic: 4.168 on 3 and 32 DF,  p-value: 0.01338
```

The summary indicates that dose is not significantly associated with response ($p = 0.0509$). Age is not significantly associated with response ($p = 0.6775$), and weight is also non-significant ($p = 0.6185$). The Adjusted R-squared = 0.2136 indicates that the model explains about 21% of the variability in the response.

Some researchers might stop at this point and conclude that dose, age and weight do not have a significant effect on the response. However, this would be an incorrect conclusion.

One strategy that researchers sometimes use is to remove non-significant variables from a regression model. Let's look at that method.

MODEL 2: RESPONSE = DOSE

The second model we examine, Model 2, is the linear regression of RESPONSE versus DOSE, without AGE or WEIGHT in the model.

```
lm.dose = lm(response ~ dose, data= mouse.data)
summary(lm.dose)
```

```

Call:
lm(formula = response ~ dose, data = mouse.data)

Residuals:
    Min      1Q  Median      3Q     Max
-6.4206 -2.8706 -0.2635  2.8008  6.2365

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  20.1349     1.0688  18.839   <2e-16 ***
dose          0.6571     0.3530   1.862   0.0713 .
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 3.617 on 34 degrees of freedom
Multiple R-squared:  0.09249,   Adjusted R-squared:  0.0658
F-statistic: 3.465 on 1 and 34 DF,  p-value: 0.07133
```

In Model 2, the p-value for DOSE is still not significant ($p = 0.0713$). However, the Adjusted R-squared = 0.0658 indicates that this model explains much less of the variance in RECOVER than Model 1 which included age and weight, and had Adjusted R-squared = 0.21, despite the fact that age and weight were not significant in Model 1.

Removing both age and weight from the model leaves a large amount of unexplained variability in the response, making it harder to get a significant p-value for dose.

The observation that the model without AGE and WEIGHT gives a much worse fit to the data (low R-squared = 0.0658) plus the observed high correlation between AGE and WEIGHT suggests that correlation may be causing a problem in the regression.

The estimated variance of the AGE and WEIGHT parameters are greatly inflated (Variance Inflation) because of this correlation, giving us the non-significant p-values.

One way to deal with this correlation problem is to create models that include only one of the two correlated variables, which is what we do in Model 3 and Model 4.

MODEL 3: RESPONSE = DOSE + AGE

```
lm.dose.age = lm(response ~ dose + age, data = mouse.data)
summary(lm.dose.age)
```

```

Call:
lm(formula = response ~ dose + age, data = mouse.data)

Residuals:
    Min      1Q  Median      3Q     Max
-5.3302 -2.7421  0.2222  2.0841  6.2032

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  25.2159     2.0103  12.543 4.19e-14 ***
dose          0.6571     0.3202   2.052  0.04815 *
age          -0.9238     0.3202  -2.885  0.00685 **
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 3.281 on 33 degrees of freedom
Multiple R-squared:  0.2753,    Adjusted R-squared:  0.2314
F-statistic: 6.268 on 2 and 33 DF,  p-value: 0.004929
```

In this model, including AGE but excluding WEIGHT, the p-value for AGE is significant (p = 0.00685). The p-value for dose is also now significant (p = 0.048). The model adjusted R-squared of 0.23 indicates that this model explains 23% of the variance in the RESPONSE, similar to that for the full model (Model 1) that also included weight.

MODEL 4: RESPONSE = DOSE + WEIGHT

```
lm.dose.weight = lm(response ~ dose + weight, data=
mouse.data)
summary(lm.dose.weight)
```

```
Call:
lm(formula = response ~ dose + weight, data = mouse.data)

Residuals:
    Min      1Q  Median      3Q     Max
-6.0111 -2.8682  0.2899  2.6433  5.4945

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  24.6003     1.8180  13.531 5.09e-15 ***
dose          0.6571     0.3198   2.055  0.04790 *
weight       -0.3033     0.1045  -2.902  0.00656 **
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 3.277 on 33 degrees of freedom
Multiple R-squared:  0.277, Adjusted R-squared:  0.2332
F-statistic: 6.322 on 2 and 33 DF,  p-value: 0.004739
```

In this model, including WEIGHT but excluding AGE, the p-value for WEIGHT is significant (p=<0.00656). The p-value for dose is also now significant (p = 0.0479). The model adjusted R-squared of around 0.23 is similar to that for the full model (Model 1).

12.2 What have we learned, and what should we do in future experiments?

These four models illustrate that, when we have correlated variables, they can lead to non-significant p-values for the correlated parameters and give us quite misleading conclusions about the effect of the variables on the response.

By dealing appropriately with the correlated variables, we end up with a significant p-value for the treatment.

This type of analysis should be pre-specified in the experiment protocol and the statistical analysis plan in your laboratory notebook. It is not a good idea to wait until after the results come back non-significant and then start looking for correlated variables. A better practice is to look at the correlation matrix before performing any regression analyses. If, instead, you wait till after the analyses are done, and then start looking at different combinations of

variables to avoid correlation, you create problems with multiple hypothesis testing. See the chapter on "The wrong way to get $p < 0.05$".

R has some additional tools that help detect collinearity problems; these tools include the Tolerance, Variance Inflation Factor (VIF) and Collinearity Diagnostics.

The regression estimates of the coefficients for correlated variables may change quite a lot when there are even small changes in observations included in the data set. A further problem is that the estimated variance of the parameters may be greatly inflated (Variance Inflation) which in turn can lead to non-significant p-values for the correlated parameters.

12.3 How to deal with correlated variables

The most common ways to deal with correlated variables are:

- Remove one or more of the correlated variables from the model. Stepwise variable selection may be useful.
- Combine the correlated variables (for example, standardize the variables and then take the average).
- Use an alternative regression methodology, such as lasso, ridge regression, principal components regression or partial least squares.

As we noted above, in the model including WEIGHT but excluding AGE, the p-value for WEIGHT is significant and the model adjusted R-square is close to that for the full model. A plausible biological explanation is that a given dose is more diluted in a heavier animal. Dropping the AGE variable may be a good strategy here. Sometimes when we have correlated variables one of the variables is cheap and easy to measure, whereas the other variables may be expensive and hard to measure. Non-statistical issues may determine the choice of what to include in the model.

12.4 Final example: the OncotypeDx breast cancer recurrence test

Back around the year 2000, I was consulting for Genomic Health, developing what would become the algorithm for the OncotypeDx breast cancer recurrence test. Among the candidate genes for inclusion were a large number

of cell cycle genes. Using only one of these genes did not give us as good predictions as combining several of them. But the cell cycle genes were quite correlated with each other, and each additional gene added to the cost of the assay. After trying several different ways to combine or select among the cell cycle genes, I found that including a small number of them with relatively low pairwise correlations gave a regression model with good predictive accuracy. Freedom to operate in terms of patents and intellectual property also played a role. So sometimes statistics is not the only concern :)

13 Look (out) for interactions: when subgroups respond differently to treatment

In your experiments, you likely have seen subgroups that appear to respond differently to the same treatment. In medicine, the effect of a treatment on a patient can depend on the patient's age, sex, genetics, disease history, disease severity, and many other factors. When the effect of a treatment depends on the level of another variable (such as genetics, age, sex, disease severity) there is an interaction. Undetected interactions can make p-values worse or even non-significant, as we'll demonstrate.

We sometimes encounter a group of outliers in a data set, such as we saw in the chapter on robust analysis. A group of similar outliers can be a clue that there is an interaction: one population (subgroup) is responding differently from the majority of observations that belong to another population (subgroup).

One reason you want to look for interactions is that identifying them can contribute substantially to better understanding of the underlying biology or disease. A reason to look out for them is that, if undetected, they can hurt your p-values.

The chapter on interactions in the companion book, "A Gentle Introduction to Statistics for Biological Research" provides details of methods to analyze interactions. In this chapter you will see some examples of how interactions can affect p-values and methods you can use to detect and analyze interactions to get better p-values.

13.1 Example: how interactions can make p-values worse

A post-doc researcher, whom we'll call Patricia, was interested in the effects of caloric restriction (CR) on the performance of mice on a learning test. She

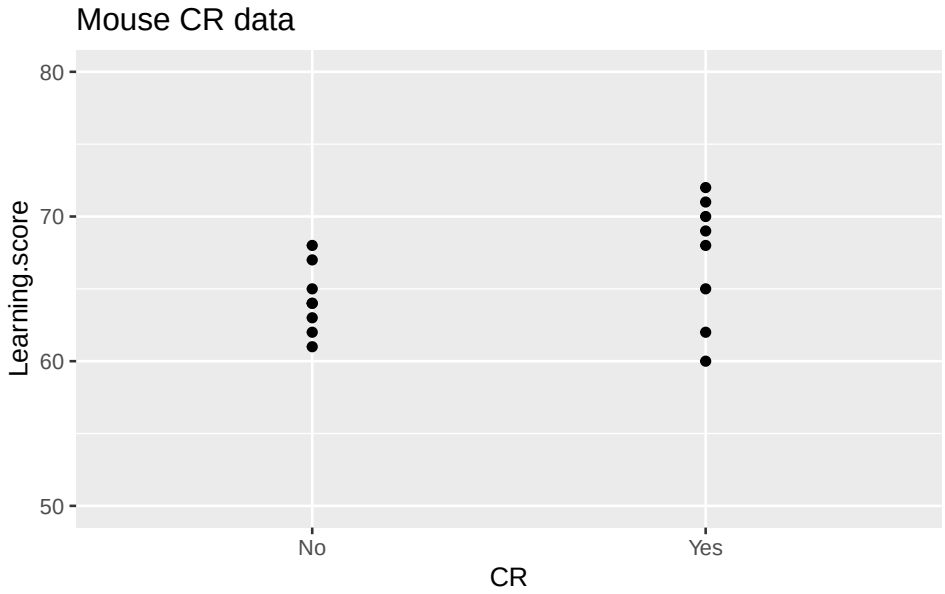
performed an experiment and collected the following data.

```
CR.data = data.frame(CR = c("Yes", "Yes", "Yes", "Yes", "Yes",
"Yes", "Yes", "Yes", "No", "No", "No", "No", "No", "No", "No",
"No"),
Learning.score = c(65, 68, 72, 60, 69, 71, 62, 70, 64, 67, 62,
61, 68, 63, 64, 65))

CR.data
```

```
    CR Learning.score
1  Yes              65
2  Yes              68
3  Yes              72
4  Yes              60
5  Yes              69
6  Yes              71
7  Yes              62
8  Yes              70
9   No              64
10  No              67
11  No              62
12  No              61
13  No              68
14  No              63
15  No              64
16  No              65
```

The graph of learning score versus CR did not look too promising.

```
library(ggplot2)
mouse.CR.graph = ggplot(CR.data, aes(y=Learning.score, x=CR))
+
  geom_point() + ggtitle("Mouse CR data") +
  scale_y_continuous(breaks=0:80*10, limits=c(50, 80))
plot(mouse.CR.graph)
```



The t-test, implemented using either the t.test function or the lm function, was not significant ($p = 0.12$).

```
with(CR.data, t.test(Learning.score ~ CR, var.equal = TRUE))
```

```
    Two Sample t-test

data:  Learning.score by CR
t = -1.6387, df = 14, p-value = 0.1235
alternative hypothesis: true difference in means between group
No and group Yes is not equal to 0
95 percent confidence interval:
 -6.6379395  0.8879395
sample estimates:
 mean in group No mean in group Yes
           64.250            67.125
```

```
# t-test using lm
mouse.lm.CR=with(CR.data, lm(Learning.score ~ CR))
summary(mouse.lm.CR)
```

```

Call:
lm(formula = Learning.score ~ CR)
```

```
Residuals:
   Min     1Q Median     3Q    Max
-7.125 -2.156  0.250  2.781  4.875

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)   64.250      1.241  51.790   <2e-16 ***
CRYes          2.875      1.754   1.639    0.124
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 3.509 on 14 degrees of freedom
Multiple R-squared:  0.1609,    Adjusted R-squared:  0.101
F-statistic: 2.685 on 1 and 14 DF,  p-value: 0.1235
```

Patricia thought that the age of the mice might affect learning scores. She went back and collected the age information. There were 10 old mice and 6 young mice.

```
CR.Age.data = data.frame(CR = c("Yes", "Yes", "Yes", "Yes",
"Yes", "Yes", "Yes", "Yes","No", "No", "No", "No", "No", "No",
"No", "No"),
Learning.score = c(65, 68, 72, 60, 69, 71, 62, 70, 64, 67, 62,
61, 68, 63, 64, 65),
Age = c("Young", "Old", "Old", "Young", "Old", "Old", "Young",
"Old", "Young", "Young", "Old", "Young", "Old", "Old", "Old",
"Old"))

CR.Age.data
```

```
    CR Learning.score   Age
1  Yes             65 Young
2  Yes             68   Old
3  Yes             72   Old
4  Yes             60 Young
5  Yes             69   Old
6  Yes             71   Old
7  Yes             62 Young
8  Yes             70   Old
9   No             64 Young
10  No             67 Young
```

```
11  No              62   Old
12  No              61 Young
13  No              68   Old
14  No              63   Old
15  No              64   Old
16  No              65   Old
```

Patricia ran a 2-way analysis of variance, using the lm function, and included both CR and age as the explanatory variables.

```
mouse.lm.CR.Age=with(CR.Age.data, lm(Learning.score ~ CR +
Age))
summary(mouse.lm.CR.Age)
```

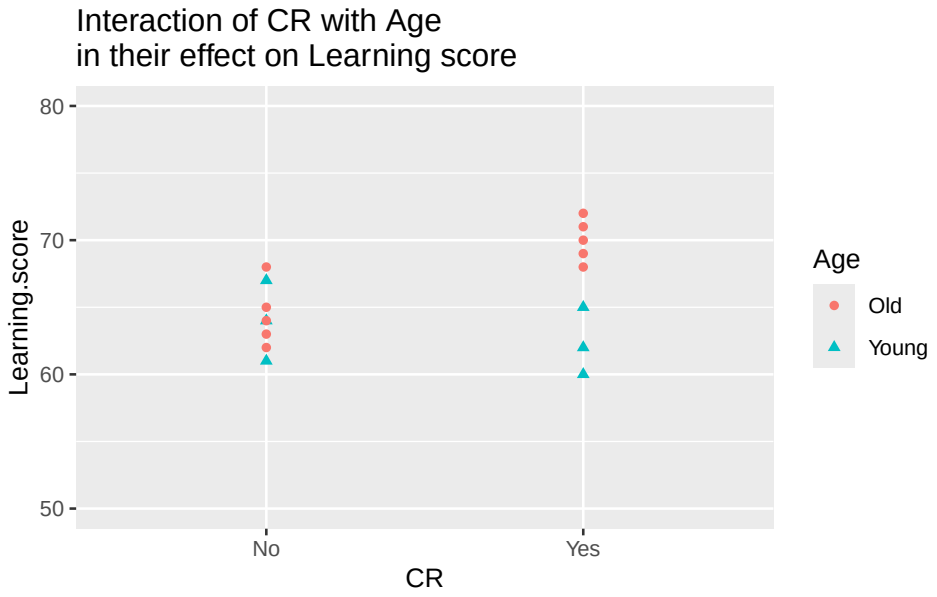
```

Call:
lm(formula = Learning.score ~ CR + Age)

Residuals:
    Min      1Q  Median      3Q     Max
-4.6042 -1.9729 -0.1375  2.2458  5.2708

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)   65.762      1.180  55.736   <2e-16 ***
CRYes          2.875      1.463   1.965   0.0712 .
AgeYoung      -4.033      1.511  -2.668   0.0193 *
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 2.927 on 13 degrees of freedom
Multiple R-squared:  0.4579,    Adjusted R-squared:  0.3745
F-statistic:  5.49 on 2 and 13 DF,  p-value: 0.01869
```

Unfortunately, even controlling for age, the p-value for CR was not significant ($p = 0.0712$), though it did improve compared to the t-test ($p = 0.124$).

Patricia decided to take a closer look at the graph of the learning scores, labeling them with both CR and age.

```
mouse.interaction.graph = ggplot(CR.Age.data,
```

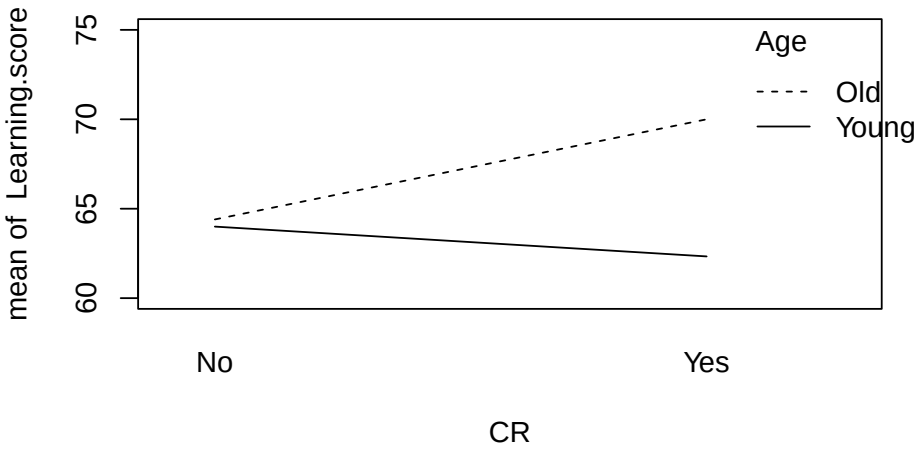
```
                                    aes(y=Learning.score, x=CR,
                                    group=Age, col=Age,
                                    shape=Age)) +
  geom_point() +
  ggtitle("Interaction of CR with Age \nin their effect on
  Learning score") +
  scale_y_continuous(breaks=0:80*10, limits=c(50, 80))

plot(mouse.interaction.graph)
```



The graph suggested that CR was having an effect in the old mice but possibly not having an effect in the young mice.

The R function interaction.plot is another helpful way to visualize the data.

```
with(CR.Age.data, interaction.plot(CR, Age,  Learning.score,
ylim = c(60, 75)))
```



The interaction plot draws two lines:

- The first, dotted line, connects the mean learning scores for CR = No to CR = Yes among the old mice.
- The second, solid line, connects the mean learning scores for CR = No to CR = Yes among the young mice.

If there was no interaction, these two lines would be parallel (have the same slope). Instead, these lines are not parallel. They have different slopes.

This is a classic interaction: the effect of treatment (CR) on learning scores is different in old mice compared to young mice.

To find interactions you can examine graphs, as we have done. In addition, you use analysis of variance to do a formal hypothesis test for significant interactions. The interaction test is a test to see if the lines in the interaction plots all have the same slope. If the lines in the interaction plots have the same slope there is no interaction. The alternative is that the lines do not have the same slope. If the interaction test gives $p < 0.05$, we conclude that the lines have different slopes, and that there is an interaction.

To perform the interaction test, you need to make one change in the R code. Here is the R code that we used before, without the interaction test:

```
mouse.lm.CR.Age = with(CR.Age.data, lm(Learning.score ~ CR + Age))
```

Notice that there is a + sign in the formula (Learning.score ~ CR + Age).

To perform the interaction test, you change the + sign in the formula to a *, like this:

mouse.lm.CR.Age.interaction = with(CR.Age.data, lm(Learning.score ~ CR * Age))

Here is the R code, now with the interaction test:

```
mouse.lm.CR.Age.interaction = with(CR.Age.data,
lm(Learning.score ~ CR * Age))
summary(mouse.lm.CR.Age.interaction)
```

```

Call:
lm(formula = Learning.score ~ CR * Age)

Residuals:
    Min      1Q  Median      3Q     Max
-3.0000 -1.5500 -0.1667  1.2500  3.6000

Coefficients:
               Estimate Std. Error t value Pr(>|t|)
(Intercept)      64.400      1.015  63.421  < 2e-16 ***
CRYes             5.600      1.436   3.900  0.00211 **
AgeYoung         -0.400      1.658  -0.241  0.81345
CRYes:AgeYoung   -7.267      2.345  -3.099  0.00921 **
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 2.271 on 12 degrees of freedom
Multiple R-squared:  0.6989,    Adjusted R-squared:  0.6236
F-statistic: 9.283 on 3 and 12 DF,  p-value: 0.001883
```

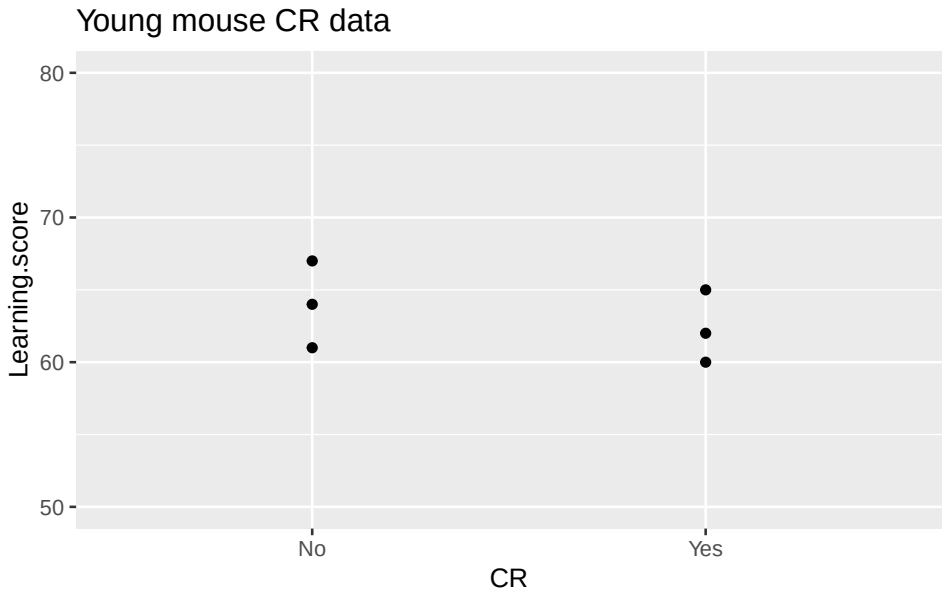
The p-value for the interaction test is in the last row in the coefficients. The text CRYes:AgeYoung, with a colon between the two variable names, indicates it is the interaction test. The interaction test is significant ($p = 0.0092$). Patricia had found that the lines had different slopes, and that there was an interaction.

When you find an interaction, there are several possible ways to examine it more closely. See the chapter on interactions in the companion book for

details. For now, we'll create subsets of the data to examine the young and old mice separately.

Here is the subset and graph for the young mice.

```
young.mouse.data = subset(CR.Age.data, Age=="Young")

young.mouse.CR.graph = ggplot(young.mouse.data,
aes(y=Learning.score, x=CR)) +
  geom_point() +
  ggtitle("Young mouse CR data") +
  scale_y_continuous(breaks=0:80*10, limits=c(50, 80))

plot(young.mouse.CR.graph)
```



Here is the test of learning score versus CR in the young mice.

```
young.mouse.lm      = lm(Learning.score ~ CR,
data=young.mouse.data)
summary(young.mouse.lm)
```

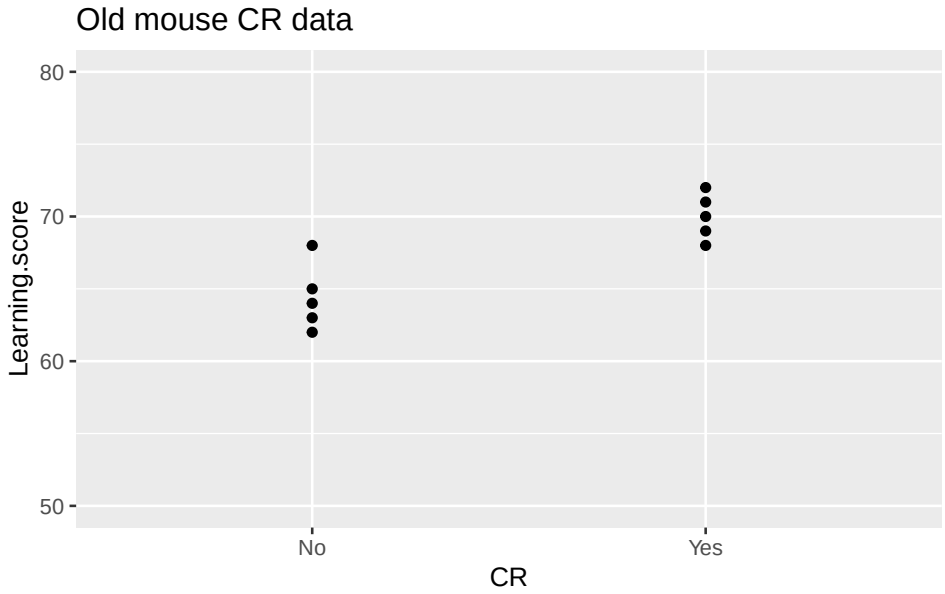
```

Call:
lm(formula = Learning.score ~ CR, data = young.mouse.data)
```

```
Residuals:
         1          4          7          9         10
         12
 2.667e+00 -2.333e+00 -3.333e-01  6.661e-16  3.000e+00
 -3.000e+00

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)   64.000      1.599  40.035 2.33e-06 ***
CRYes         -1.667      2.261  -0.737    0.502
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 2.769 on 4 degrees of freedom
Multiple R-squared:  0.1196,    Adjusted R-squared:  -0.1005
F-statistic: 0.5435 on 1 and 4 DF,  p-value: 0.5019
```

For the young mice, the p-value for CR is not significant (p = 0.5). This indicates that CR has no effect on learning score in young mice. We have to be cautious about this conclusion because the sample size of 3 mice per group is very small and has little power.

Next, the subset and graph for the old mice.

```
old.mouse.data   = subset(CR.Age.data, Age=="Old")
old.mouse.CR.graph = ggplot(old.mouse.data,
aes(y=Learning.score, x=CR)) +
  geom_point() +
  ggtitle("Old mouse CR data") +
  scale_y_continuous(breaks=0:80*10, limits=c(50, 80))
plot(old.mouse.CR.graph)
```



Here is the test of learning score versus CR in the old mice.

```
old.mouse.lm      = lm(Learning.score ~ CR,
data=old.mouse.data)
summary(old.mouse.lm)
```

```

Call:
lm(formula = Learning.score ~ CR, data = old.mouse.data)

Residuals:
   Min     1Q Median     3Q    Max
  -2.4   -1.3   -0.2    0.9    3.6

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  64.4000     0.8832  72.919 1.39e-12 ***
CRYes         5.6000     1.2490   4.484  0.00205 **
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 1.975 on 8 degrees of freedom
Multiple R-squared:  0.7153,    Adjusted R-squared:  0.6797
F-statistic:  20.1 on 1 and 8 DF,  p-value: 0.002046
```

For the old mice, the p-value for CR is significant (p = 0.002). This indicates that CR has an effect on learning scores in old mice.

13.2 What have we seen and learned?

Patricia's original t-test of learning score versus CR gave a non-significant result (p = 0.12).

Adding a second explanatory variable, Age, improved the p-value for CR from p = 0.12 to p = 0.07, and showed that age was significant (p = 0.019).

A careful look at a plot of learning score versus the CR, colored by age, showed that there was likely a significant interaction. The effect of CR was different in old and young mice. A hypothesis test and analysis of subgroups confirmed the significant interaction.

If Patricia had stopped after the first t-test, she would have had a negative result. Instead, by looking (out) for possible interactions, she had a much more interesting result: CR had a significant effect on learning in old mice but did not have a significant effect in young mice.

13.3 Multiple hypothesis testing and interactions

You need to be careful about the multiple hypothesis testing problem when you test for interactions.

Suppose you are examining the effects of 10 microRNAs on a phenotype. You suspect that there may be interactions among the microRNAs in their effect on the phenotype. Even if you examine only the 2-way interactions, there are choose(10,2) = 45 tests:

```
choose(10,2) # Gives 45.
```

```
[1] 45
```

Adding possible 3-way interactions gives you another 120 tests:

```
choose(10,3) # Gives 120.
```

```
[1] 120
```

If you do 45 + 120 = 165 hypothesis tests you are pretty much guaranteed to get some false positive results.

See the chapter on "The wrong way to get $p < 0.05$" for more about the multiple hypothesis testing problem and methods to deal with it.

13.4 When should you look for interactions?

The short answer is, if you have sufficient data, it is likely a good idea to always look for interactions. If you do, and you don't find any, it indicates that your treatment likely has a similar effect across subgroups. If you find interactions that were not previously reported, there is a good chance that you've found something of scientific interest, and you've likely gotten better p-values than you would just doing tests that ignored potential interactions.

Ideally, when you write your experimental plan, you should pre-specify what interactions you will test for. The list should be based on your knowledge of what variables are most likely to interact. Doing a lot of post-hoc interaction tests can cause problems with multiple hypothesis testing. If you do find interactions by post-hoc testing, consider it to be a hypothesis-generating exercise that requires further experiments for confirmation.

14 Use two primary analyses each with alpha 0.025 instead of one with 0.05

Scientific journals more and more require that authors use statistical methods to deal with the multiple hypothesis testing problem. Specifically, they require that the overall probability of a false positive result, after testing multiple hypotheses, remains less than 0.05.

One approach used in experiments and clinical trials is to have a single primary hypothesis, with $p < 0.05$ as the required alpha for significance. If the primary hypothesis is significant (at $p < 0.05$), then you can test the second hypothesis. If the second hypothesis is significant, you can test the third hypothesis, and so on. This is sometimes called hierarchical testing. It requires that you guess the best order for testing the hypotheses.

We're going to look at an alternative that can give you better power and doesn't require that you guess which hypotheses are most likely to be significant.

We're going to make use of the Bonferroni method to adjust for multiple hypothesis testing. The Bonferroni method says, if you want to test two hypotheses and preserve the overall alpha at 0.05, then you test each hypothesis using alpha of $0.05/2 = 0.025$. The Bonferroni method is described in the chapter "The wrong way and the right way to get $p < 0.05$: Multiple hypothesis testing".

Instead of using hierarchical testing, you can have two primary analyses and test each of them with alpha of $0.05/2 = 0.025$ (the Bonferroni method). Perhaps surprisingly, testing two primary analyses, each with $p < 0.025$ as the required alpha for significance, can increase the overall power for the study. Let's see why.

Suppose we think that two genes may independently affect a response. We would be happy if the experiment showed that either of the two genes is associated with the response.

14.1 Power for a test of a single gene using alpha of 0.05

We'll start with a sample size calculation for the hierarchical testing approach, where the primary analysis tests only one gene, with $p < 0.05$ as the required alpha. For this example, we make the following assumptions.

Assumptions:

- Difference in response between wildtype and mutant = delta = 1
- Standard deviation of response within each group = 2
- Desired power = 0.8
- Significance level alpha = 0.05 (sig.level in the R code)

Here is the estimated sample size using the R function power.t.test.

```
power.t.test(delta=1, power = 0.8, sd = 2, sig.level  = 0.05)
```

```
     Two-sample t test power calculation

              n = 63.76576
          delta = 1
             sd = 2
      sig.level = 0.05
          power = 0.8
    alternative = two.sided

NOTE: n is number in *each* group
```

Given these assumptions, the required sample size is 64 in each group. This sample size gives power of 0.8, meaning an 80% chance that the experiment will yield a positive result. Unfortunately, we don't know which of the two genes to specify as the primary and which as the secondary. If we guess wrong, the primary could be non-significant, the secondary could give $p < 0.05$, but because of the rules for hierarchical hypothesis testing we could not claim that the secondary analysis result was significant.

14.2 Power for tests of two genes using alpha of 0.025 each

As the alternative, here let's examine power when the primary analysis tests both of the two genes, each with $p < 0.025$ as the required alpha. We fix the sample size at $n = 64$ per group, so we have the same sample size as for testing one gene as the primary. What power do we have for the test of each gene?

Assumptions:

- Sample size = 64
- Difference in response between wildtype and mutant = delta = 1
- Standard deviation of response within each group = 2
- Significance level alpha = 0.025 (sig.level in the R code)

Here is the estimated power using the R function power.t.test, with sample size fixed as n = 64.

```
power.t.test(n = 64, delta=1, sd = 2, sig.level  = 0.025)
```

```
     Two-sample t test power calculation

              n = 64
          delta = 1
             sd = 2
      sig.level = 0.025
          power = 0.7118426
    alternative = two.sided

NOTE: n is number in *each* group
```

For these assumptions, the power is 0.71 for each gene. However, we are testing two genes as co-primaries, and we will be happy if either of them is significant.

The power calculation above tells us the probability that each of the genes by itself will be significant, which is 0.71. But we want the probability that either gene will be significant.

Let's call the two genes A and B.

The probability that A is significant is 0.71. The probability that A is not significant is 1 - 0.71 = 0.29. P(A is not significant) = 0.29.

Same for B:

The probability that B is significant is 0.71. The probability that B is not significant is 1 - 0.71 = 0.29. P(B is not significant) = 0.29.

From the laws of probability, if two events are independent, the probability that they both occur is the product of their individual probabilities.

So, the probability that both A is not significant and B is not significant is P(A is not significant) * P(B is not significant) = 0.29 * 0.29 = 0.084.

The probability that at least one of them is significant is just 1 minus the probability that both are not significant, which is 1 – 0.084 = 0.916, or 91.6%. There is a 91.6% probability that at least one of the genes will be significant.

To summarize: the probability that one gene will not be significant is 1 – 0.71 = 0.29. If the genes are independent, then the probability they will both fail to be significant is 0.29 * 0.29 = 0.084. The probability that at least one of the two genes will be significant is 1 – 0.084 = 0.916, or 91.6%. Thus, provided we are happy if either gene is significant, our power for the entire experiment is 0.916 or 91.6%. This is an improvement over the power of 80% when we test the genes in a hierarchical order.

The actual difference in power will depend on the effect size (delta) and the standard deviation. But the concept will still apply. The actual power will also depend on the degree of independence, as discussed next.

The key assumption: independent tests versus correlated tests

The key assumption that makes this method work is that the effects of the two genes are independent. If they are not independent, but instead are correlated, then the probability they will both fail is no longer the product of the probability of each of them failing. The actual probability will depend on the degree of correlation. As an example, suppose we are evaluating two doses of a drug. The effects of the two doses are likely not independent; if one dose works well it is plausible that the other dose is more likely to work well. We need to consider if the additional power from testing each with alpha of 0.025 is sufficient to offset the potential loss from correlation.

14.3 Bonferroni and Holm's adjustments for multiple hypothesis testing

We specified alpha of $0.05/2 = 0.025$ to test two hypotheses using the Bonferroni method to adjust for multiple hypothesis testing. If either hypothesis gives $p < 0.025$, then it is significant. If both of the two hypotheses give $p < 0.025$ then both are significant.

An alternative to the Bonferroni method is the Holm's test. Using Holm's test, we first sort the p-values for the two genes. If the smaller of the two p-values is less than 0.025, then the larger of the two p-values must be less than 0.05, rather than less than 0.025, to be considered significant. Instead of requiring that both hypotheses be less than 0.025, you could have one less than 0.025 and the second less than 0.05. Using Holm's will give slightly more power than using Bonferroni.

Both the Bonferroni method and the Holm's test are described in the chapter "The wrong way and the right way to get $p < 0.05$: Multiple hypothesis testing".

15 Avoid one-variable-at-a-time experiments. Use factorial experiment design.

You have likely heard the advice, "Hold everything constant, change one thing at a time"? This is the one-variable-at-a-time (OVAT) approach.

As you will see in this chapter, OVAT is often a very inefficient and ineffective way to perform an experiment. An alternative to OVAT is factorial experiment design, which you may have encountered as Design of Experiments (DOE).

Compared to OVAT, factorial design of experiments has several advantages:

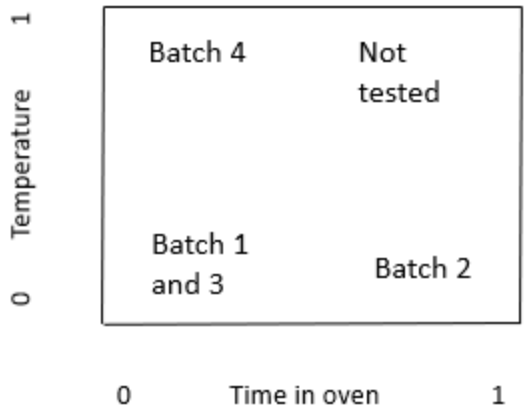
- Quickly and efficiently test the effects of multiple factors.
- Increase power.
- Get better p-values.
- Detect interactions missed by OVAT.

Factorial designs provide more information with less work than OVAT experiments, as we'll see. Let's start with a familiar example: baking cookies.

15.1 An intuitive view of experiment design: baking cookies

Suppose we are baking cookies. We want to maximize the yield of good cookies. We think that the time in the oven and the temperature in the oven could affect the yield. We plan to bake 4 batches using different combinations of time and temperature in the oven.

Plan 1. Here is one possible plan for the 4 batches.



The x axis is time in the oven. 0 is a short time in the oven, and 1 is a long time.

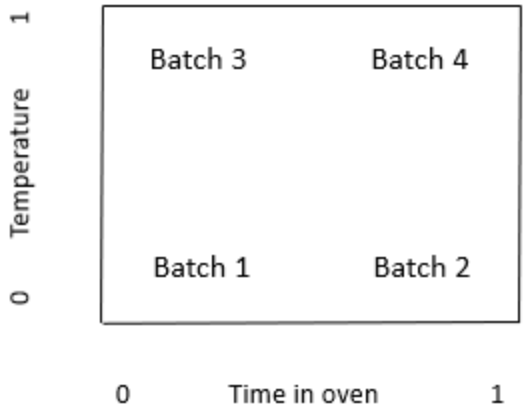
The y axis is temperature in the oven. 0 is a low temperature in the oven, and 1 is a high temperature.

The table shows the time and temperature for each batch.

Batch number	Time	Temperature
1	0	0
2	1	0
3	0	0
4	0	1

Does Plan 1 seem like a good way to design the experiment? Why or why not? Clearly, it seems pretty inefficient and uninformative to test the short time and low temperature (0,0) combination twice, but not to test the short time and high temperature (1,0) combination at all.

Plan 2. Here is an alternative plan for the 4 batches.



The table shows the time and temperature for each batch.

Batch number	Time	Temperature
1	0	0
2	1	0
3	0	1
4	1	1

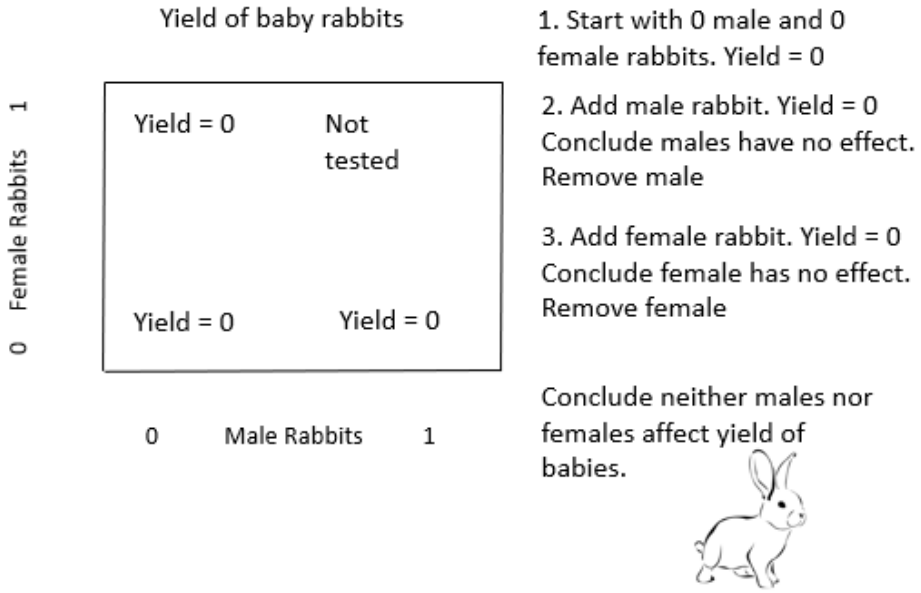
Does Plan 2 look like a better plan?

15.2 The problems with "One variable at a time"

So, what's wrong with one variable at a time? Let's look at an example of the OVAT method from the book "Improving Almost Anything" by George Box (Box 2006).

Suppose we want to maximize the number of baby rabbits. We think that adult male rabbits and adult female rabbits could affect the yield of baby rabbits. We plan to test the effect of different combinations of male and female rabbits.

Here is the experiment, using the "One-variable-at-a-time" method.

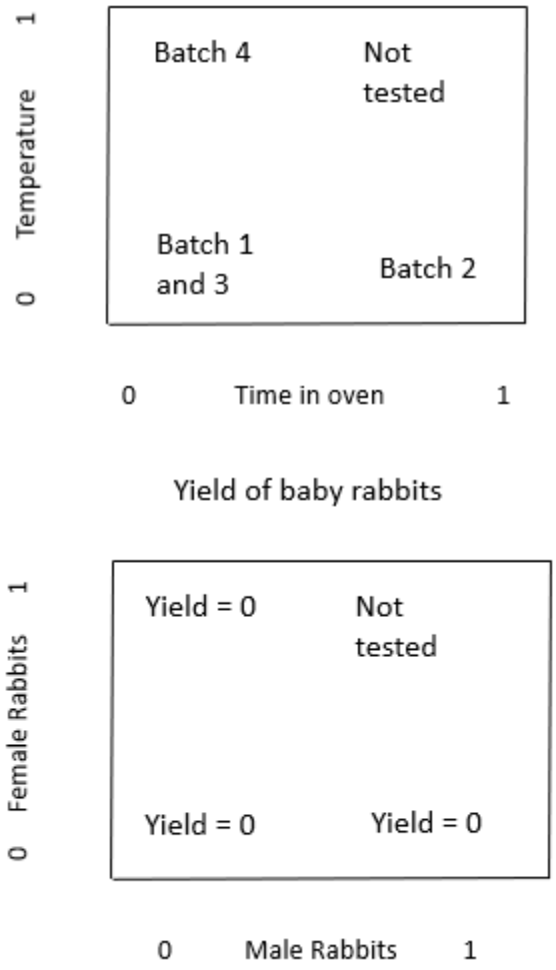


This example illustrates one of the problems with the OVAT method: it misses important interactions. An interaction is when the effect of one factor (male rabbits) depends on the level of another factor (female rabbits).

15.3 Are interactions important in medicine and in biological research?

Does every treatment work for every patient? Clearly no. The effect of a treatment in a given patient can depend on the patient's genetics, age, disease severity, and many other factors. Because the effect of a treatment is different for different genetics, age, and so on, there is an interaction. The entire field of personalized medicine exists because there are important interactions; different people respond differently to the same drug. To make treatments more effective, we need to look at combinations of treatment with genetics, age, disease severity, and so on. We cannot look at only one variable at a time.

Look again at the experiment designs used for the baby rabbits and in Plan 1 for baking.



These two designs don't examine the space of factor combinations efficiently. As we will see, both of these designs are examples of one-variable-at-a-time ("OVAT") experiment designs. OVAT experiments are typically analyzed with t-tests. We will see shortly that OVAT has another problem: It makes very inefficient use of the number of samples, giving low power and poor p-values. Factorial designs can give much more information, and smaller p-values, for the same number of samples.

15.4 Factorial experiment design: an alternative to OVAT

Let's look at an example of a factorial design. This example is also from George Box, "Improving almost anything".

"Engineers (at the bearing manufacturer SKF) wanted to find the effect of changing to a less expensive 'cage' design."

"The results were assessed by an accelerated life test. … The runs were expensive because they needed to be made on an actual production line and the experimenters were planning to make four runs with the standard cage and four with the modified cage."

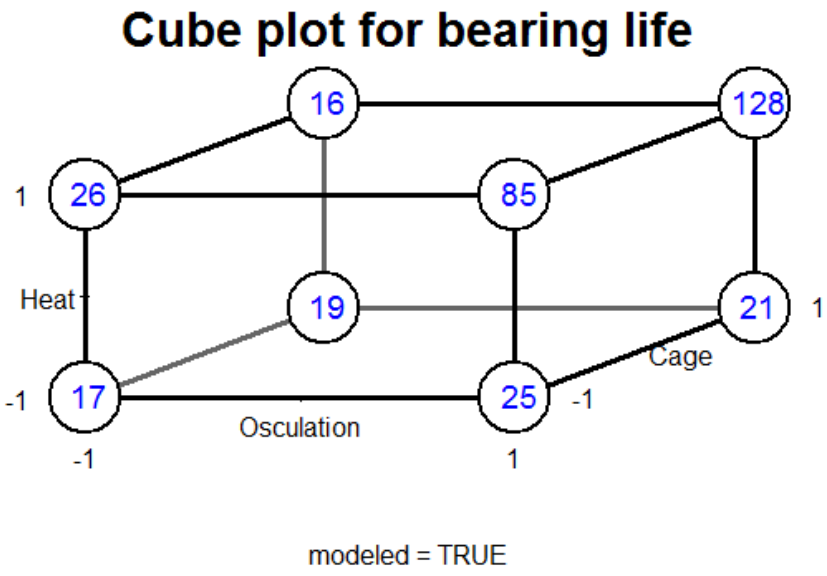
Notice that this is a very typical experiment design: One factor, 2 treatment conditions (levels), 4 runs each, total 8 runs.

"Christer asked if there were other factors they would like to test. They said there were, but that making added runs would exceed their budget. Christer showed them how they could test two additional factors 'for free' – without increasing the number of runs and without reducing the accuracy of their estimate of the cage effect. In this arrangement, called a 2^3 factorial design, each of the three factors would be run at two levels and all the eight possible combinations included. The various combinations can conveniently be shown as the vertices of a cube …"

"Figure 2(b) [the cube plot below] shows a 2^3 factorial experiment reported by Christer Hellstrand…"

Table 15.1: Bearing life versus heat and osculation

	Osculation −	Osculation +
Heat −	18	21
Heat +	23	**106**



"In each case, the standard condition is indicated by a minus sign and the modified condition by a plus sign. The factors changed were heat treatment, outer ring osculation, and cage design. The numbers show the relative lengths of lives of the bearings. If you look at [the cube plot], you can see that the choice of cage design did not make a lot of difference."

"But, if you average the pairs of numbers for cage design, you get the [table below], which shows what the two other factors did. … It led to the extraordinary discovery that, in this particular application, the life of a bearing can be increased fivefold if the two factor(s) outer ring osculation and inner ring heat treatments are increased *together*. This and similar experiments have saved tens of millions of dollars."

"Remembering that bearings like this one have been made for decades, it is at first surprising that it could take so long to discover so important an improvement. A likely explanation is that, because most engineers have, until

recently, employed only one factor at a time experimentation, interaction effects have been missed."

"There are hundreds of thousands of engineers in this country, and even if the 2^3 factorial was the only kind of design they ever used, and even if the only method of analysis that was employed was to eyeball the data, this alone could have an enormous impact on experimental efficiency, the rate of innovation, and competitive position."

This experiment gives an example of an interaction: if both Heat and Osculation are changed at the same time, the effect is much more than just the sum of increasing each of them one at a time.

Further, the variable that the engineers first thought of testing, the Cage design, had very little effect compared to the Heat and Osculation interaction. If the OVAT design had been used, neither Heat nor Osculation would have been tested.

I recommend you get a copy of Box's book "Improving Almost Anything" (Box 2006). You will find it both fun and informative reading.

Now let's apply these concepts to a typical biology experiment.

15.5 Example: experiment design for 3 factors

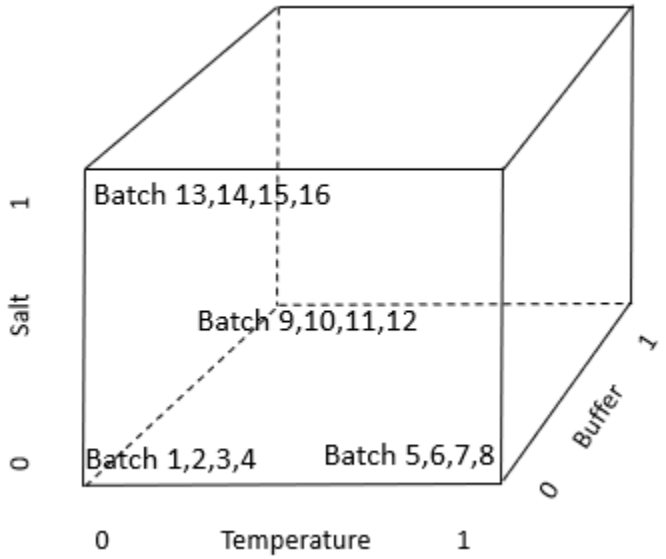
Suppose you are planning an experiment. You are considering changing the temperature, the salt concentration, or the buffer to improve the results (for example, maximizing the yield). How can you efficiently find a good combination of the 3 factors?

Plan 1. Here is one possible plan to test the effects of the 3 variables. We will run 16 batches. The table shows the Temperature, Buffer, and Salt for each of the 16 batches in Plan 1. Notice that this design is following the OVAT rule: only change one variable at a time.

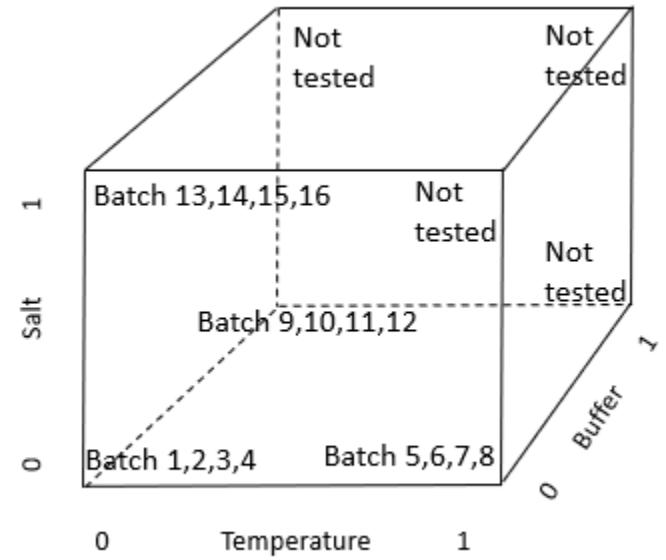
Batch	Treatment combination
1	All factors at level 0
2	All factors at level 0
3	All factors at level 0
4	All factors at level 0
5	Temp level 1. Other factors at 0.
6	Temp level 1. Other factors at 0.

Batch	Treatment combination
7	Temp level 1. Other factors at 0.
8	Temp level 1. Other factors at 0.
9	Buffer level 1. Other factors at 0.
10	Buffer level 1. Other factors at 0.
11	Buffer level 1. Other factors at 0.
12	Buffer level 1. Other factors at 0.
13	Salt level 1. Other factors at 0.
14	Salt level 1. Other factors at 0.
15	Salt level 1. Other factors at 0.
16	Salt level 1. Other factors at 0.

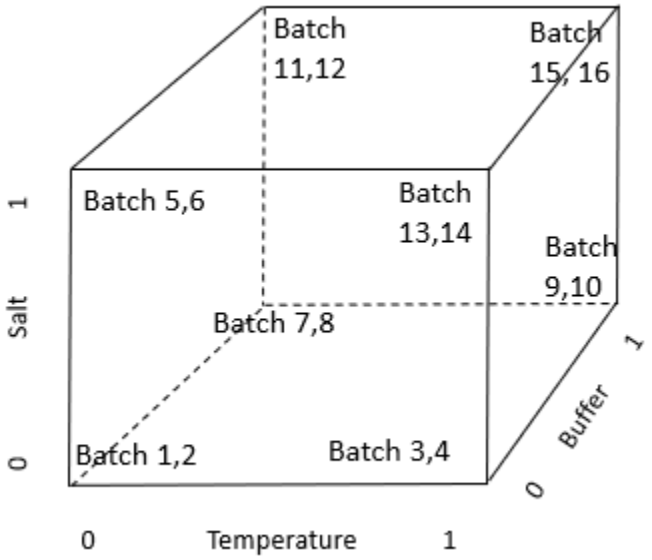
Here is a geometric view of the 16 batches using the OVAT experiment design.



Here is Plan 1 again, now showing explicitly the combinations that are not tested. Does this remind you of the baby rabbit OVAT design? In this OVAT design, many combinations are not tested.



Plan 2. Here is an alternative plan for the 16 batches. This is the factorial design. Notice that every combination of factor levels is tested.



Here is the table showing the treatment assignments for each batch in the

factorial design. For the factorial design, a zero, 0, indicates that the variable is at the current level. A one, 1, indicates that the variable is at the alternative level.

(ii) Factorial Design			
Batch	Temperature	Buffer	Salt
1	0	0	0
2	0	0	0
3	1	0	0
4	1	0	0
5	0	1	0
6	0	1	0
7	0	0	1
8	0	0	1
9	1	1	0
10	1	1	0
11	1	0	1
12	1	0	1
13	0	1	1
14	0	1	1
15	1	1	1
16	1	1	1

With Plan 1 (OVAT) we cannot discover interactions. To do that, we would need to do more batches.

With Plan 2 (factorial design) we can discover interactions. With Plan 2 we can examine all the combinations of the variables.

15.6 Factorial designs have more power than OVAT designs

Factorial designs have another important advantage over OVAT designs: factorial designs have more power to detect effects. To see why, let's compare the two designs side-by-side for the 3-factor experiment.

Table 1. Comparison of (i) a one-variable-at-a-time (OVAT) design to (ii) a factorial design to study 3 variables in 16 batches. For the factorial design, a

zero, 0, indicates that the variable is at the current level. A one, 1, indicates that the variable is at the alternative level.

(i) One variable at a time (OVAT)			**(ii) Factorial Design**			
Batch	Treatment combination		Batch	Temperature	Buffer	Salt
1	All factors at level 0		1	0	0	0
2	All factors at level 0		2	0	0	0
3	All factors at level 0		3	1	0	0
4	All factors at level 0		4	1	0	0
5	Temp level 1		5	0	1	0
6	Temp level 1		6	0	1	0
7	Temp level 1		7	0	0	1
8	Temp level 1		8	0	0	1
9	Buffer level 1		9	1	1	0
10	Buffer level 1		10	1	1	0
11	Buffer level 1		11	1	0	1
12	Buffer level 1		12	1	0	1
13	Salt level 1		13	0	1	1
14	Salt level 1		14	0	1	1
15	Salt level 1		15	1	1	1
16	Salt level 1		16	1	1	1

Notice that, in the OVAT design, we only observe the alternative (1) level of each variable in 4 of the 16 batches. Having only 4 observations limits our power to detect the effects of a variable. Can we observe the alternative level of each variable 8 times, instead of 4 (thus increasing the effective sample size), but still use only 16 batches?

The right-hand side of Table 1 shows the alternative experiment, using a factorial design. For this factorial design, the 16 batches use all possible combinations of the 3 variables. With 3 variables either present or absent, we have $2^3 = 8$ combinations. Including a replicate gives us $2 \times 8 = 16$ batches. Each row in the factorial design in Table 1 gives the variables used in one batch.

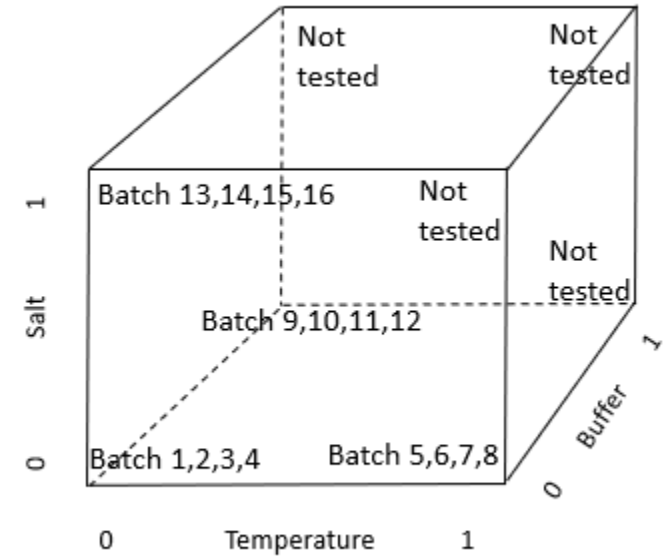
For example, for Batch 1, 0 indicates that temperature, buffer, and salt are all at the current level. For Batch 3, the 1 indicates that temperature is at the alternative level, while 0 indicates buffer and salt are at the current level. Rows 9 to 16 indicate that combinations of variables are used; in rows 15 and 16, the 1 indicates that temperature, buffer, and salt are all at the alternative level. Having more than one variable at alternative levels in a batch allows us to observe interactions between the variables in their effect on yield.

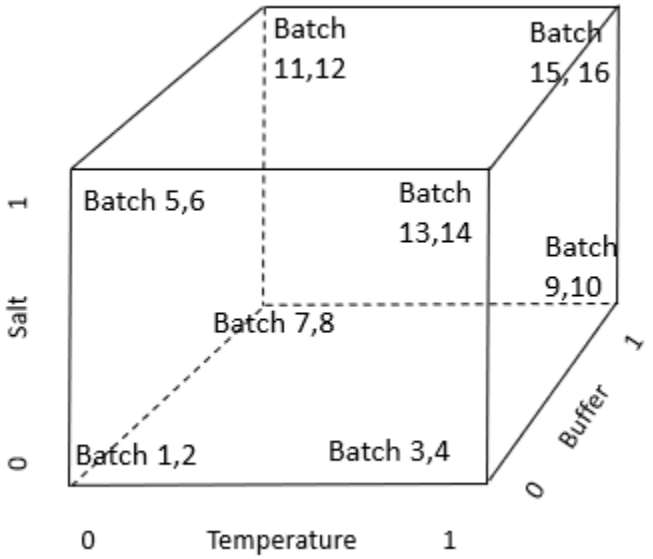
How does the factorial design compare to the OVAT design? Both designs use 16 batches. In the factorial design the effect of each variable is observed 8 times. In the OVAT design the effect of each variable is observed only 4 times. By changing from OVAT to the factorial design, we have doubled the number of observations of the effect of each variable; we have doubled the effective sample size and increased our power to detect the effect of each variable, without increasing the total number of batches. The factorial design also tells us about interactions, which are impossible to detect in the OVAT design. With additional experiments, OVAT might or might not eventually detect interactions.

15.7 What about replication in OVAT versus factorial designs?

If we compare the OVAT and factorial designs for the previous experiment, we see that the OVAT design has 4 replicates of each treatment combination, while the factorial design had 2 replicates of each treatment combination. Is this a reason for concern?

In some circumstances, yes. If the treatment assignments become confused (for example, by pipetting errors), or if specimens are likely to get the wrong treatment, then the error might be detected more readily in an OVAT design.



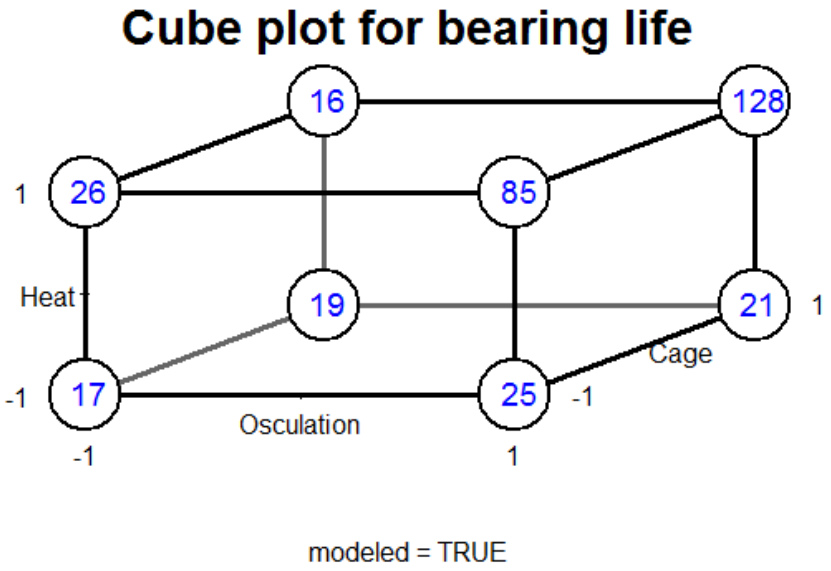


However, the factorial design may still be preferable to OVAT even with labeling errors. In the OVAT design, the alternative level of each factor is only tested 4 times. In the factorial design, the alternative level is tested 8 times. By testing 8 times instead of only 4, the factorial design can provide

greater ability to detect an incorrectly treated or mis-labeled specimen.

15.8 Statistical analysis of factorial designs

Hypothesis tests for factorial designs are not always necessary. In the ball-bearing example, visual inspection of the results was sufficient to show the large interaction effect and to determine which combination gave the best yield.



When we want a p-value, we use analysis of variance or multiple regression to test the treatment effects and interactions. In Part 2 of the book, the chapters on multiway ANOVA, multiple regression, and interactions give more details of this type of analysis.

Here I'll show a multiple regression analysis. In this example, I'll assume there are no interactions. See the chapters in Part 2 on ANOVA and regression with interactions for example interaction analyses.

Suppose we have the following data for the salt, buffer, and temperature example, including the yield.

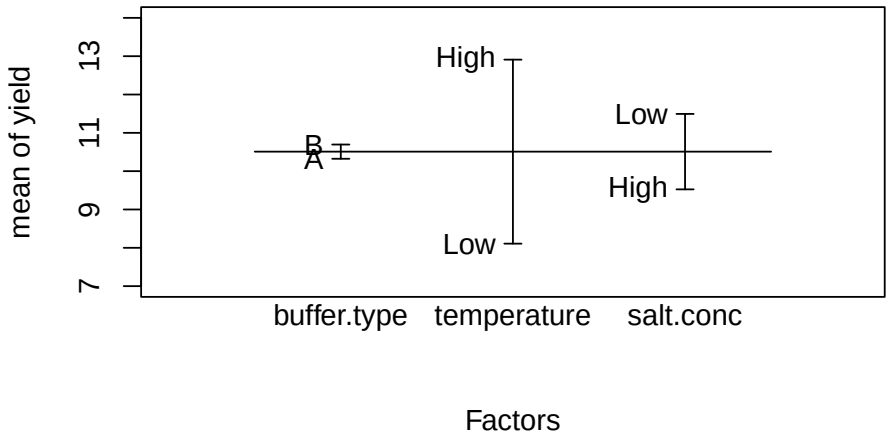
```
temperature = factor(rep(c("Low","Low","High","High"), 4))
salt.conc   = factor(rep(c("Low","High","Low","High"), 4))
```

```
buffer.type = factor(rep(c("A","A","A","A","B","B","B","B"),
2))
yield       = c(7.88, 6.54, 16.12, 11.14, 9.26, 10.43, 13.92,
8.47,
                7.63, 6.11, 15.45, 11.72, 9.80,  7.22, 11.89,
                14.57)
yield.data  = data.frame(temperature, salt.conc, buffer.type,
yield)

yield.data
```

```
   temperature salt.conc buffer.type yield
1          Low       Low           A  7.88
2          Low      High           A  6.54
3         High       Low           A 16.12
4         High      High           A 11.14
5          Low       Low           B  9.26
6          Low      High           B 10.43
7         High       Low           B 13.92
8         High      High           B  8.47
9          Low       Low           A  7.63
10         Low      High           A  6.11
11        High       Low           A 15.45
12        High      High           A 11.72
13         Low       Low           B  9.80
14         Low      High           B  7.22
15        High       Low           B 11.89
16        High      High           B 14.57
```

The R function `plot.design` provides a quick display of the mean yield for each treatment level. Note that, in `plot.design`, the response variable is numeric, and the explanatory factors are text (e.g., "low" and "high").

```
# Graph the results using plot.design.
plot.design(yield ~ buffer.type + temperature + salt.conc,
            data = yield.data, ylim = c(7, 14))
```



The `plot.design` figure indicates the following.

- There is a large effect of temperature on yield. High temperature gives better yield.
- There is a moderate effect of salt concentration on yield. Low salt gives better yield.
- There is very little effect of buffer type on yield.

Here is the regression analysis and the output.

```
yield.data.lm = lm(yield ~ buffer.type + temperature +
salt.conc, data = yield.data)

summary(yield.data.lm)
```

```

Call:
lm(formula = yield ~ buffer.type + temperature + salt.conc,
data = yield.data)

Residuals:
    Min      1Q  Median      3Q     Max
-3.6412 -0.8784 -0.1250  0.8262  3.1200

Coefficients:
```

```
                Estimate Std. Error t value Pr(>|t|)
(Intercept)      11.7400     0.9883  11.879 5.41e-08 ***
buffer.typeB      0.3712     0.9883   0.376 0.713729
temperatureLow   -4.8013     0.9883  -4.858 0.000393 ***
salt.concLow      1.9688     0.9883   1.992 0.069610 .
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 1.977 on 12 degrees of freedom
Multiple R-squared:  0.6978,    Adjusted R-squared:  0.6223
F-statistic: 9.237 on 3 and 12 DF,  p-value: 0.001921
```

The regression analysis gives the same results as the plot, and provides p-values and quantification of the effects.

- Temperature has a significant effect on yield, p=0.00039. High temperature increases yield by 4.8.
- Salt concentration has a near significant effect on yield, p=0.0696. For the purpose of future experiments, I would use the low concentration. Low salt concentration appears to increase yield by about 1.97.
- Buffer type does not have a significant effect on yield, p=0.71. If one of the buffers is cheaper or easier to work with, we might make the selection on that basis.

This experiment has provided a lot of information with a relatively small number of batches.

15.9 When may OVAT be preferable?

There are circumstances where OVAT may be preferable. As noted above, if the treatment assignments become confused (for example, by pipetting errors) the confusion may be detected more readily in an OVAT design. If combinations of variables may be dangerous or lethal, OVAT is preferable. In some cases, such as re-programming a robot to create a factorial layout, the logistics of implementing factorial design may be difficult. However, OVAT design should be selected when such concerns exist for your experiment, rather than by default.

15.10 Recommended YouTube videos

Kevin Dunn. "Process improvement using data"

See the attached Excel file "Factorial design recommended videos" for the video titles and duration.

15.11 Recommended free textbook PDF

A free PDF of the text "A First Course in Design and Analysis of Experiments" by Gary Oehlert is available at this website: http://users.stat.umn.edu/~gary/Book.html

16 Rescue failed experiments using fractional factorial designs

Have you ever had a failed experiment? I certainly have. All the measurements came out as 0, or the cells died, or the seeds didn't sprout, or even the positive controls came out negative.

How can you quickly figure out what is going wrong with the minimum amount of work? How can you quickly identify which factors are affecting the response? The answer is fractional factorial designs.

16.1 Example: How to bake good cookies: A fractional factorial experiment

As a familiar example, suppose we are baking cookies, but we keep ending up with a lot of burnt cookies, a lot of soggy undercooked cookies, or a lot of bad tasting cookies. We want to maximize the yield of good tasting cookies. How do we quickly find out what is going wrong?

We think of 7 factors that might be affecting the yield of good tasting cookies:

- Flour type (white or brown)
- Temperature in the oven (low or high)
- Baking soda amount (low or high)
- Time in the oven (short or long)
- Water amount (low or high)
- Sifting (sift or no)
- Butter among (low or high)

Here is a fractional factorial design to analyze cookie yield versus these 7 factors in only 8 batches of cookies.

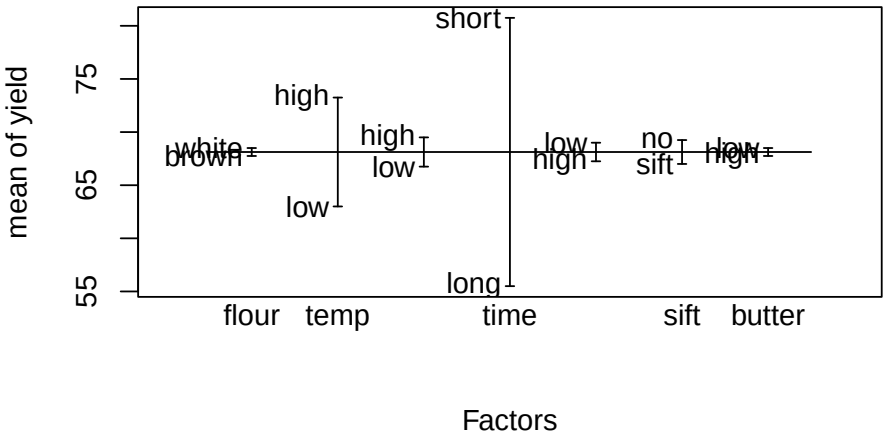
```
cookie.data = data.frame(flour =
c("brown","white","brown","white","brown","white","brown","white"),
temp = c("low","low","high","high","low","low","high","high"),
soda = c("low","low","low","low","high","high","high","high"),
time =
c("long","short","short","long","long","short","short","long"),
water =
c("high","low","high","low","low","high","low","high"),
sift = c("sift","sift","no","no","no","no","sift","sift"),
butter =
c("low","high","high","low","high","low","low","high"),
yield = c(47, 74, 84, 62, 53, 78, 87, 60))
cookie.data[1:7] = lapply(cookie.data[1:7], factor)
cookie.data
```

```
  flour temp soda  time water sift butter yield
1 brown  low  low  long  high sift    low    47
2 white  low  low short   low sift   high    74
3 brown high  low short  high   no   high    84
4 white high  low  long   low   no    low    62
5 brown  low high  long   low   no   high    53
6 white  low high short  high   no    low    78
7 brown high high short   low sift    low    87
8 white high high  long  high sift   high    60
```

Each row is a batch. The 7 columns flour, … butter, are the factors set to their high or low values we think may affect the yield. The last column is the yield of good cookies we get using the specified values for the factors for each batch. For example, for the combination of factors in row 1 (flour = brown, temp = low, etc.), the yield is 47 good cookies.

We can quickly see which factors are important by plotting the averages for each level of the factors with the R function `plot.design(cookie.data)`.

```
plot.design(cookie.data)
```



The graph that `plot.design(cookie.data)` produces shows each factor on the x-axis, and the average yield at each level of the factor on the y axis. The factor "time" has the biggest difference between settings: a yield near 80 for a short time in the oven versus a yield near 55 for a long time in the oven. From the graph we see that time has the largest effect on yield, followed by temperature. The other factors seem to have little effect on yield.

Notice in the cookie data that each level of each factor gets tested in 4 batches. For example, the variable temp is low in batches 1, 2, 5, and 6. It is high in the other 4 batches.

Batch 7, with 87 good cookies, and batch 3, with 84 good cookies, are the best. Which factor levels are giving us the high yields? We would like to know the average yield for each variable when it is at each level. One way to get this is with the R function `aggregate()`.

```
with(cookie.data, aggregate(yield ~ time, FUN = mean))
```

```
   time yield
1  long 55.50
2 short 80.75
```

The average yield with time long is 55.5 good cookies, while the average yield with time short is 80.75 good cookies. The difference is 80.75 - 55.5 = 25.25

good cookies. Apparently leaving them in the oven too long may be giving us a lot of burnt cookies.

We can get the difference between the levels for each factor using the `lm()` and `summary()` functions. (These give the results for an analysis of variance of the cookie data.)

```
lm.cookie = lm(yield ~ ., data = cookie.data)
summary(lm.cookie)
```

```

Call:
lm(formula = yield ~ ., data = cookie.data)

Residuals:
ALL 8 residuals are 0: no residual degrees of freedom!

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)    61.50        NaN     NaN      NaN
flourwhite      0.75        NaN     NaN      NaN
templow       -10.25        NaN     NaN      NaN
sodalow        -2.75        NaN     NaN      NaN
timeshort      25.25        NaN     NaN      NaN
waterlow        1.75        NaN     NaN      NaN
siftsift       -2.25        NaN     NaN      NaN
butterlow       0.75        NaN     NaN      NaN

Residual standard error: NaN on 0 degrees of freedom
Multiple R-squared:      1, Adjusted R-squared:    NaN
F-statistic:   NaN on 7 and 0 DF,  p-value: NA
```

The output from the `lm()` and `summary()` functions confirms what we saw in the graph, and gives us the quantitative differences: 25.25 for time, -10.25 for temperature, and values near 1 or 2 for the other factors.

Suppose for a moment that "sifting" was a labor-intensive or expensive step. This experiment would be evidence that the step could be eliminated with little effect on yield. The observed yield is actually 2.25 lower when sifting is done.

Suppose that butter was an expensive reagent. This experiment would suggest that the butter might be used with little effect on yield.

Consider now how much we have learned with only 8 batches:

- We have identified 2 important factors.
- We have evidence that the other factors have little effect on yield.
- We have evidence that a labor-intensive step might be eliminated.
- We have evidence that the amount of an expensive reagent could be reduced.

How many batches would it take to get this information if we did not use a fractional factorial design?

If we did one-variable-at-a-time (OVAT), we might do the following.

- Run 1 batch with white flour and 1 batch with brown flour = 2 batches for flour.
- Run 1 batch with temp low and 1 batch with temp high = 2 batches for temp.
- And so on for all 7 factors.

This OVAT method would require 7 factors * 2 batches per factor = 14 batches, compared to the 8 batches for the factorial experiment design. But OVAT has an even bigger weakness: each level of each factor is only examined in 1 batch. In contrast, in the factorial experiment design, each level of each factor is examined in 4 batches. These additional replicates give much more reliable estimates of the effects, and much greater power to detect statistical significance and get $p<0.05$ for the effect of a factor.

How do we get the p-values for the effect of the factors? For this example, the graph indicates that temperature and time are the important factors. We can re-run the `lm()` and `summary()` analyses using just these two factors, which will give us the p-values.

```
lm.cookie.2vars = lm(yield ~ time + temp, data = cookie.data)
summary(lm.cookie.2vars)
```

```

Call:
lm(formula = yield ~ time + temp, data = cookie.data)

Residuals:
     1      2      3      4      5      6      7      8
-3.375 -1.625 -1.875  1.375  2.625  2.375  1.125 -0.625

Coefficients:
```

```
            Estimate Std. Error t value Pr(>|t|)
(Intercept)   60.625      1.588   38.18 2.32e-07 ***
timeshort     25.250      1.834   13.77 3.63e-05 ***
templow      -10.250      1.834   -5.59  0.00253 **
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 2.593 on 5 degrees of freedom
Multiple R-squared:  0.9779,    Adjusted R-squared:  0.969
F-statistic: 110.4 on 2 and 5 DF,  p-value: 7.292e-05
```

The analysis shows that both time and temperature have significant effects ($p<0.05$).

We can do a test for an interaction between time and temperature in their effect on yield.

```
lm.cookie.2vars.interaction = lm(yield ~ time * temp, data =
cookie.data)
summary(lm.cookie.2vars.interaction)
```

```

Call:
lm(formula = yield ~ time * temp, data = cookie.data)

Residuals:
   1    2    3    4    5    6    7    8
-3.0 -2.0 -1.5  1.0  3.0  2.0  1.5 -1.0

Coefficients:
                 Estimate Std. Error t value Pr(>|t|)
(Intercept)        61.000      2.016  30.264  7.1e-06 ***
timeshort          24.500      2.850   8.595  0.00101 **
templow           -11.000      2.850  -3.859  0.01816 *
timeshort:templow   1.500      4.031   0.372  0.72869
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 2.85 on 4 degrees of freedom
Multiple R-squared:  0.9786,    Adjusted R-squared:  0.9626
F-statistic: 60.98 on 3 and 4 DF,  p-value: 0.0008523
```

In this case, the interaction test is not significant (p=0.73), so we can just use the analyses of the main effects.

16.2 Additional resources

The YouTube video "Experiments 4A – the trade-offs when doing half-fraction factorials" by Kevin Dunn introduces fractional factorial designs.

A free PDF of the text *A First Course in Design and Analysis of Experiments* by Gary Oehlert is available at http://users.stat.umn.edu/~gary/Book.html.

For examples of how to do the analyses using R, see the textbook *Design and Analysis of Experiments with R* by John Lawson (CRC Press), and the accompanying R package `daewr`.

17 The wrong way and the right way to get p < 0.05: multiple hypothesis testing

If you perform enough hypothesis tests, eventually one of them will give a p-value less than 0.05, even if the treatment has no effect. Specifically, about 1 in 20 (0.05) of the tests will give a "significant" p-value (<0.05) when all the null hypotheses are true.

This is called the "multiple testing" or "multiple comparison" problem. It is sometimes also called data dredging.

Journals in clinical medicine and regulatory agencies have become particularly sensitive to the problem of false positive results caused by testing multiple hypotheses. The standards for clinical journals and the Food and Drug Administration have become quite strict for dealing correctly with p-value adjustments that account for multiple comparisons. Reviewers for journals in biological research are not quite as strict, but more and more expect appropriate methods to avoid false positive results caused by multiple hypothesis testing.

For this chapter, I'll assume that you plan to perform a study with some number of primary and secondary analyses, and you want to use a method to control for multiple hypothesis testing that will be acceptable to journal reviewers.

Example: One of my venture capital clients called me with a question regarding a company he was interested in. In their PowerPoint slide deck, the company said that they had tested 100 proteins to see if any were differentially expressed in patients with a particular disease compared to healthy controls. They said that they had found 10 proteins that differed significantly between disease and healthy controls, with $p < 0.1$ (not $p < 0.05$).

If we require $p < 0.1$ for significance, and there is no difference in expression between disease and controls, we expect to see about $0.1 * 100 = 10$ proteins that were "significant". So, the results that the company was reporting were

exactly what we would expect if the proteins had no relationship at all to disease.

I spoke with the Chief Scientific Officer of the company. He agreed with my analysis and also agreed that their results could have occurred just due to chance. Their results were not in fact evidence of any association with disease status. He then said that they presented these results to potential clinical collaborators who could provide specimens and enroll patients for their clinical studies. The clinical collaborators had agreed to participate based on these results. So, the company had kept the results in the slide deck!

17.1 The probability of a false positive result with multiple comparisons

What is the probability of a false positive result when we do multiple comparisons?

P(at least one false positive result) = 1 - P(no false positive results)

For a single t-test, the probability of getting a false positive result is alpha = 0.05.

The probability of not getting a false positive result is 1 – alpha = 1 - 0.05 = 0.95.

If we do k = 2 t-tests, the probability of getting no false positives on any test is 0.95^k = 0.95^2.

P(at least one false positive result)

= 1 - P(zero false positive results)

= 1 – (1 - .05) ^ k

= 1 – (1 - .05) ^ 2

= 1 – (.95) ^ 2

= 0.0975

We've increased the risk of a false positive result from p = 0.05 for one test, to p = 0.0975 doing two tests.

The probability of getting at least one false positive (Type 1 error) when testing k hypotheses is

P(at least one false positive result)

= 1 - P(zero false positive results)

= 1 – (0.95) ^ k

What is the probability of at least one false positive (Type I error) when testing 5 hypotheses?

P(at least one false positive result)

= 1 - P(zero false positive results)

= 1 – (1 - .05) ^ k

= 1 – (1 - .05) ^ 5

= 1 – (.95) ^ 5

= 0.226

If we do 10 t-tests with alpha = 0.05, then

P(at least one false positive result)

= 1 – (.95) ^ 10

= 0.40

For a given target p-value (alpha, the Type I error rate) such as 0.05, testing k hypotheses, the probability of at least one false rejection is 1-(1-alpha)^k. As the table below shows, the probability of getting at least one false positive result rises quickly as we test more hypotheses.

17.2 Methods to adjust p-values to control for multiple hypothesis testing

Many methods have been developed to control for possible false positive results due to multiple testing. I won't try to cover them all, but instead will describe a few that are commonly used. The R package multcomp has extensive capabilities for multiple comparison analyses.

Bonferroni correction

The most conservative method is the Bonferroni correction. If you use Bonferroni, you won't get many false positives, but you will get a lot of false negative results. The Bonferroni method gives you very little power to detect

real relationships. Alternative methods are less conservative and give you more power.

To calculate the Bonferroni adjusted p-values, multiply each unadjusted p-value by the number of hypotheses tested, k, up to a maximum Bonferroni adjusted p-value of 1. If the Bonferroni adjusted p-value is still less than the original alpha (say, 0.05), then reject the null hypothesis.

Example of Bonferroni adjusted p-values:

Suppose that 3 hypotheses are tested.

Unadjusted p-values are:

p1 = 0.001

p2 = 0.021

p3 = 0.07

Bonferroni adjusted p-values are:

p1 = 0.001 * 3 = 0.003

p2 = 0.021 * 3 = 0.063

p3 = 0.07 * 3 = 0.21

Hypothesis 1, with adjusted p = 0.003, remains significant. Hypothesis 2, with adjusted p = 0.063, is no longer significant. Hypothesis 3, with adjusted p = 0.21, is not significant, with or without adjustment.

Holm's test

Holm's test is a more powerful alternative to the Bonferroni correction, so it may give more positive results, but still protects the specified alpha (typically 0.05).

For the Bonferroni correction, we apply the same critical value (alpha/k) to all the hypotheses simultaneously. It is called a "single-step" method.

The Holm's test is a stepwise method, also called a sequential rejection method, because it examines each hypothesis in an ordered sequence, and the decision to accept or reject the null depends on the results of the previous hypothesis tests.

The Holm's test is less conservative than the Bonferroni correction and is therefore more powerful. The Holm's test uses a stepwise procedure to

examine the ordered set of null hypotheses, beginning with the smallest P value, and continuing until it fails to reject a null hypothesis.

Here is an example of a Holm's test.

Suppose we have k = 3 t-tests. Assume target alpha(T) = 0.05.

Unadjusted p-values, sorted from smallest to largest, are:

p1 = 0.001 p2 = 0.021 p3 = 0.074

For the jth test, calculate alpha(j) = alpha(T)/(k – j +1)

For test j = 1,

alpha(j) = alpha(T)/(k – j +1)

= 0.05/(3 – 1 + 1)

= 0.05 / 3

= 0.0167

For test j=1, the observed p1 = 0.001 is less than alpha(j) = 0.0167, so we reject the null hypothesis.

For test j = 2,

alpha(j) = alpha(T)/(k – j +1)

= 0.05/(3 – 2 + 1)

= 0.05 / 2

= 0.025

For test j=2, the observed p2 = 0.021 is greater than alpha(j) = 0.025, so we reject the null hypothesis. Notice that this is different from the result using the Bonferroni correction.

For test j = 3,

alpha(j) = alpha(T)/(k – j +1)

= 0.05/(3 – 3 + 1)

= 0.05 / 1

= 0.05

For test j=3, the observed p2 = 0.074 is greater than alpha(j) = 0.05, so we do not reject the null hypothesis.

17.3 Hierarchical hypothesis testing

A different way to deal with the multiple hypothesis testing problem is to specify, before you begin the experiment or clinical trial, the order in which you will test hypotheses. Here is how it works.

- If the first hypothesis is significant, then test the second hypothesis.
- If the first and second hypotheses are significant, test the third hypothesis.
- Continue in the pre-specified order until you encounter the first non-significant result.
- As soon as you encounter a non-significant result, stop all further testing.

This method will protect the alpha at the specified level (such as 0.05) for all the hypotheses. However, you have to correctly guess the order in which to test the hypotheses. Suppose that your first hypothesis is not significant, with $p = 0.07$. Even if a later hypothesis has $p < 0.05$, you still cannot claim it is significant, because of using the hierarchical method to control for false positives.

17.4 False discovery rate: an alternative to controlling false positives

None of the standard methods for controlling false positives is likely to yield satisfactory results if you are testing a large number of hypotheses. For example, suppose you are trying to determine if any of several hundred genes, proteins or microRNAs are differentially expressed in disease. You can't pre-specify an order of hierarchical hypothesis testing if you don't know anything about their effects. Using a Bonferroni correction on hundreds or potentially thousands of genes or proteins will kill your p-values. An alternative is to estimate the False Discovery Rate (FDR). The FDR is the proportion of positive (significant) results that are likely to be false positives. It is much more liberal than controlling alpha for false positives.

See the chapter on False Discovery Rate (FDR) in the companion book for more explanation and examples.

18 Is P < 0.05 the right goal?

There has been considerable discussion and debate regarding the use and misuse of p-values and the $p < 0.05$ criterion. While p-values will likely continue to be requirements for most scientific publications and regulatory reviews, it is worth considering when p-values are or are not appropriate, what can be done to improve their use, and what alternatives and additional methods could be used to improve the quality and reproducibility of scientific research.

The American Statistical Association (ASA) published a statement on p-values that outlined many of the issues (Wasserstein and Lazar 2016):

Ronald L. Wasserstein & Nicole A. Lazar (2016) The ASA Statement on p-Values: Context, Process, and Purpose, The American Statistician, 70:2, 129-133, DOI: 10.1080/00031305.2016.1154108

https://doi.org/10.1080/00031305.2016.1154108

https://www.amstat.org/asa/files/pdfs/p-valuestatement.pdf and

https://amstat.tandfonline.com/doi/full/10.1080/00031305.2016.1154108#.Vt2XIOaE2M

The statement describes some of the background:

"The ASA Board was also stimulated by highly visible discussions over the last few years. For example, ScienceNews (Siegfried 2010) wrote: "It's science's dirtiest secret: The 'scientific method' of testing hypotheses by statistical analysis stands on a flimsy foundation." A November 2013, article in Phys.org Science News Wire (2013) cited "numerous deep flaws" in null hypothesis significance testing. A ScienceNews article (Siegfried 2014) on February 7, 2014, said "statistical techniques for testing hypotheses …have more flaws than Facebook's privacy policies." A week later, statistician and "Simply Statistics" blogger Jeff Leek responded. "The problem is not that people use P-values poorly," Leek wrote, "it is that the vast majority of data analysis is not performed by people properly trained to perform data analysis" (Leek 2014)."

The ASA statement proposed the following six principles.

- P-values can indicate how incompatible the data are with a specified statistical model.
- P-values do not measure the probability that the studied hypothesis is true, or the probability that the data were produced by random chance alone.
- Scientific conclusions and business or policy decisions should not be based only on whether a p-value passes a specific threshold.
- Proper inference requires full reporting and transparency.
- A p-value, or statistical significance, does not measure the size of an effect or the importance of a result.
- By itself, a p-value does not provide a good measure of evidence regarding a model or hypothesis.

The statement includes explanations and discussion of these principles. In addition, the supplementary material includes 21 letters from statisticians and researchers commenting on the strengths, weaknesses, and inadequacies of the ASA statement and the principles.

Whether or not one believes that p-values or $p < 0.05$ are appropriate tools, it remains desirable to reduce unexplained variability, reduce sample size, increase power, and deal with multiple hypothesis testing, among other issues.

19 What if you still get p > 0.05?

What do you do if you've tried all these methods and still get p > 0.05? There are several possibilities to consider.

19.1 A possibly false negative result

Here are a few potential causes of a false negative result and action you might take.

Random variation due to sampling

In the chapter, "Increase sample size last. Reduce unexplained variability first", there are simulations that show how much a p-value can vary just by taking another random sample from the same population. We saw that, even with just 4 samples, the p-values ranged from 0.028 to 0.25. P-values go up or down by an order of magnitude just as a result of taking another sample. This is one reason why it is so important to do a power and sample size calculation, to give yourself a good chance of getting p < 0.05.

Observations that appear to be wrong or inconsistent: what is the cause?

A few observations that appear to be wrong or inconsistent can give rise to non-significant p-values. In work with my clients, I have found that such anomalous observations can be caused by an operator who is new and not sufficiently trained, by specimens that were shipped at temperatures that caused the sample to degrade, by particular sites that don't collect, store, and process specimens correctly, by particular instruments that have not been calibrated, by reagent lots that have expired or gone bad, and similar problems related to the process but not related to the treatment. If you keep a record of such information for each specimen you run, you can see if any such process variables are correlated with the anomalous observations. Specifically, you can regress your response (y) variable against each of these process variables to see if any of them consistently correlate with the anomalous observations.

A variation on this method is to create a new variable in which you classify each observation as Anomalous yes or no. Then use this "Anomalous yes or no" variable as the response variable in a logistic regression, where the explanatory variables in the logistic regression are treatment and each of the process variables for which you have information. In the regression models you usually want to either (i) include treatment group as an explanatory variable, or (ii) do the analyses separately for each treatment group. I suggest including treatment group as an explanatory variable, as this avoids reducing the number of observations in the analysis.

Non-inferiority rather than superiority

Particularly in medical studies, if a new intervention has fewer safety problems than an existing intervention, it may be sufficient to show that the new intervention is not worse than the existing intervention with respect to the efficacy. In a biological setting, it may be scientifically interesting, for example, to show that the effect of one genetic mutation is not less than the effect of another genetic mutation. In these situations, an alternative to a formal statistical test for difference is a formal statistical test for non-inferiority. The sample size required for non-inferiority studies depends on the allowable margin – how much worse can the new treatment be and still be considered non-inferior? Unfortunately, unless the non-inferiority margin is large, the sample size required to show non-inferiority can be quite large.

The treatment effect is in the extremes rather than in the means: quantile regression.

In some cases, the effect of a treatment is not to create a difference between the means or medians of two groups, but rather to create a difference in the more extreme values. For example, suppose an intervention is designed to prevent pre-term births. As pre-term births are generally less than 10% of births, the intervention may have little effect on the mean or median gestational age at birth. But an intervention might have a large effect on the shortest 5% of gestational age at birth (premature babies). In such situations, one method of analysis is to test for a difference only among the subgroup of babies who make up the shortest 5% of gestational ages at birth. This can be done by explicitly separating out the subjects in the shortest 5% in each treatment group. An alternative method is quantile regression, which tests for differences at different quantiles (percentiles) of the response variable, where the quantiles could be the 5th percentile, the 50th percentile (median) or another percentile of interest. See the R package quantreg for examples and code.

19.2 A true negative result

It may be that the treatment really doesn't have the effect size that you hoped for or doesn't have any effect at all. If that is true, it can still be helpful to other researchers if you include this information in your final report:

- Show that the sample size and power were sufficient to detect a scientifically or medically interesting effect size. Surprisingly, many published "negative" results actually have insufficient sample size and power. For example, studies sometimes include only those patients available at the particular clinic where the physician/investigator works and are underpowered right from the start.
- Describe the covariates that you included in the model, along with their effect sizes, p-values, and standard deviation within treatment groups from your analyses. Reporting the standard deviation within treatment groups can be quite helpful to other researchers working in your field when they need to do power and sample size calculations.
- Report the adjusted R-squared, which is the proportion of variance in the response that is explained by the model. The adjusted R-squared is between 0 and 1. If it is much below 1, it indicates that there is a lot of unexplained variability, and that future work would likely benefit from trying to identify and reduce those sources of variability.
- Describe alternative analyses that you performed, such as rank-based analyses, analyses excluding highly correlated covariates, tests for interactions, or other analyses that demonstrate a thorough examination of the data.

After that, it may be time to move on to another research question.

20 Don't know R? Use Claude to run these analyses for you

I've used R for the examples in this book because R is free, powerful, and what I use most in my own consulting work. But most biologists I work with don't know R. They know Excel, and may know a little Prism. Learning R well enough to run a two-way ANOVA, a Cox proportional hazards model, or a rank-based regression is, realistically, a multi-month investment that most bench scientists don't have time for.

You no longer need to make that investment to use the methods in this book. You can describe your experiment and your data, in plain English, to Claude (an AI assistant available at claude.ai), and ask it to perform the analysis for you. It will write the R code, run it, and explain the result. If you ask, it will give the same kind of plain-English interpretation I've given throughout this book. While other AI/LLM products have similar capabilities, I have, to date, found that Claude gives the best results.

This chapter is not a replacement for the rest of the book. It's the opposite: knowing what these methods are called and when to use them is exactly what lets you ask Claude for the right analysis instead of the easiest one. An AI assistant will happily run a t-test on your data if you ask for a t-test. Better if you know to ask for two-way ANOVA instead.

20.1 A short protocol for getting good results

Treat Claude as a fast, well-read collaborator who writes the code for you and explains the result. Here are some suggestions to make the results more reliable and more useful.

- Give Claude your actual data if you can. If you haven't collected the data yet, give it an Excel file with examples of the data you plan to collect. You can attach a CSV file, an Excel file, or even a Prism export directly in your conversation. Claude can read it and use the

real values, rather than you having to retype numbers (and possibly introduce errors).

- Describe your experimental design in plain language: what you measured, what the treatment groups are, whether there are covariates like age or sex, and whether observations are paired, repeated, or independent. This is the information a statistician would ask for, and it's the information Claude needs to recommend the right method.
- Name the method if you know it. If you've read this book, you often will. "Run a two-way ANOVA" or "use Rfit for rank-based regression" gets you a faster, more targeted answer than a vague request.
- If you're not sure which method applies ask Claude to describe alternative methods, recommend one and explain why — citing the kind of reasoning used in this book (data type, number of explanatory variables, outliers, sample size).
- Always ask Clause to produce an R script file with all the required R code and the full output, not just a conclusion. This keeps the analysis transparent: you, a labmate, a statistician, or a journal reviewer can rerun the exact code later and get the exact same answer. If you report the interpretation but don't have the R code, you may find the analysis is not reproducible, which can be rather embarrassing.
- Ask follow-up questions. A conversation, unlike a menu of canned tests in point-and-click software, lets you push back, ask "what if," and request a different method if the first one doesn't fit. Use that.

20.2 A complete example, start to finish

Here is roughly how a conversation might go, using the mouse weight example from the first chapter.

> **Example query:** "I have weight measurements (in grams) from 12 mice: 6 wild-type and 6 mutant, with both males and females in each group. I've attached the data as a CSV with columns Treatment, Sex, and Grams. A t-test on Treatment alone wasn't significant, but I suspect Sex is adding unexplained variability. Can you run a two-way ANOVA testing for the effect of Treatment, controlling for Sex, show me the R code and full output, and tell me in plain English whether Treatment is significant?"

Claude would read the attached file, write R code using `lm()` with the formula `Grams ~ Sex + Treatment` (the same approach used in the first chapter), run it, and return both the code and the output table. It would then explain, in plain English, that Sex is a significant source of variability, and that once it's controlled for, Treatment is also significant — the same conclusion reached earlier in the book, $p = 0.0239$. You could then ask follow-up questions, such as:

> **Example query:** "Can you show me a boxplot of weight by treatment, separately for each sex, so I can see this visually?"
>
> **Example query:** "Does the result hold up if I check the assumptions of ANOVA? Please check the residuals for normality and equal variance and tell me if you see any problems."

Each of these is a single, plain-English sentence. None of them requires you to know what `lm()`, `residuals()`, or `shapiro.test()` are — though Claude will tell you, if you ask.

20.3 Example queries for the methods in this book

Below are example queries for the topic of each chapter. Adapt the wording to your own experiment; what matters is including the same information a statistician would need: what you measured, how the groups are defined, and what else might be affecting the response.

20.3.1 Control for variables with multiple regression or ANOVA

> **Example query:** "I'm comparing a response between two treatment groups, but I also have sex and age recorded for each subject, and I suspect they affect the response too. Can you run a multiple regression or ANOVA that controls for sex and age while testing for the treatment effect?"

20.3.2 Power and sample size

> **Example query:** "I'm planning an experiment comparing two groups. Based on a pilot study, I expect a difference of about 4 units with a standard deviation of 2 within each group. How

many animals do I need per group for 80% power at alpha = 0.05? Please show the calculation."

20.3.3 Reduce variability before increasing sample size

Example query: "My pilot data have a standard deviation of 6 within each group, and I need an impractically large sample size to reach 80% power. Can you help me think through what sources of variability I might be able to control for or remove, and recalculate the required sample size if I could cut the standard deviation in half?"

20.3.4 Robust methods for data with outliers

Example query: "My data look mostly normal but I have two or three extreme outliers I can't justify removing. Can you run a robust regression using RobStatTM that reduces the influence of those points, alongside an ordinary regression, so I can compare the two?"

20.3.5 Rank-based methods (Wilcoxon, Kruskal-Wallis, Rfit)

Example query: "My data clearly aren't normally distributed (please check and show me a histogram). Can you run a Wilcoxon rank-sum test comparing the two groups, and also a rank-based regression using Rfit if I have more than one explanatory variable?"

20.3.6 Survival analysis

Example query: "I have time-to-event data with some animals censored (they were still alive at the end of the study). Can you run a Cox proportional hazards regression testing for a treatment effect, controlling for [your covariates], and show me the hazard ratio with a confidence interval?"

20.3.7 Count data

Example query: "My response variable is a count (number of pups per litter, including zeros). A t-test doesn't seem appropriate. Can you run a negative binomial regression and also a Wilcoxon test, and tell me which seems more appropriate for these data and why?"

20.3.8 Don't categorize a quantitative variable

Example query: "I was planning to split my continuous outcome into 'responders' and 'non-responders' at some cutoff and run a chi-squared test. Before I do, can you compare the power of that approach to analyzing the continuous outcome directly, using my actual data?"

20.3.9 CMH tests for ordered categories

Example query: "I have a contingency table where both the treatment groups and the outcome categories are ordered (for example, dose: low/medium/high, and response: none/partial/complete). A plain chi-squared test ignores that ordering. Can you run a Cochran-Mantel-Haenszel test that accounts for it?"

20.3.10 Balancing covariates is not enough

Example query: "My treatment and control groups look balanced on age and weight at baseline — similar means in each group. Does that mean I don't need to adjust for them in the analysis, or should I include them as covariates anyway? Can you show me the analysis both ways?"

20.3.11 Correlated explanatory variables

Example query: "I have several explanatory variables I'd like to include in a regression, but I suspect some of them are correlated with each other. Can you check for multicollinearity (e.g., variance inflation factors) before I interpret the regression coefficients?"

20.3.12 Interactions between subgroups and treatment

Example query: "I want to check whether the treatment effect is different in males versus females (or in any other subgroup I specify), not just whether there's an overall effect. Can you test for a treatment-by-sex interaction and explain what it would mean if it's significant?"

20.3.13 Two primary analyses instead of one

Example query: "I have two outcomes (or two genes, or two endpoints) that I'd be happy to find an effect on either one. Instead of picking one as primary, can you help me set up two co-primary analyses, each tested at alpha = 0.025, and tell me what sample size I need?"

20.3.14 Multiple hypothesis testing, done correctly

Example query: "I'm testing [number] hypotheses in this experiment. Can you apply an appropriate correction for multiple comparisons — Bonferroni, Holm's, or false discovery rate, whichever you think fits best — and explain the difference so I can decide which to report?"

20.3.15 When p > 0.05, and what p-values do and don't mean

Example query: "My result wasn't significant. Before I conclude there's no effect, can you check whether my study actually had enough power to detect an effect of the size I care about? And can you suggest what else might explain a non-significant result besides 'no true effect'?"

20.4 Asking for model checking and diagnostics

A result is only as trustworthy as the assumptions behind it. You don't need to know the names of these assumptions to ask Claude to check them — just ask.

Example query: "Please check whether the assumptions of this ANOVA are reasonably met — normality of residuals and equal variance across groups — and show me the diagnostic plots."

Example query: "Are there any influential outliers in this regression that are having an outsized effect on the result? Can you identify them and show me what happens to the conclusion if I exclude them?"

Example query: "For this Cox regression, can you check the proportional hazards assumption and tell me whether it's reasonably satisfied?"

Example query: "For this negative binomial model, is there evidence of overdispersion that the model isn't accounting for?"

If an assumption looks shaky, ask Claude what it would recommend instead — a transformation, a robust or rank-based method from this book, or a different model entirely.

20.5 Asking for alternative analyses and sensitivity checks

One of the most useful things you can do with a conversational tool, that you can't easily do in Prism, is ask "what if" questions.

Example query: "How would the conclusion change if I used a rank-based method instead of ordinary ANOVA?"

Example query: "Does the result still hold if I exclude the one animal that looked sick during the study?"

Example query: "I have three reasonable ways to define my outcome variable. Can you run the analysis all three ways and show me whether the conclusion is consistent?"

Example query: "If I had used a t-test instead of controlling for [covariate], would I have reached a different conclusion? Can you show me both?"

Running several reasonable analyses and checking whether they agree is good scientific practice — provided you decide which analysis is primary before you see the results, and report the others as sensitivity analyses rather than quietly picking whichever one gives you $p < 0.05$. See the chapter on multiple hypothesis testing for why that distinction matters.

20.6 Asking for a plain-English interpretation

This is where an AI assistant is particularly useful for biologists who need to communicate results to people who aren't statisticians — including, often, themselves on a Friday afternoon.

> **Example query:** "Can you explain this result in plain English, the way you'd explain it to a biologist who doesn't know statistics?"
>
> **Example query:** "Can you write one or two sentences I could use in a paper's results section to describe this analysis and its outcome?"
>
> **Example query:** "My PI asked me to summarize this in the lab meeting tomorrow. Can you give me three bullet points a non-statistician would understand?"
>
> **Example query:** "Does this result mean the treatment doesn't work, or just that we don't have enough evidence either way? Please explain the difference."

That last question matters. As discussed earlier in this book, a non-significant p-value is not proof of no effect, and a good explanation should say so.

20.7 An AI assistant is a collaborator, not an oracle

Everything in this chapter comes with the same caveat that applies to any statistical tool, including Prism, SAS, or a hand calculation: it is only as good as the question you ask and the care you take afterward.

- Always look at the R code, not just the conclusion. If you don't understand a line, ask. "What does this line of code do?" is a perfectly good follow-up question, and a good way to actually learn R over time.

- Save the code and the output. This is your record of exactly what was done, which you (or anyone else) can rerun later and get the same answer.
- Decide your analysis plan before you look at your data, not after. An AI assistant makes it easy to try many different analyses quickly, which is wonderful for honest sensitivity analysis and dangerous for the kind of data dredging described earlier in this book. The methods in this book are about choosing more powerful, more appropriate analyses — not about trying approaches until one of them gives you $p < 0.05$. Decide what you're testing and how, in writing, before you see the results.
- For anything going into a regulatory submission, a clinical study, or a high-stakes publication, have a statistician review the analysis, just as you would review any other critical result. An AI assistant can make that statistician's time more efficient — by handling the routine coding and giving you a first-pass understanding — but it does not replace their judgment, particularly for unusual study designs.
- Mistakes are possible, especially for complex or unusual designs that don't closely resemble the worked examples in this book. Treat surprising results with the same skepticism you'd apply to a surprising result from any new piece of software: check it, and if it matters, check it again a different way.

Used this way, an AI assistant doesn't replace the thinking this book has asked of you. It removes the R syntax as a barrier between that thinking and the right analysis.

References

Box, George E. P. 2006. *Improving Almost Anything: Ideas and Essays*. Revised. John Wiley & Sons.

Brunner, Edgar, Arne C. Bathke, and Frank Konietschke. 2018. *Rank and Pseudo-Rank Procedures for Independent Observations in Factorial Designs: Using r and SAS*. Springer Series in Statistics. Springer.

Chow, Shein-Chung, Jun Shao, Hansheng Wang, and Yuliya Lokhnygina. 2017. *Sample Size Calculations in Clinical Research*. 3rd ed. Chapman & Hall/CRC Biostatistics Series. Chapman; Hall/CRC.

Hettmansperger, Thomas P., and Joseph W. McKean. 2011. *Robust Nonparametric Statistical Methods*. 2nd ed. CRC Press.

Jaeckel, Louis A. 1972. "Estimating Regression Coefficients by Minimizing the Dispersion of the Residuals." *Annals of Mathematical Statistics* 43: 1449–58.

Kloke, John D., and Joseph W. McKean. 2012. "Rfit: Rank-Based Estimation for Linear Models." *The R Journal* 4 (2): 57–64.

Kloke, John, and Joseph W. McKean. 2015. *Nonparametric Statistical Methods Using r*. Chapman; Hall/CRC.

Maronna, Ricardo A., R. Douglas Martin, Victor J. Yohai, and Matías Salibián-Barrera. 2018. *Robust Statistics: Theory and Methods (with r)*. 2nd ed. Wiley.

Serdar, Cemil Colak, Murat Cihan, Dilara Yücel, and Muhittin Abdulkadir Serdar. 2021. "Sample Size, Power and Effect Size Revisited: Simplified and Practical Approaches in Pre-Clinical, Clinical and Laboratory Studies." *Biochemia Medica* 31 (1): 010502. https://doi.org/10.11613/BM.2021.010502.

Wasserstein, Ronald L., and Nicole A. Lazar. 2016. "The ASA Statement on p-Values: Context, Process, and Purpose." *The American Statistician* 70 (2): 129–33. https://doi.org/10.1080/00031305.2016.1154108.

Wilcox, Rand R. 2022. *Introduction to Robust Estimation and Hypothesis Testing*. 5th ed. Statistical Modeling and Decision Science. Academic Press.